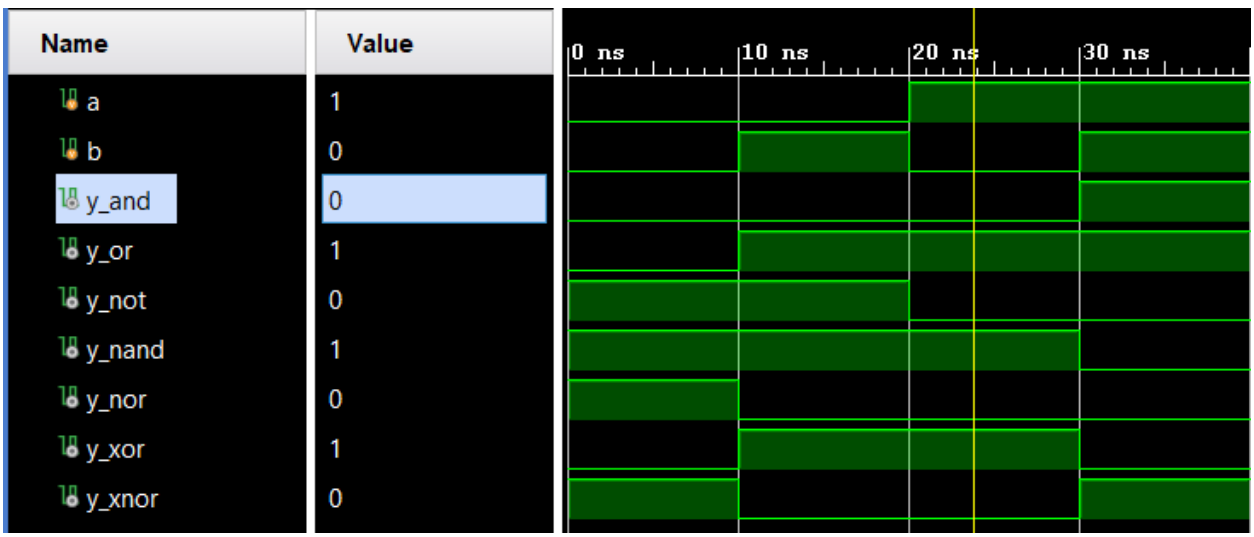
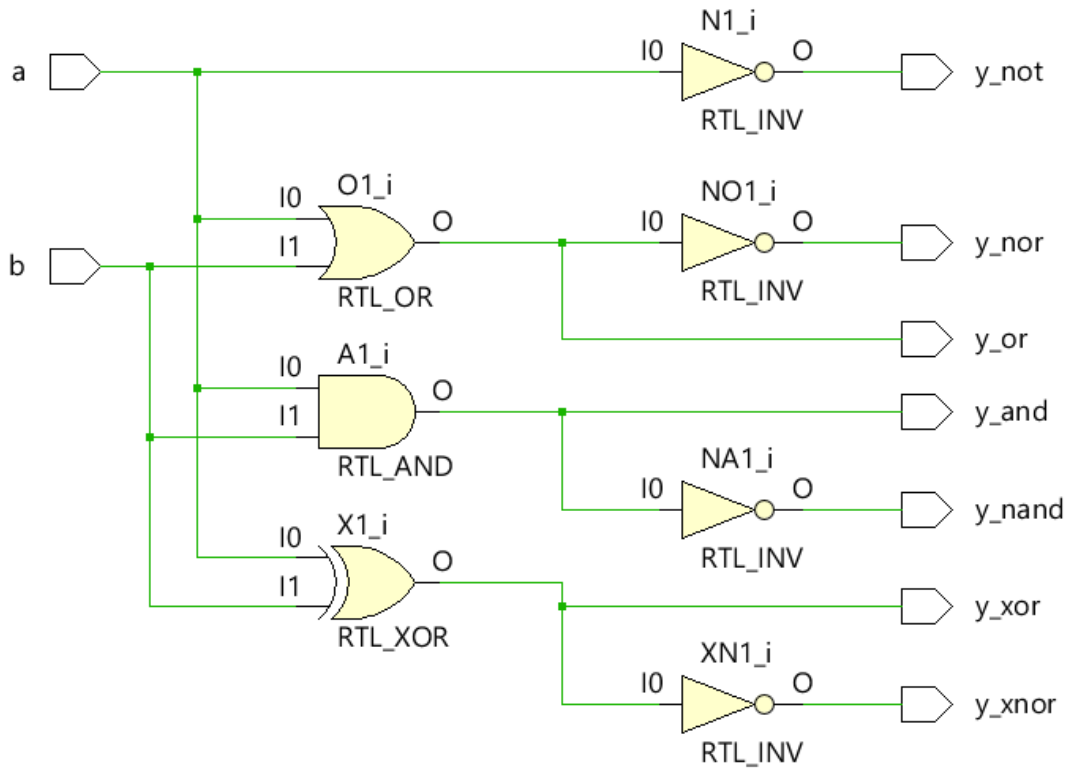
BASIC GATES

GATE LEVEL

```
module all_gates_gate(a,b,y_and,y_or,y_not,y_nand,y_nor,y_xor,y_xnor);
input a,b;
output y_and,y_or,y_not,y_nand,y_nor,y_xor,y_xnor;
and A1(y_and,a,b);or O1(y_or,a,b);
not N1(y_not,a);
nand NA1(y_nand,a,b);
nor NO1(y_nor,a,b);
xor X1(y_xor,a,b);
xnor XN1(y_xnor,a,b);
endmodule
```

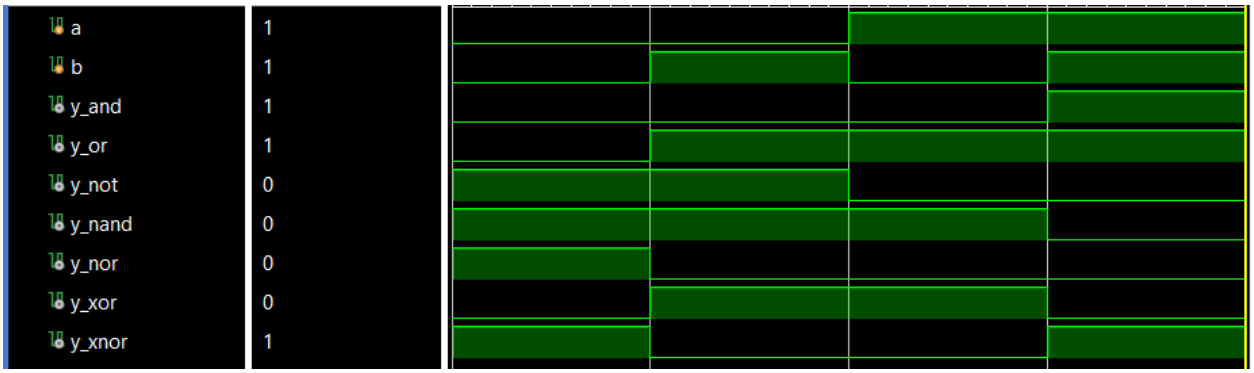


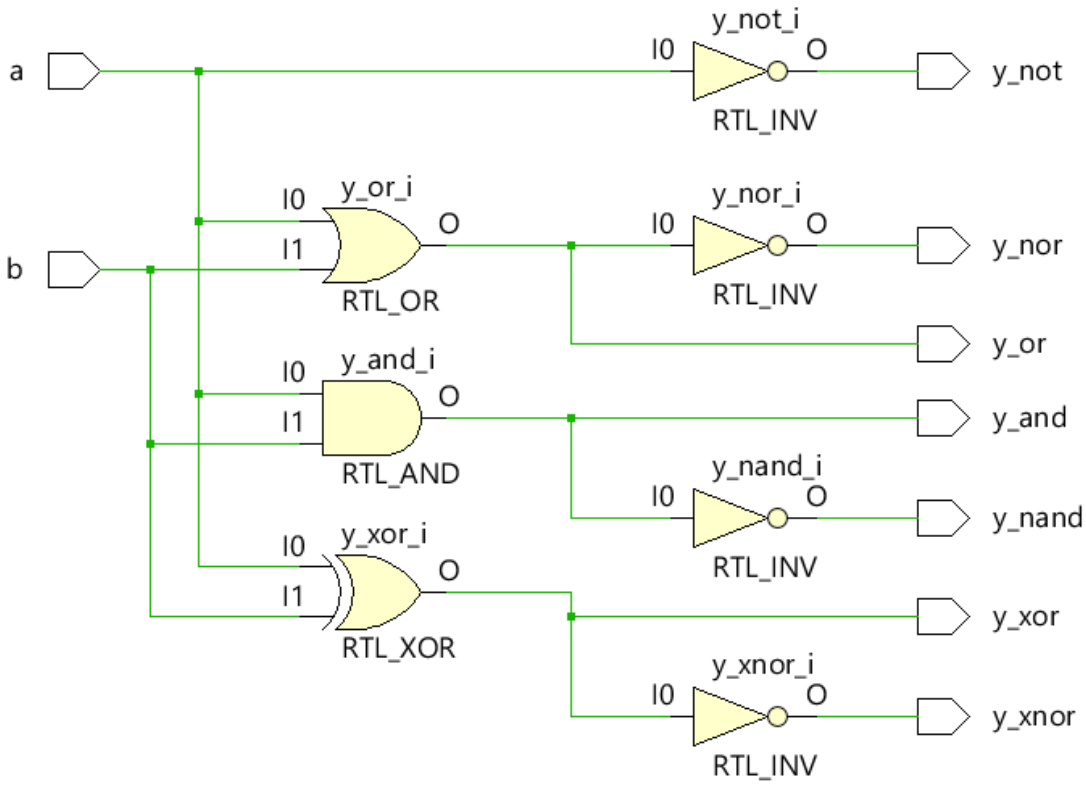
SCHEMATIC



DATA FLOW

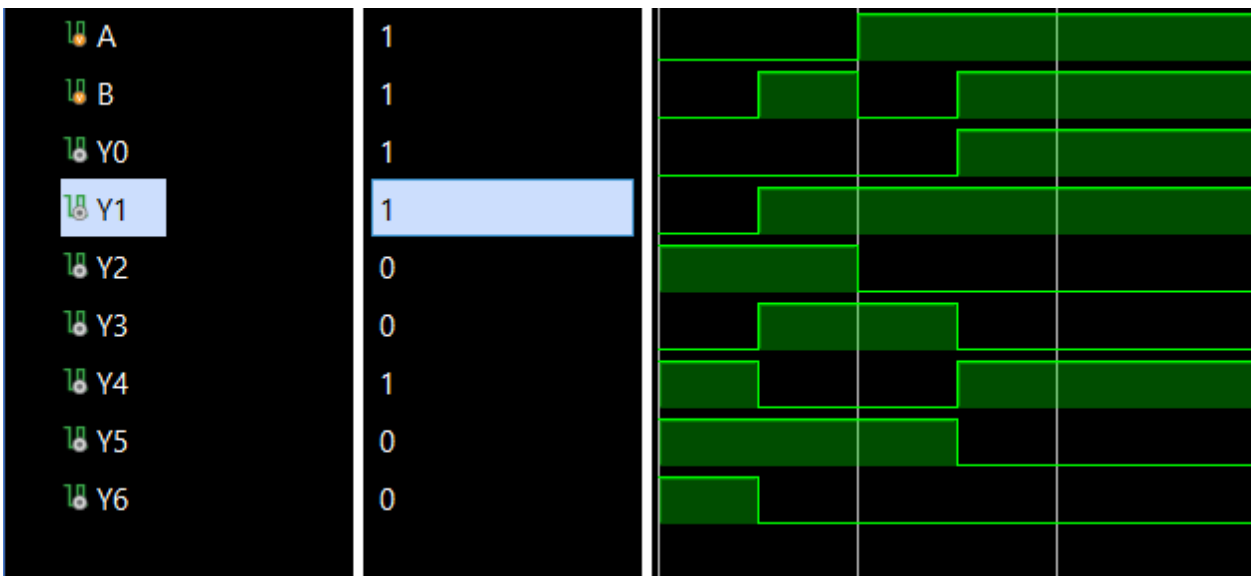
```
module all_gates_dataflow(a,b,y_and,y_or,y_not,y_nand,y_nor,y_xor,y_xnor);
input a,b;
output y_and,y_or,y_not,y_nand,y_nor,y_xor,y_xnor;
assign y_and=a&b;
assign y_or=a|b;
assign y_not=~a;
assign y_nand=~(a&b);
assign y_nor=~(a|b);
assign y_xor=a^b;
assign y_xnor=~(a^b);
endmodule
```

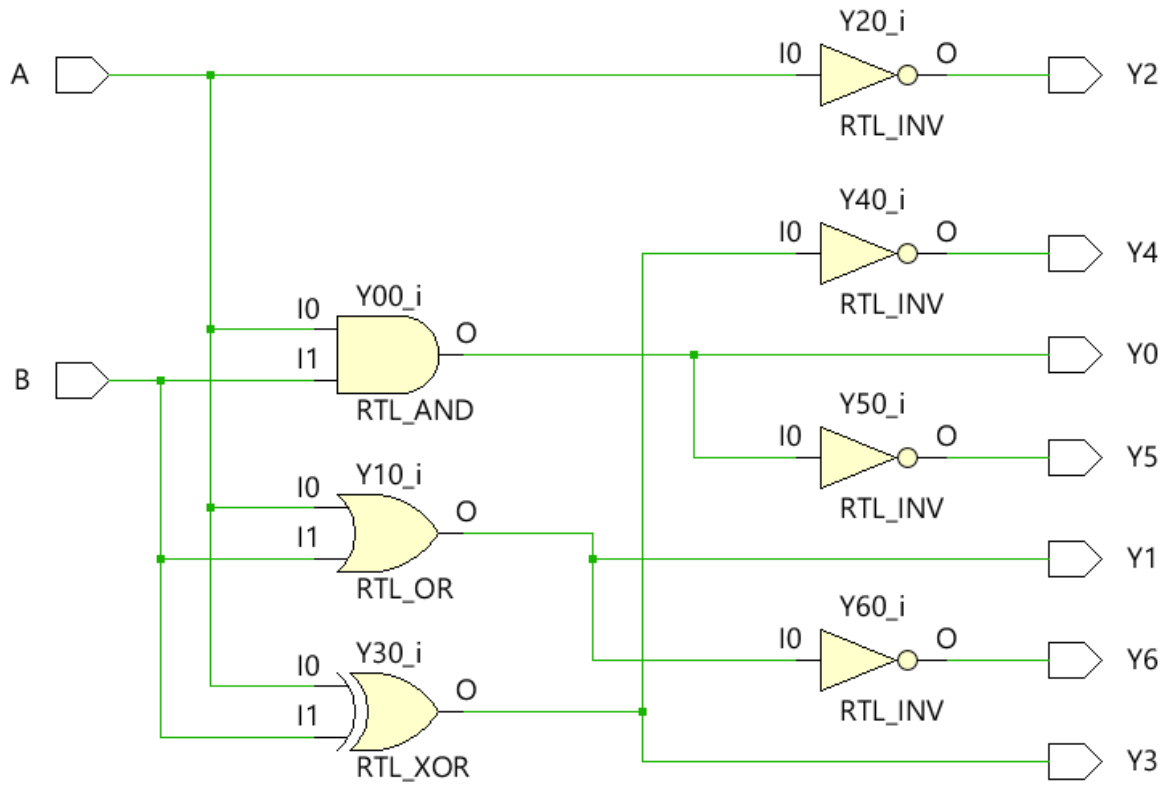




BEHAVIOURAL

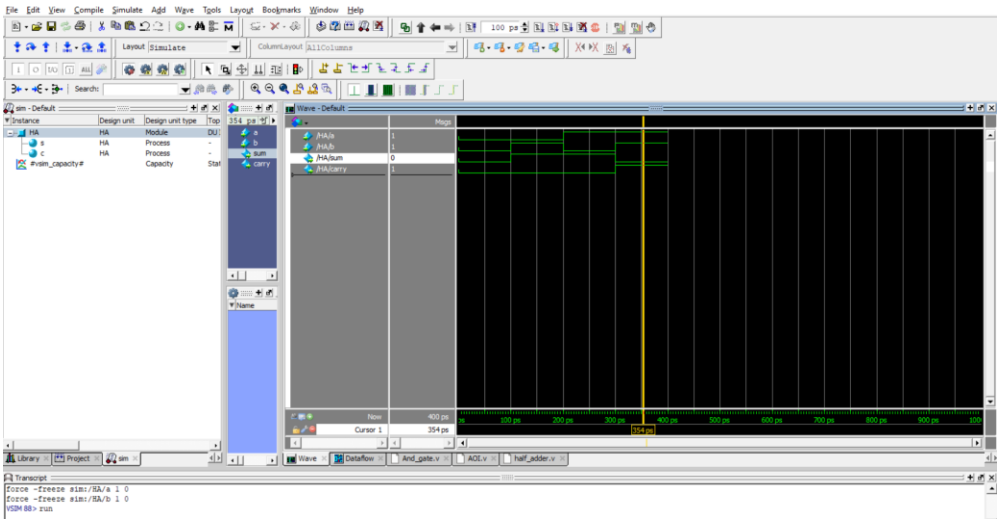
```
module Basic_gates(A,B,Y0,Y1,Y2,Y3,Y4,Y5,Y6);
input A,B;
output reg Y0,Y1,Y2,Y3,Y4,Y5,Y6;
always @(*)
begin
Y0=A&B;
Y1=A|B;
Y2=~A;
Y3=A^B;
Y4=~(A^B);
Y5=~(A&B);
Y6=~(A|B);
end
endmodule
```




A
B
Y20_i
I0
O
RTL_INV
Y2
Y40_i
I0
O
RTL_INV
Y4
Y00_i
I0
I1
O
RTL_AND
Y0
Y50_i
I0
O
RTL_INV
Y5
Y10_i
I0
I1
O
RTL_OR
Y1
Y60_i
I0
O
RTL_INV
Y6
Y30_i
I0
I1
O
RTL_XOR
Y3

HALF ADDER

GATE LOGIC



```
module HA(sum,carry,a,b);
input a,b;
output sum,carry;
xor s(sum,a,b);
and c(carry,a,b);
endmodule
```

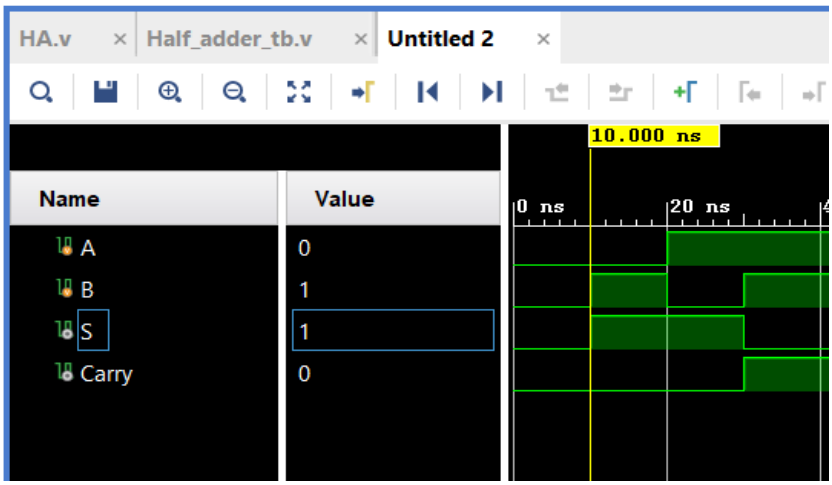
DATA LOGIC

```
module HA_datalogic(input A,B, output SUM,CARRY);
begin
assign SUM=A^B;
assign CARRY=A*B;
end
endmodule
```

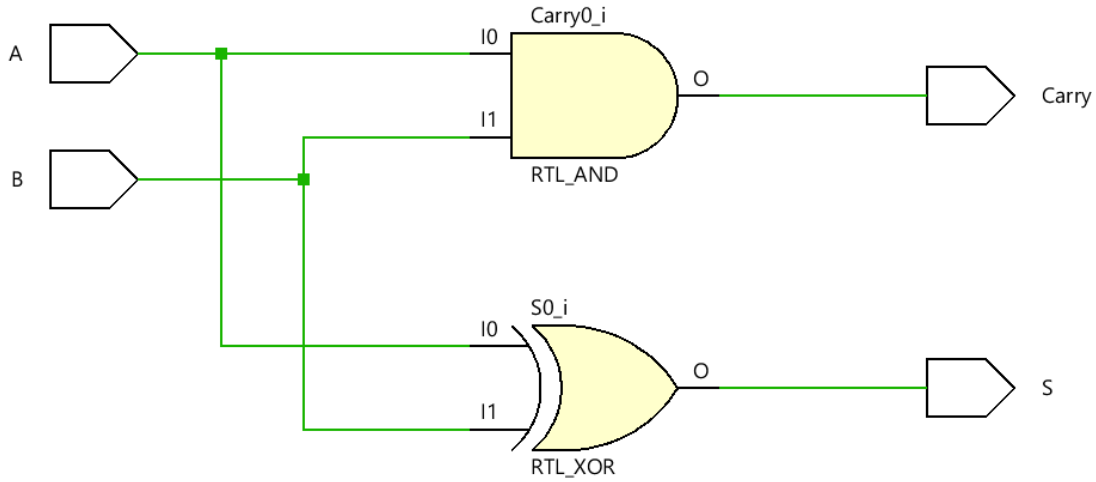
BEHAVIOUR LOGIC

```
module HA(input A,B, output reg S,Carry);
always@(*)
begin
S=A^B;
Carry=A&B;
end
endmodule
```

OUTPUT



SCHEMATIC

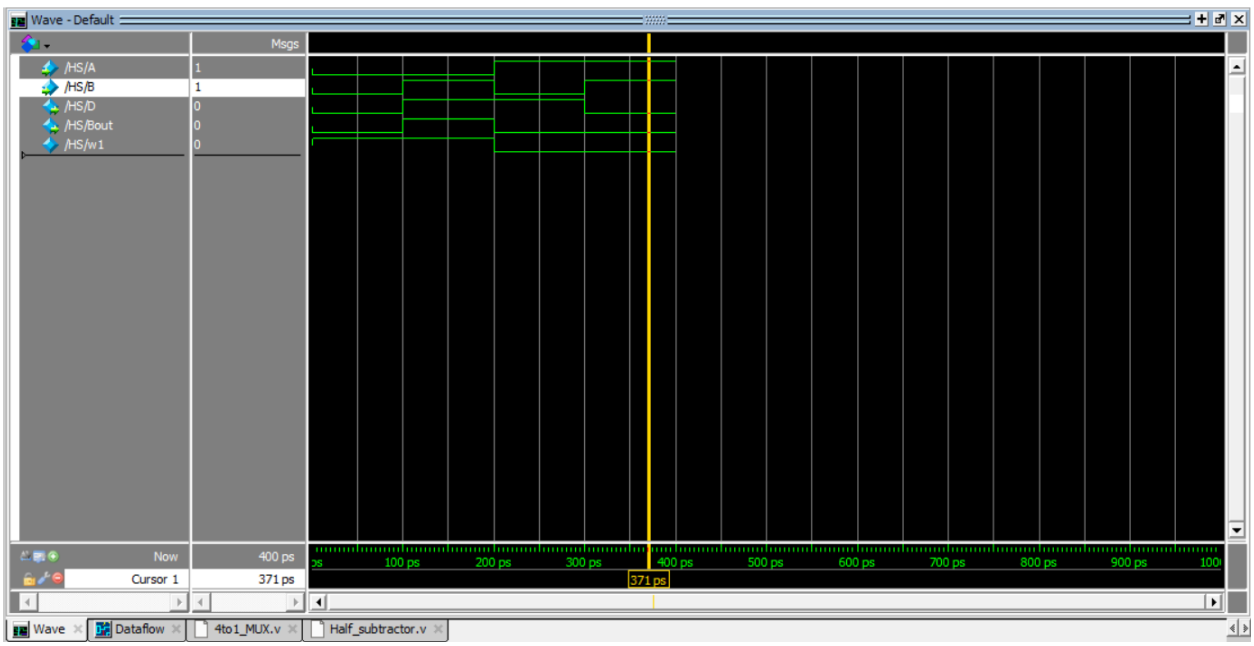


HALF SUBTRACTOR

GATE LOGIC

D:/Half_subtractor.v - Default

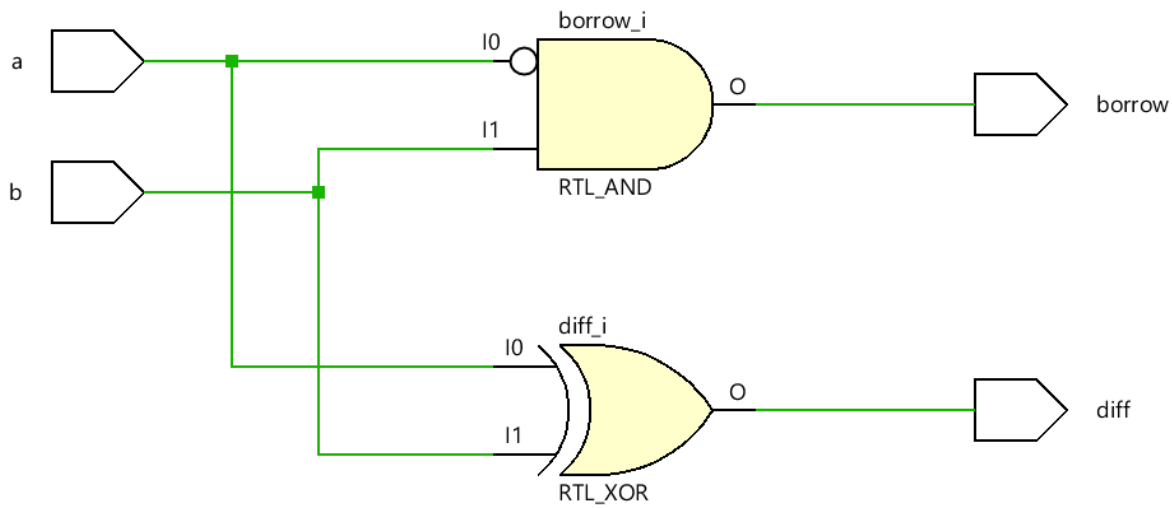
```
module HS(D,Bout,A,B);
input A,B;
output D,Bout;
wire w1;
xor A1(D,A,B);
not A2(w1,A);
and A3(Bout,w1,B);
endmodule
```

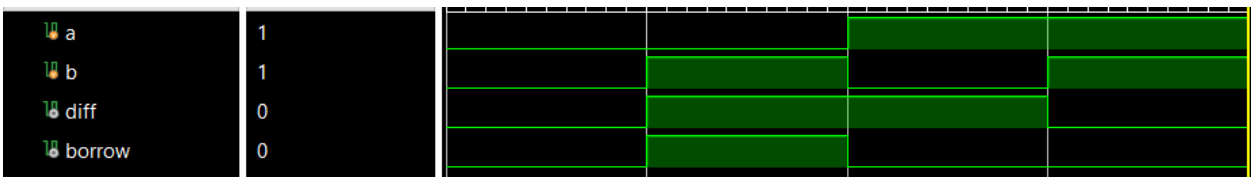


DATA FLOW

```
module half_subtractor_dataflow(a,b,diff,borrow);
input a,b;
output diff,borrow;
assign diff=a^b;
assign borrow=(~a)&b;
endmodule
```

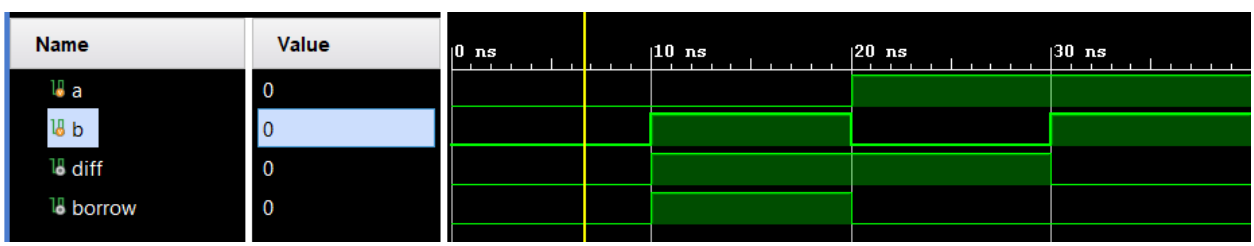
SCHEMATIC



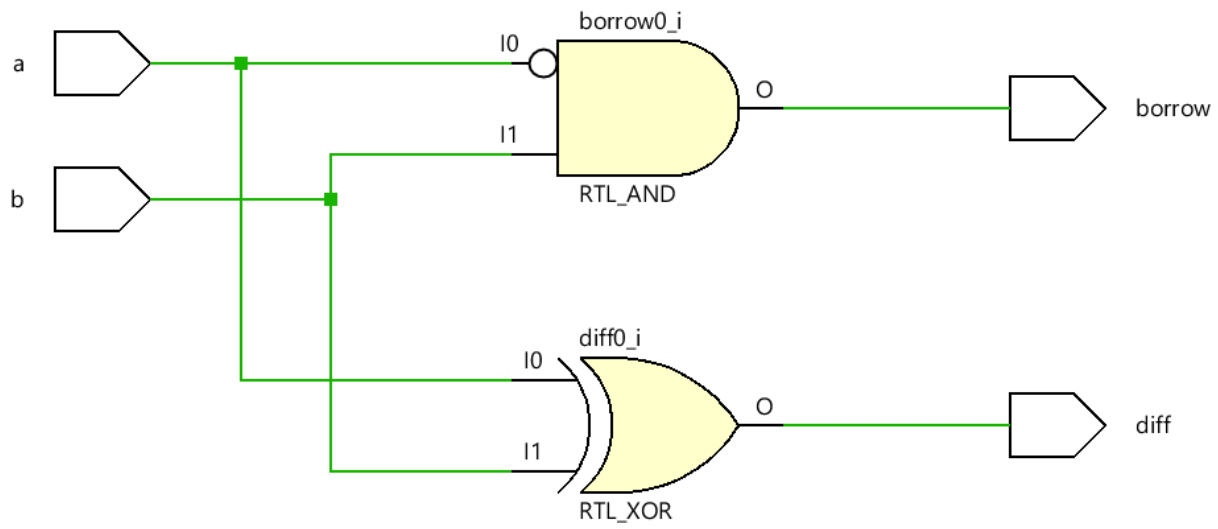


BEHAVIOURAL

```
module half_subtractor_behavioral(input a,b,output reg diff,borrow);
always @(*)
begin
diff=a^b;
borrow=(~a)&b;
end
endmodule
```

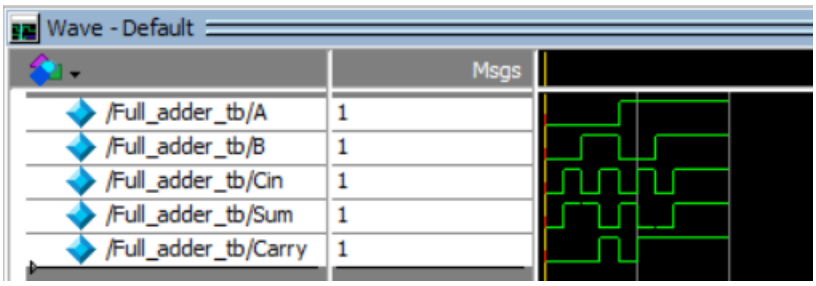


SCHEMATIC



FULL ADDER

DATA LOGIC

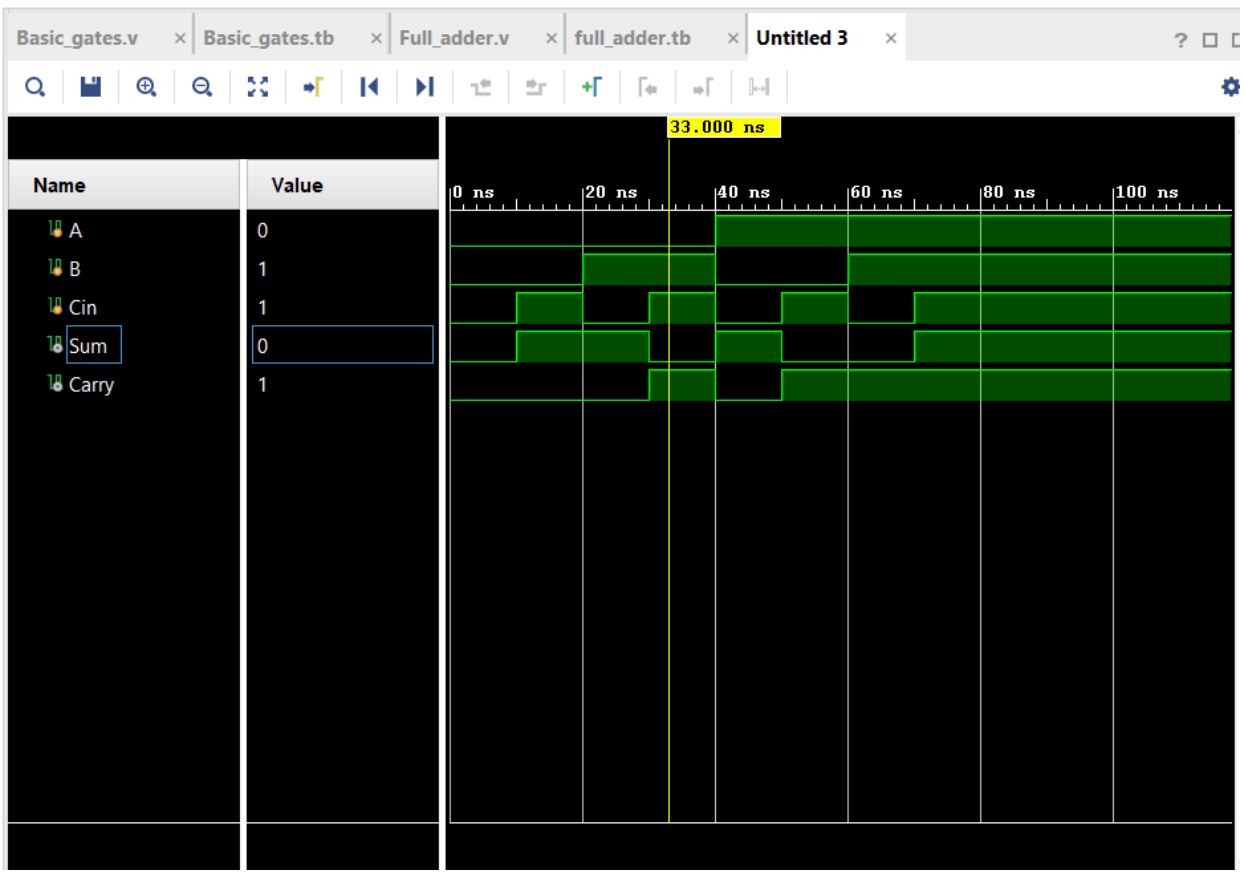


D:/Full_adder.v - Default

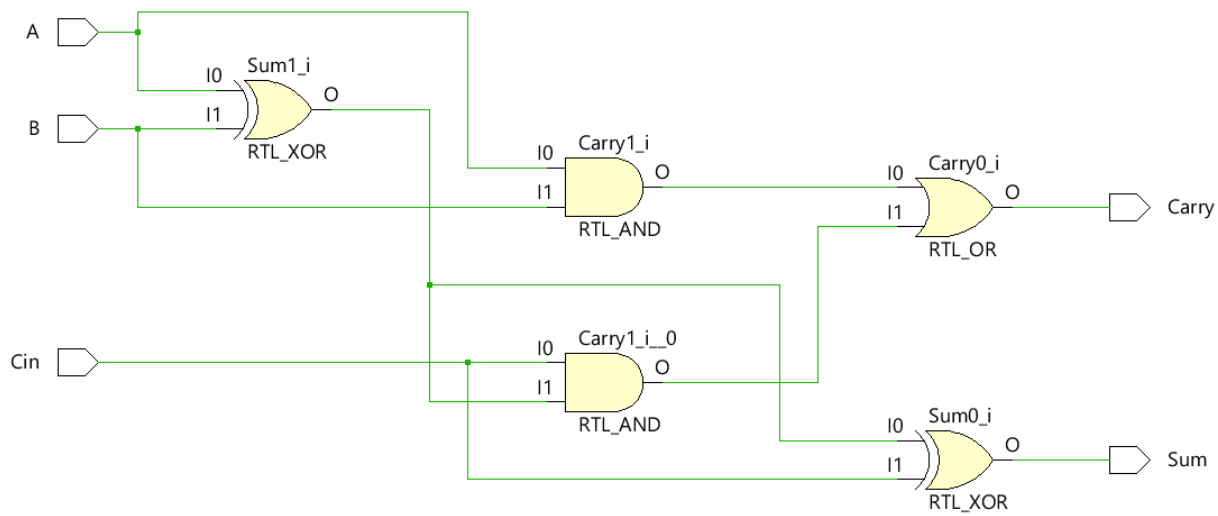
```
module Full_adder(Sum,Carry,A,B,Cin);
input A,B,Cin;
output Sum,Carry;
assign Sum1=A^B;
assign Sum=Sum1^Cin;
assign Carry1=Sum1&Cin;
assign Carry2=A&B;
assign Carry=Carry1|Carry2;
endmodule

module Full_adder_tb;
reg A,B,Cin;
wire Sum,Carry;
Full_adder dut(.A(A),.B(B),.Cin(Cin),.Sum(Sum),.Carry(Carry));
initial
begin
A=1'b0;B=1'b0;Cin=1'b0;
#10 A=1'b0;B=1'b0;Cin=1'b1;
#10 A=1'b0;B=1'b1;Cin=1'b0;
#10 A=1'b0;B=1'b1;Cin=1'b1;
#10 A=1'b1;B=1'b0;Cin=1'b0;
#10 A=1'b1;B=1'b0;Cin=1'b1;
#10 A=1'b1;B=1'b1;Cin=1'b0;
#10 A=1'b1;B=1'b1;Cin=1'b1;
end
initial
begin
$monitor("Time=%0t,A=%0b,B=%0b,Cin=%0b,Sum=%0b,Carry=%0b",$time,A,B,Cin,Sum,Carry);
end
endmodule
```

BEHAVIOUR LOGIC

```
module Full_adder(Sum,Carry,A,B,Cin);
input A,B,Cin;
output reg Sum,Carry;
always @(*)
begin
Sum=(A^B)^Cin;
Carry=(A&B)|(Cin&(A^B));
end
endmodule
```



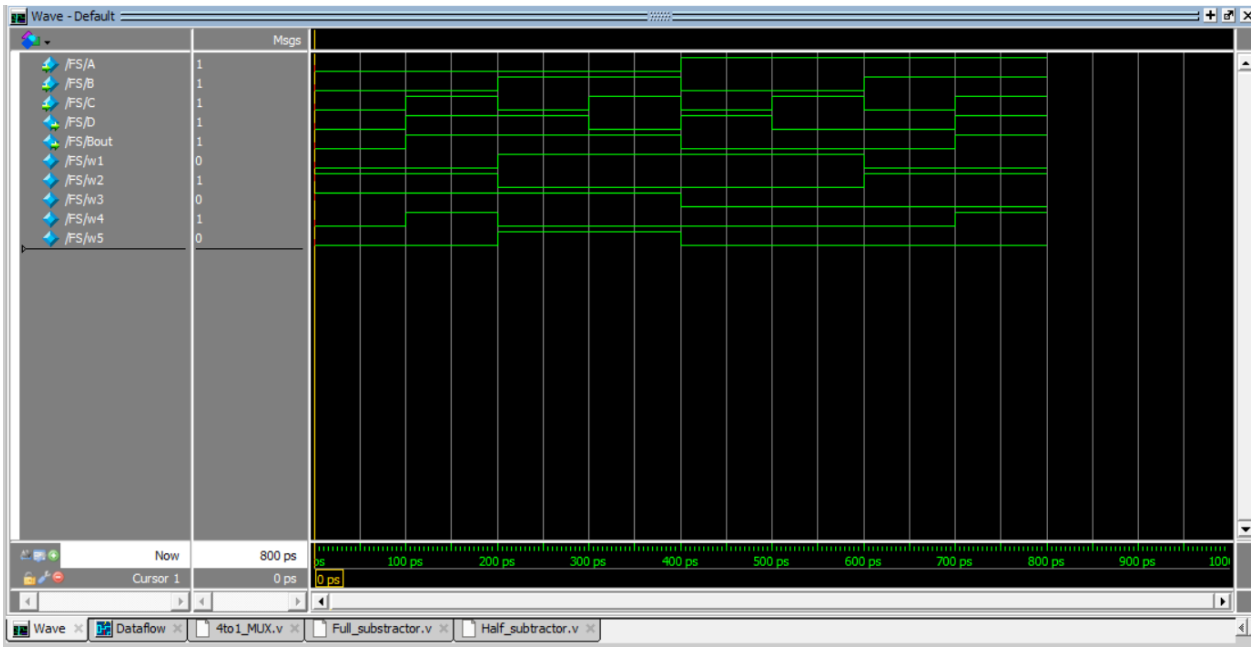
SCHEMATIC



FULL SUBTRACTOR

GATE LOGIC

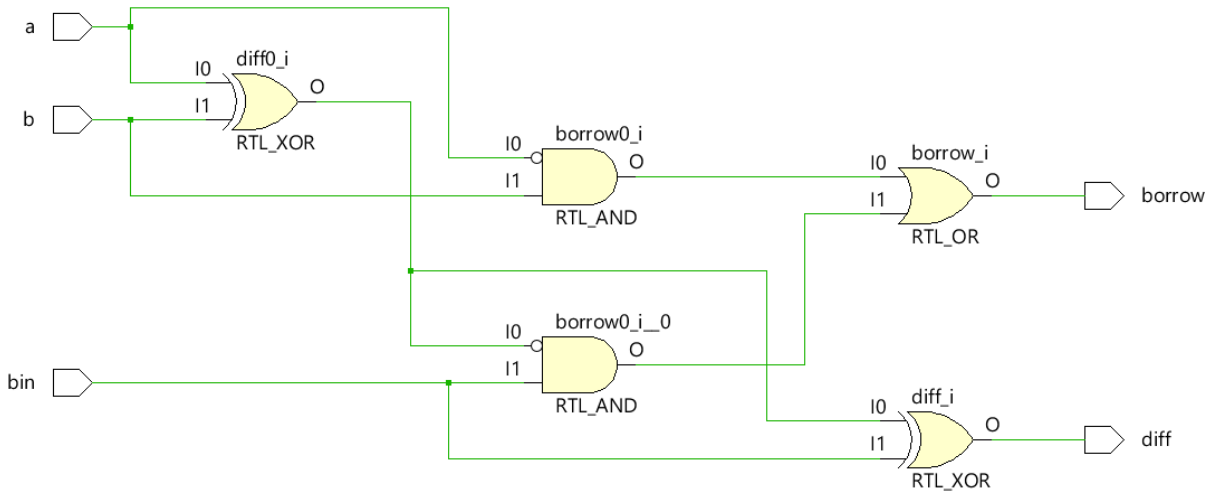
```
Ln#
1   module FS(D,Bout,A,B,C);
2   input A,B,C;
3   output D,Bout;
4   wire w1,w2,w3,w4,w5;
5   not A1(w3,A);
6   xor A2(w1,A,B);
7   xor A3(D,w1,C);
8   xnor A4(w2,B,A);
9   and A5(w4,w2,C);
10  and A6(w5,B,w3);
11  or A7(Bout,w4,w5);
12  endmodule
13
```

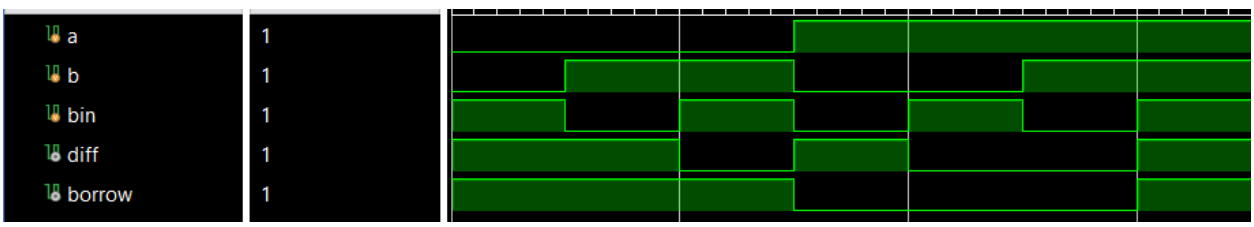


DATALOGIC

```
module full_subtractor_dataflow(a,b,bin,diff,borrow);
input a,b,bin;
output diff,borrow;
assign diff=a^b^bin;
assign borrow=(~a&b)|(~(a^b)&bin);
endmodule
```

SCHEMATIC

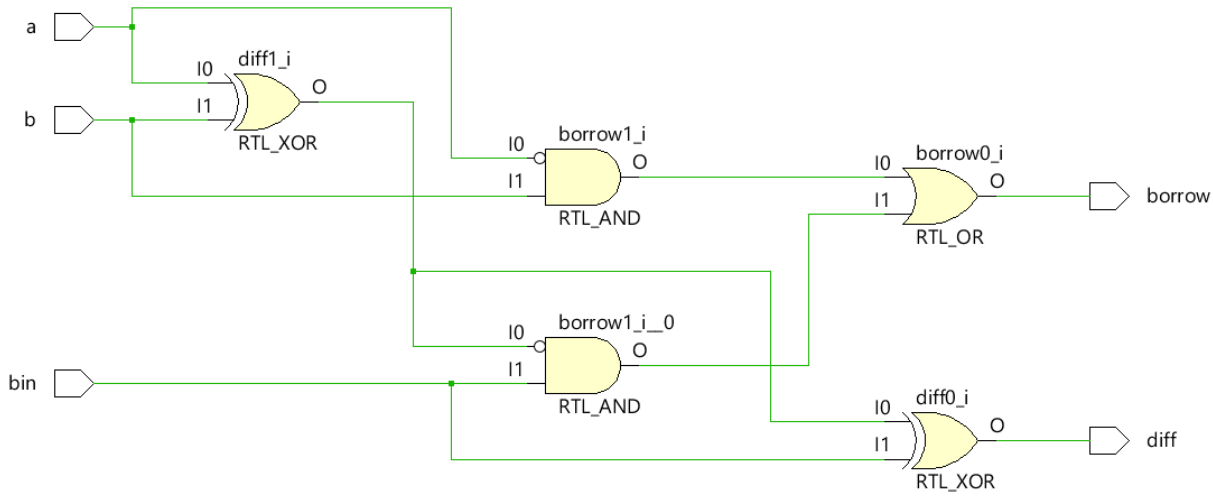


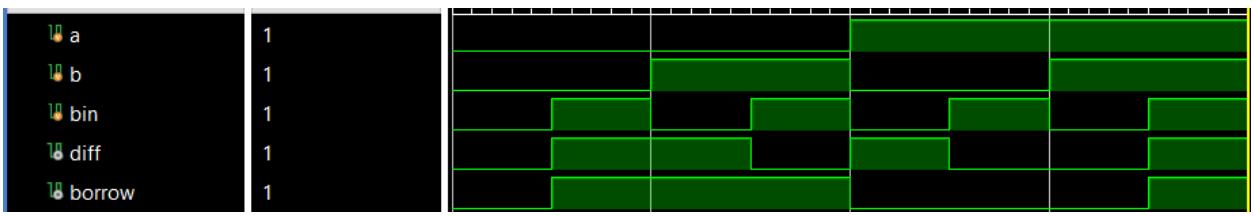


BEHAVIOURAL

```
module full_subtractor_behavioral(input a,b,bin,output reg diff,borrow);
always @(*)
begin
diff=a^b^bin;
borrow=(~a&b)|(~(a^b)&bin);
end
endmodule
```

SCHEMATIC





MUX

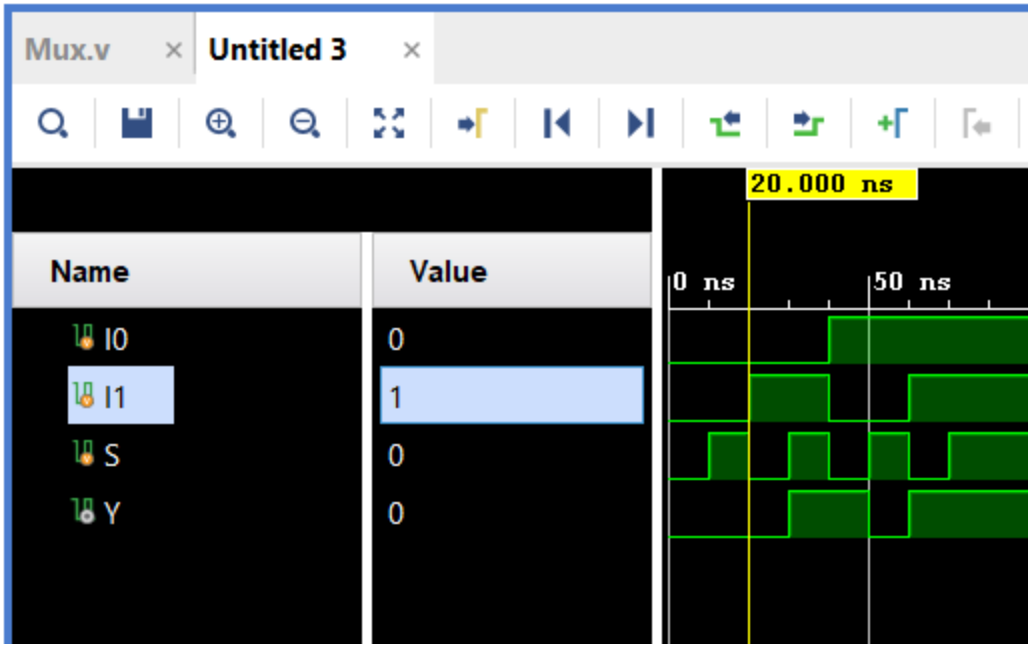
2:1

GATE LOGIC

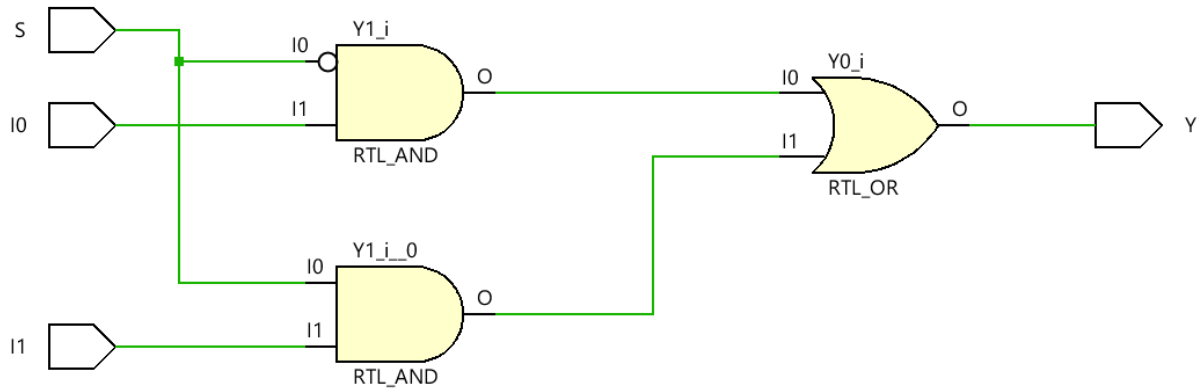
```
module MUX(Y,I0,I1,S);
input S,I0,I1;
output Y;
wire w1,w2,w3;
and A1(w1,S,I1);
and A2(w3,w2,I0);
not A3(w2,S);
or A4(Y,w1,w3);
endmodule
```

BEHAVIOUR LOGIC

```
module MUX(I0,I1,S,Y);
input I0,I1,S;
output reg Y;
always @(*)
begin
Y=((~S)&I0)|((S)&I1);
end
endmodule
```


Mux.v
Untitled 3
Name
Value
I0
I1
S
Y
0
1
0
0
20.000 ns
0 ns
50 ns

SCHEMATIC

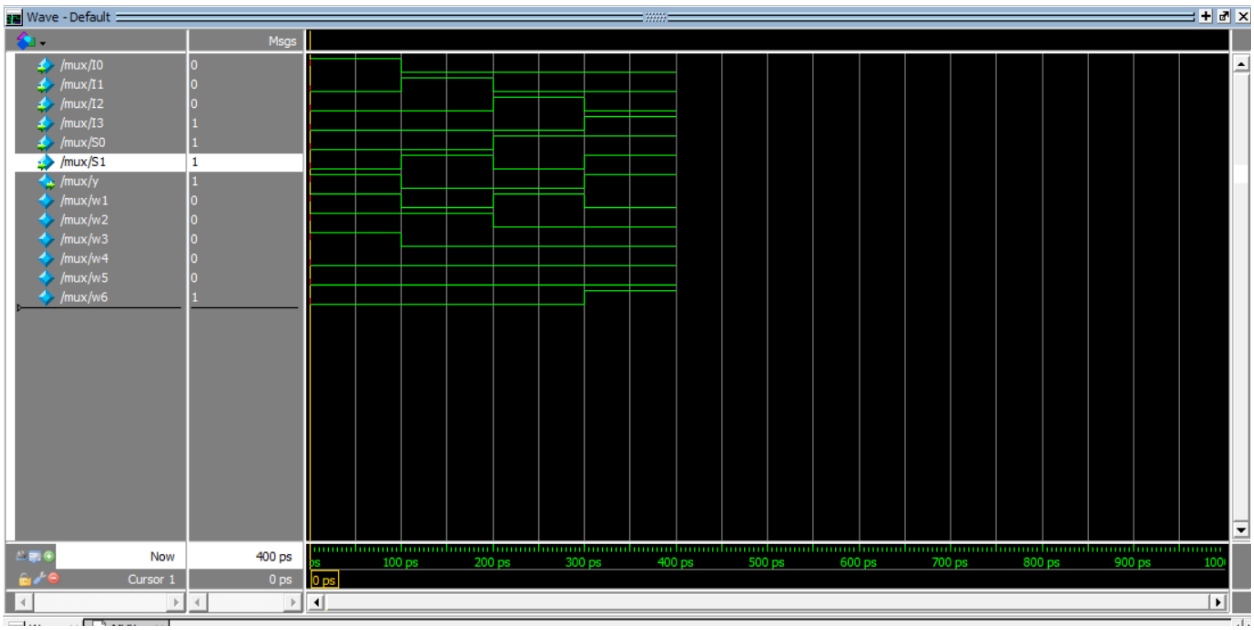

S
I0
I1
I0
I1
Y1_i
O
RTL_AND
Y1_i__0
O
RTL_AND
I0
I1
Y0_i
O
RTL_OR
Y

4:1

GATE LOGIC

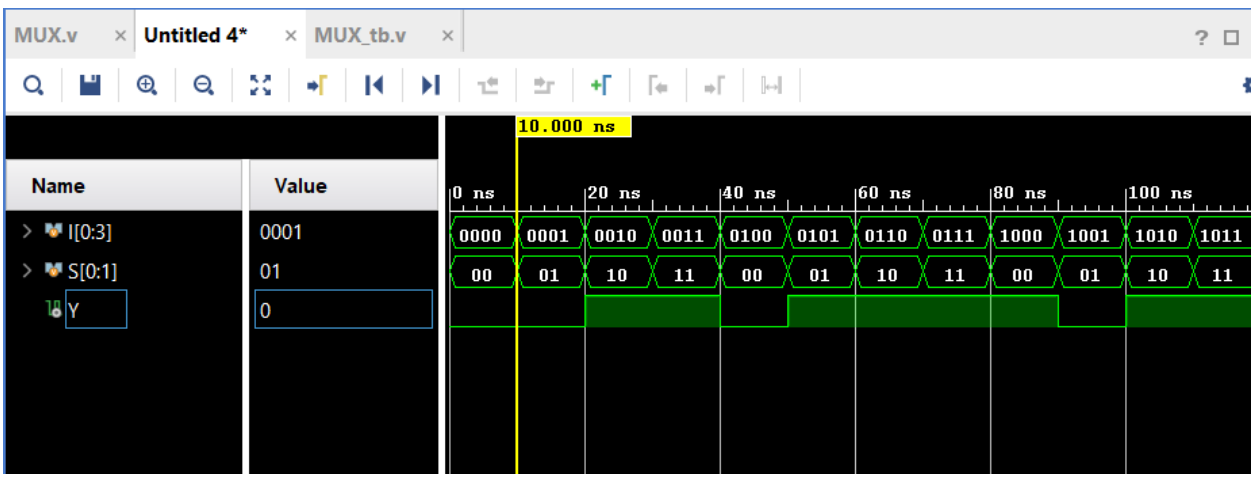
D:/4to1_MUX.v - Default

```
module mux(y,S1,S0,I0,I1,I2,I3);
input I0,I1,I2,I3,S0,S1;
output y;
wire w1,w2,w3,w4,w5,w6;
not B1(w1,S1);
not B2(w2,S0);
and A1(w3,w1,w2,I0);
and A2(w4,S0,w1,I1);
and A3(w5,S1,w2,I2);
and A4(w6,S1,S0,I3);
or  A5(y,w3,w4,w5,w6);
endmodule
```

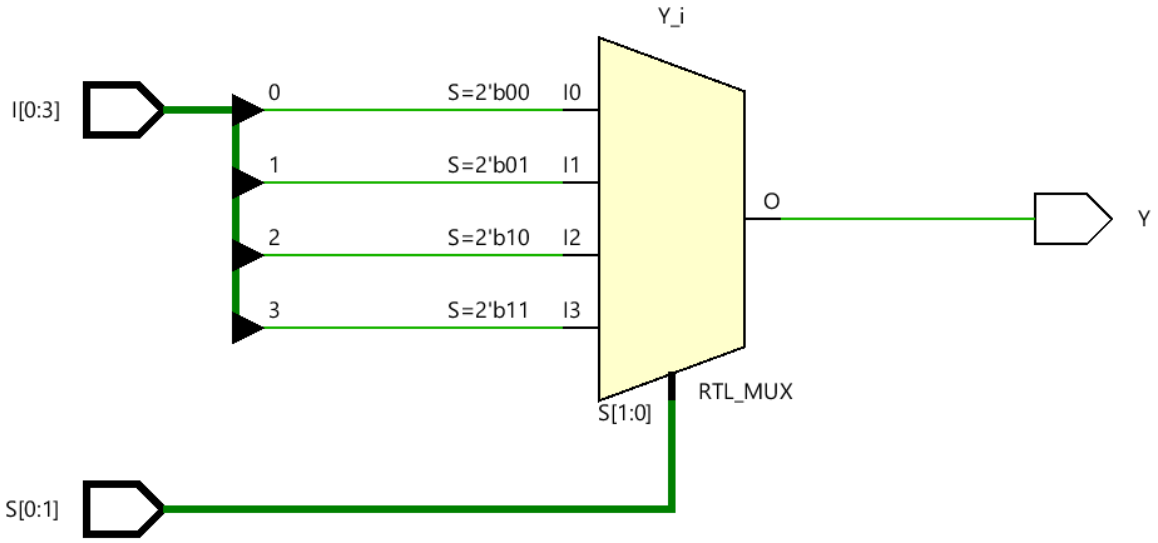


BEHAVIOUR LOGIC

```
module MUX(input[0:3]I,[0:1]S, output reg Y);
always@(*)
begin
case(S)
2'b00:Y=I[0];
2'b01:Y=I[1];
2'b10:Y=I[2];
2'b11:Y=I[3];
default:Y=1'bx;
endcase
end
endmodule
```



SCHEMATIC

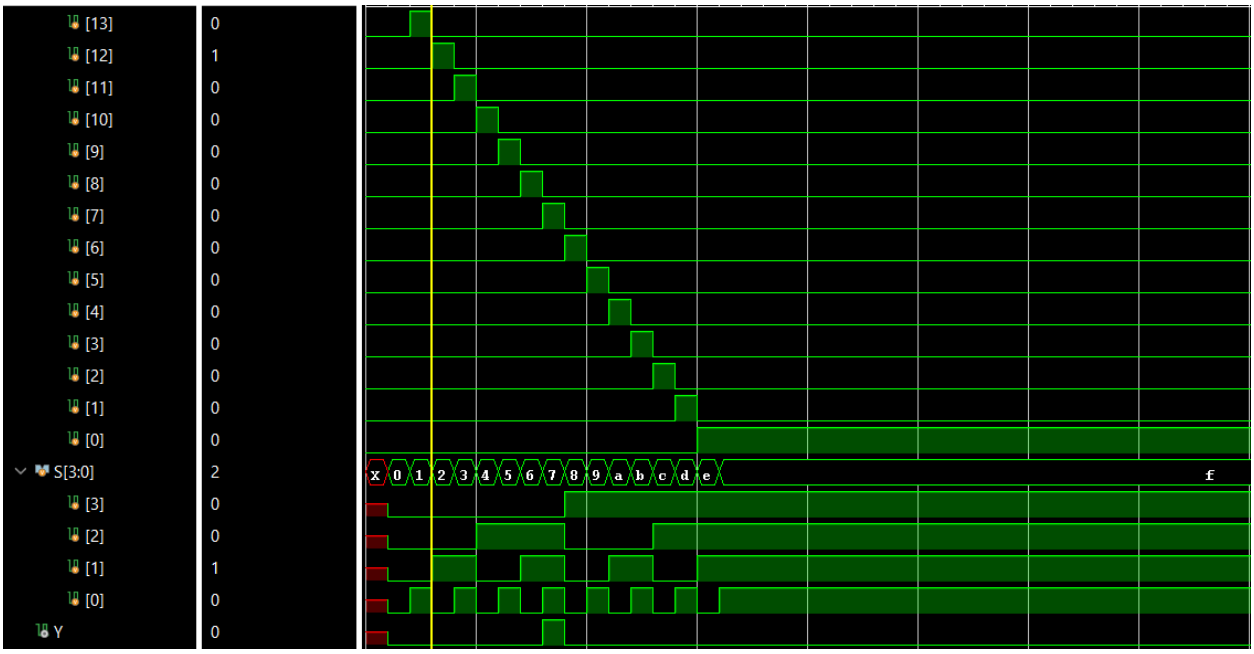

Y_i
I[0:3]
0
S=2'b00
I0
1
S=2'b01
I1
2
S=2'b10
I2
3
S=2'b11
I3
O
Y
RTL_MUX
S[1:0]
S[0:1]

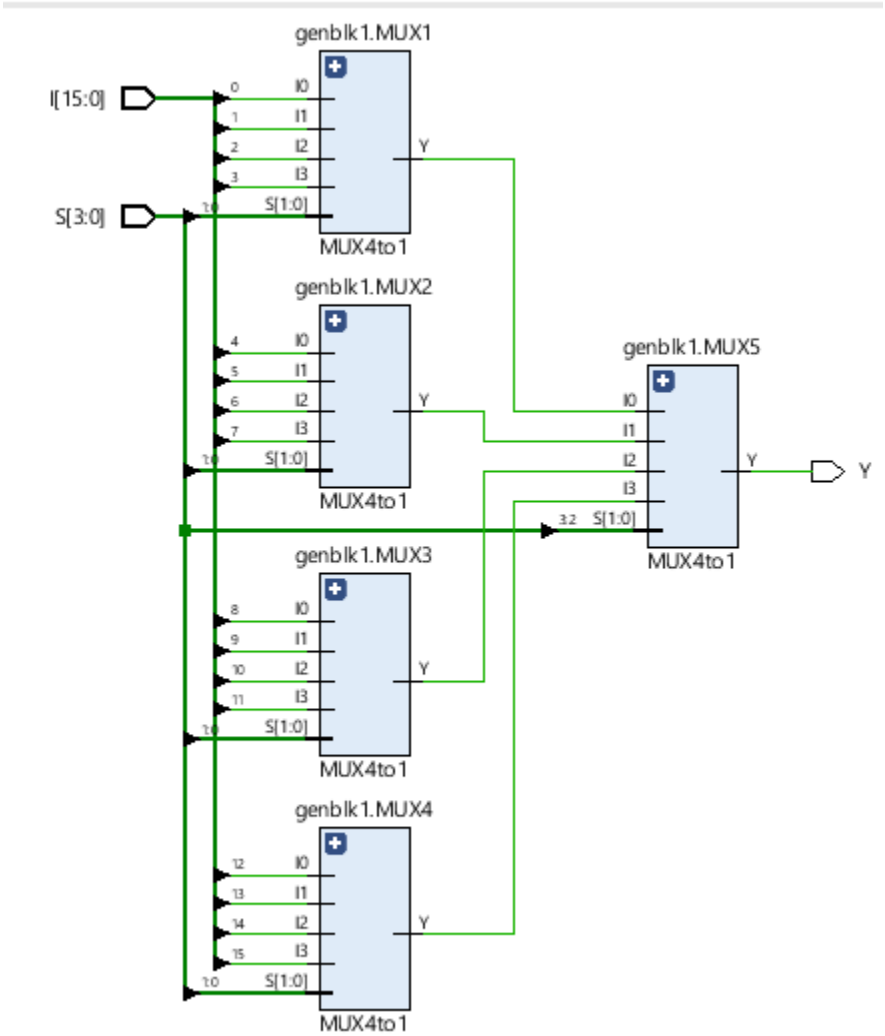
<u>MUX 16:1</u>

```
module MUX(I0,I1,S,Y);
input I0,I1,S;
output reg Y;
always @(*)
begin
Y=((~S)&I0)|((S)&I1);
end
endmodule
```

```
module MUX_8to1(
input I0,I1,I2,I3,I4,I5,I6,I7,
input[2:0]S,
output Y);
wire W1,W2,W3,W4,W5,W6;
begin
MUX MUX1(.I0(I0),.I1(I1),.S(S[0]),.Y(W1));
MUX MUX2(.I0(I2),.I1(I3),.S(S[0]),.Y(W2));
MUX MUX3(.I0(I4),.I1(I5),.S(S[0]),.Y(W3));
MUX MUX4(.I0(I6),.I1(I7),.S(S[0]),.Y(W4));
MUX MUX5(.I0(W1),.I1(W2),.S(S[1]),.Y(W5));
MUX MUX6(.I0(W3),.I1(W4),.S(S[1]),.Y(W6));
MUX MUX7(.I0(W5),.I1(W6),.S(S[2]),.Y(Y));
end
endmodule
```

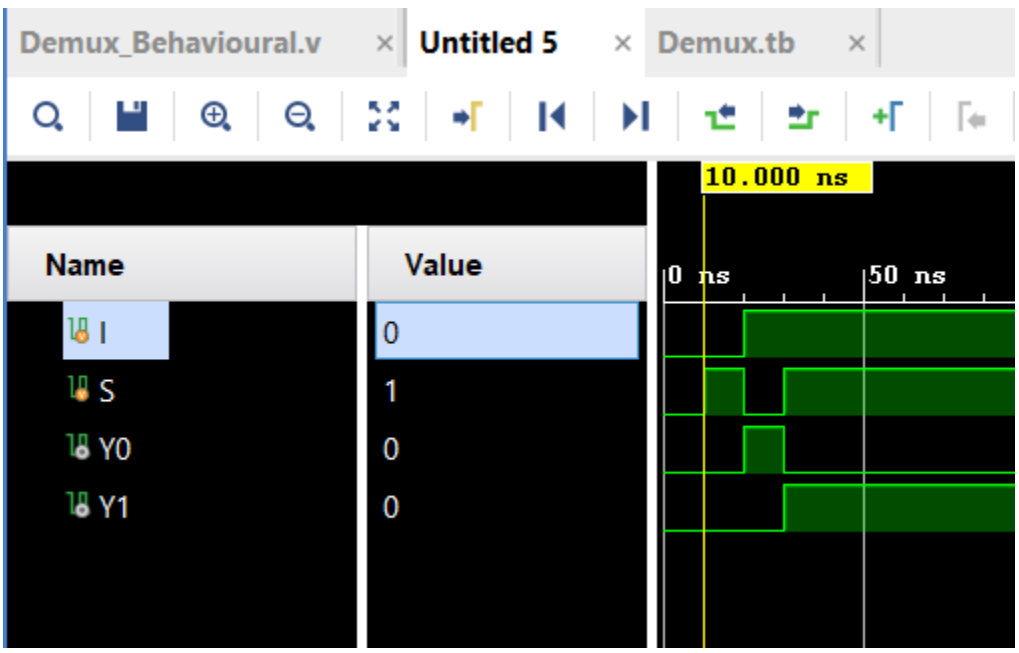
```
module MUX_16to1(input [15:0]I,input [3:0]S,output Y);
wire W1,W2,W3,W4;
begin
MUX4to1 MUX1(.I0(I[0]),.I1(I[1]),.I2(I[2]),.I3(I[3]),.S(S[1:0]),.Y(W1));
MUX4to1 MUX2(.I0(I[4]),.I1(I[5]),.I2(I[6]),.I3(I[7]),.S(S[1:0]),.Y(W2));
MUX4to1 MUX3(.I0(I[8]),.I1(I[9]),.I2(I[10]),.I3(I[11]),.S(S[1:0]),.Y(W3));
MUX4to1 MUX4(.I0(I[12]),.I1(I[13]),.I2(I[14]),.I3(I[15]),.S(S[1:0]),.Y(W4));
MUX4to1 MUX5(.I0(W1),.I1(W2),.I2(W3),.I3(W4),.S(S[3:2]),.Y(Y));
end
endmodule
```


[13] 0
[12] 1
[11] 0
[10] 0
[9] 0
[8] 0
[7] 0
[6] 0
[5] 0
[4] 0
[3] 0
[2] 0
[1] 0
[0] 0
S[3:0] 2
[3] 0
[2] 0
[1] 1
[0] 0
Y 0
x 0 1 2 3 4 5 6 7 8 9 a b c d e f

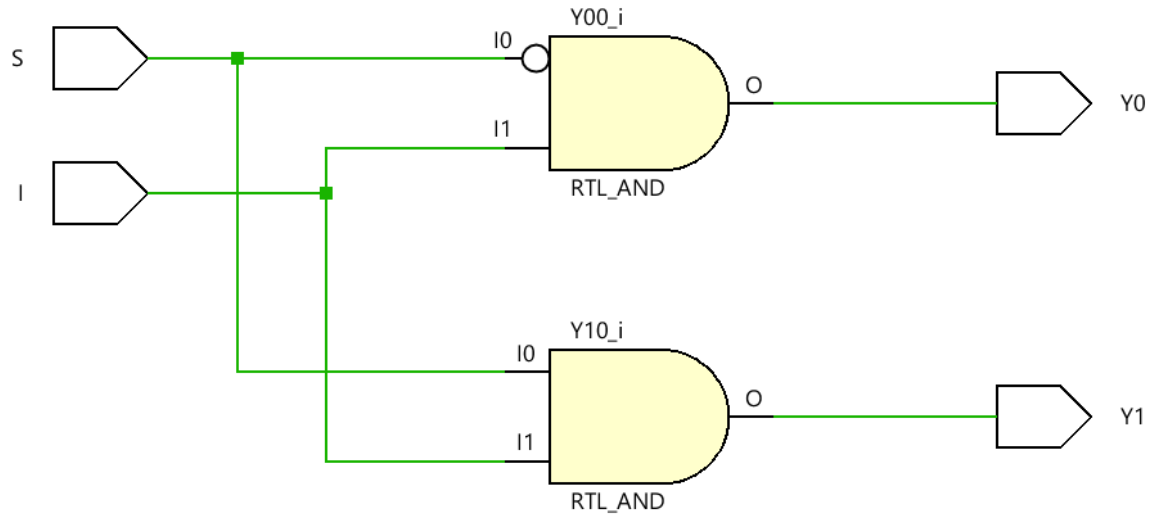

genblk1.MUX1
I[15:0]
S[3:0]
I0
I1
I2
I3
S[1:0]
Y
MUX4to1
genblk1.MUX2
genblk1.MUX3
genblk1.MUX4
genblk1.MUX5
Y

DEMUX

```
module Demux(I,S,Y0,Y1);
input I,S;
output reg Y0,Y1;
always @ (*)
begin
Y0=((~S)&I);
Y1=(S&I);
end
endmodule
```



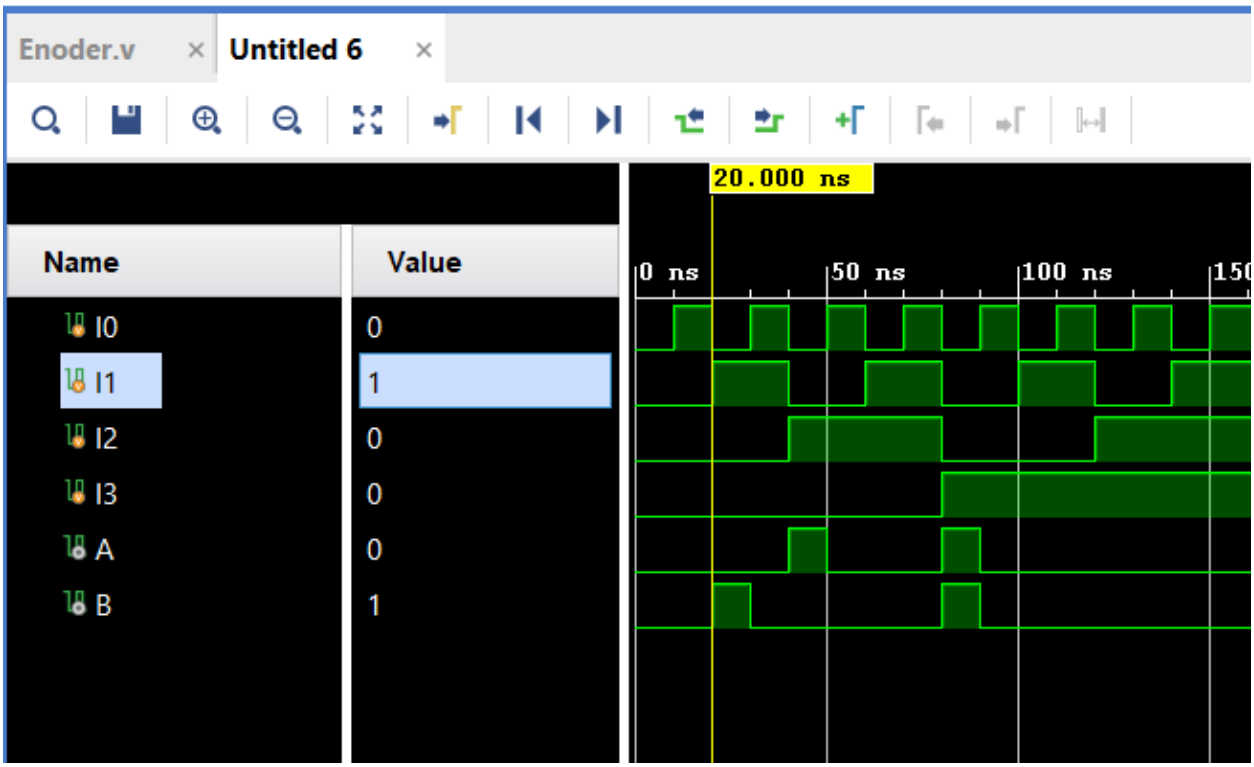
SCHEMATIC

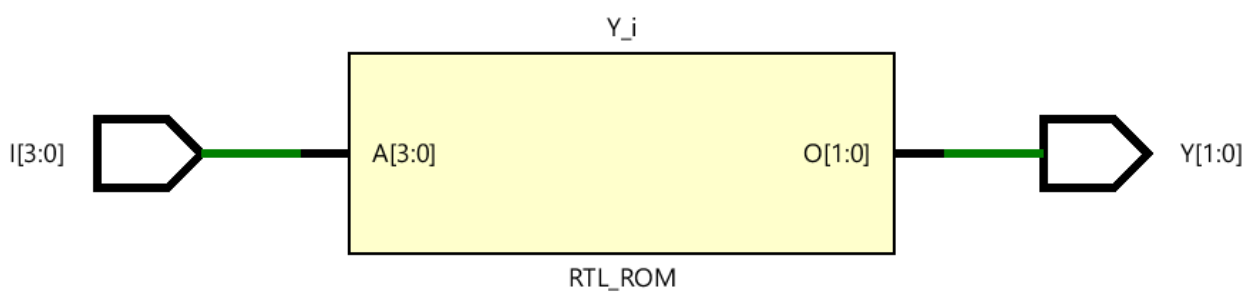


ENCODER

```
module Enoder(I0,I1,I2,I3,A,B);
input I0,I1,I2,I3;
output reg A,B;
always @(*)
begin
A=((~I0)&(~I1)&(I2)&(~I3))|((~I0)&(~I1)&(~I2)&(I3));
B=((~I0)&(I1)&(~I2)&(~I3))|((~I0)&(~I1)&(~I2)&(I3));
end
endmodule
```

```
module Encoder(input [3:0]I , output reg [1:0]Y);
always@(*)
begin
case(I)
4'b0001:Y=2'b00;
4'b0010:Y=2'b01;
4'b0100:Y=2'b10;
4'b1000:Y=2'b11;
default Y=2'bxx;
endcase
end
endmodule
```

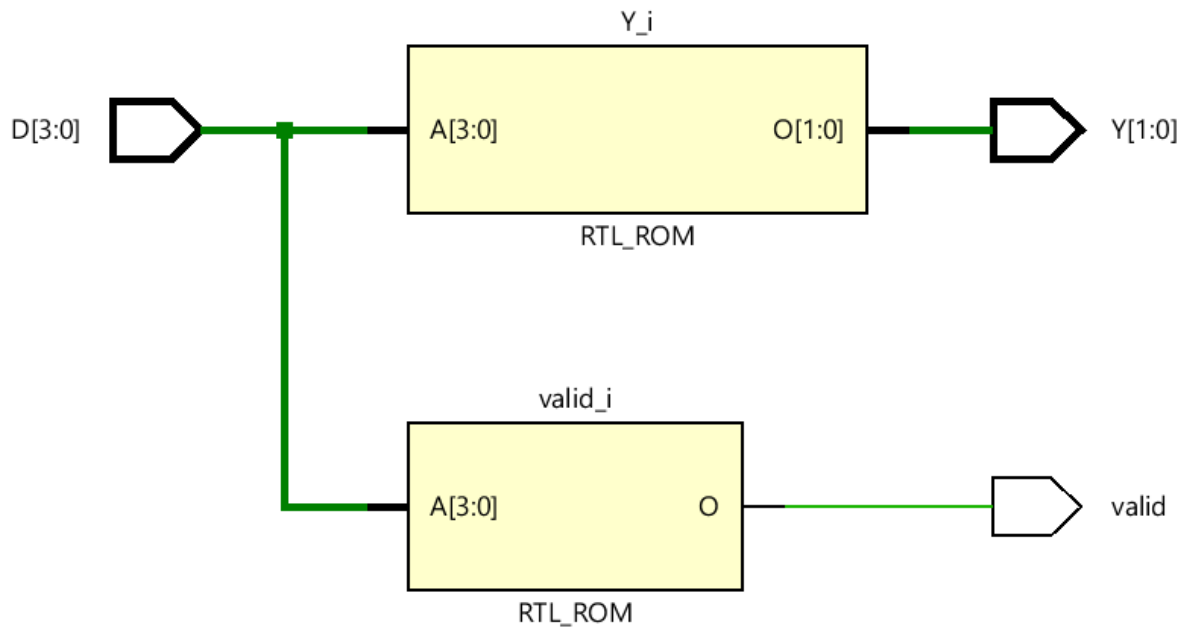


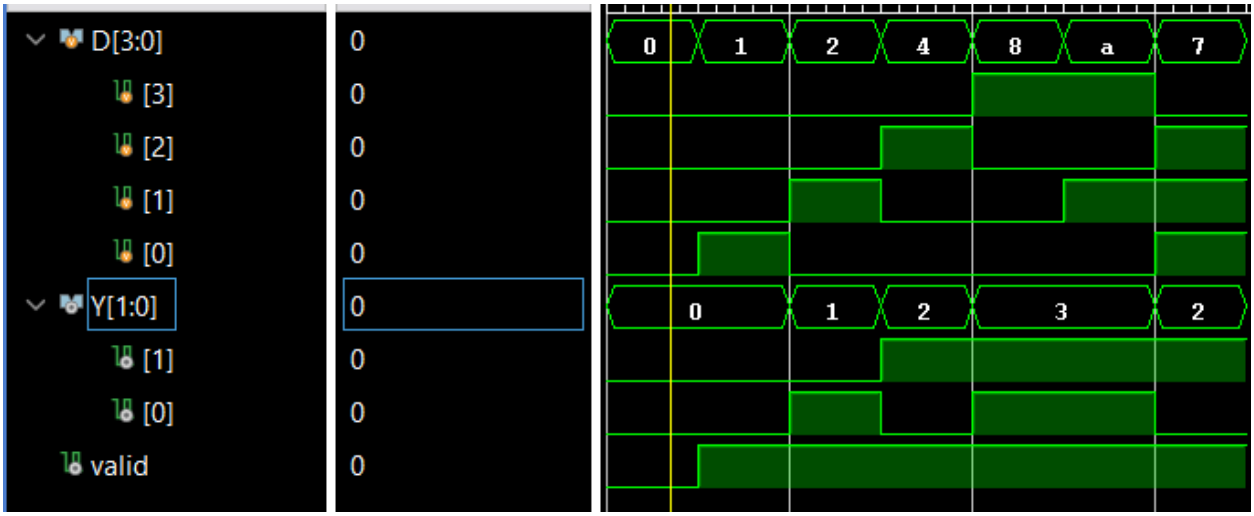


PRIORITY ENCODER

```
module PriorityEncoder4to2 (input [3:0] D,
output reg [1:0] Y,
output reg valid);
always @(*) begin
valid = 1;
casez (D)
4'b1???: Y = 2'b11;
4'b01??: Y = 2'b10;
4'b001?: Y = 2'b01;
4'b0001: Y = 2'b00;
default: begin
Y = 2'b00;
valid = 0;
end
endcase
end
endmodule
```

SCHEMATIC

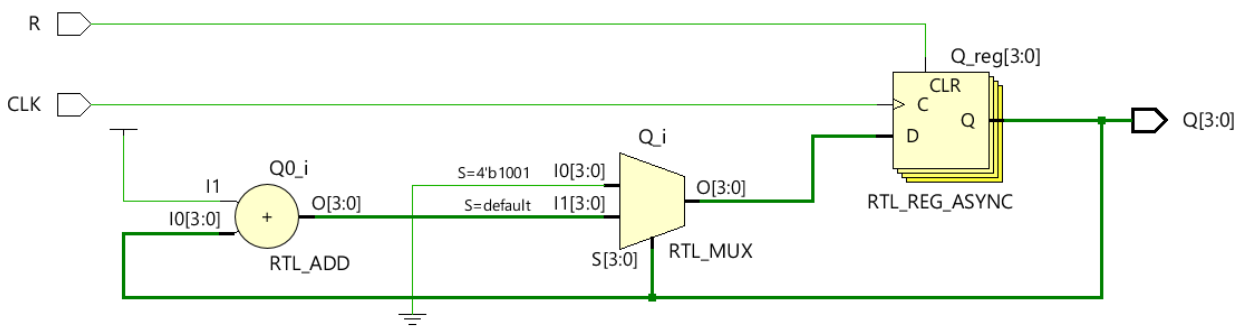

Y_i
D[3:0]
A[3:0]
O[1:0]
Y[1:0]
RTL_ROM
valid_i
A[3:0]
O
valid
RTL_ROM

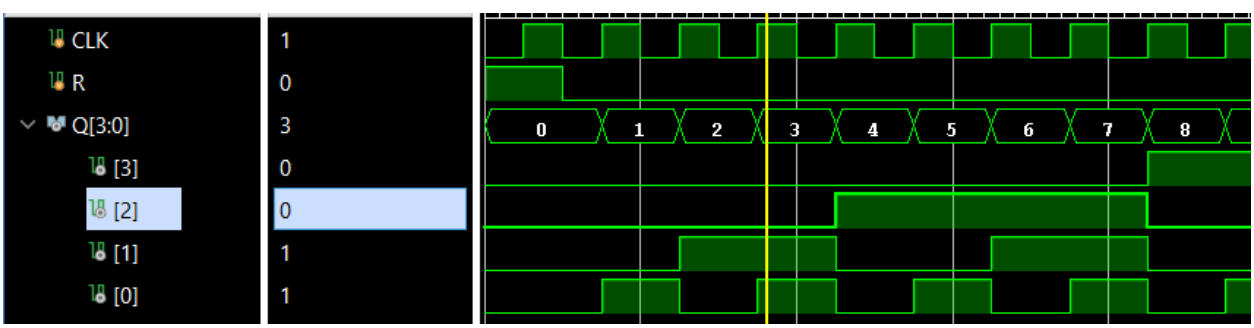

D[3:0]
[3]
[2]
[1]
[0]
Y[1:0]
[1]
[0]
valid
0
0
0
0
0
0
0
0
0
0
1
2
4
8
a
7
0
1
2
3
2

BCD-ENCODER

```
module BCD_SYNC_COUNTER (input CLK, R,output reg [3:0] Q);
always @(posedge CLK or posedge R) begin
if (R)
Q <= 4'b0000;
else if (Q == 4'b1001)
Q <= 4'b0000;
else
Q <= Q + 1;
end
endmodule
```

SCHEMATIC

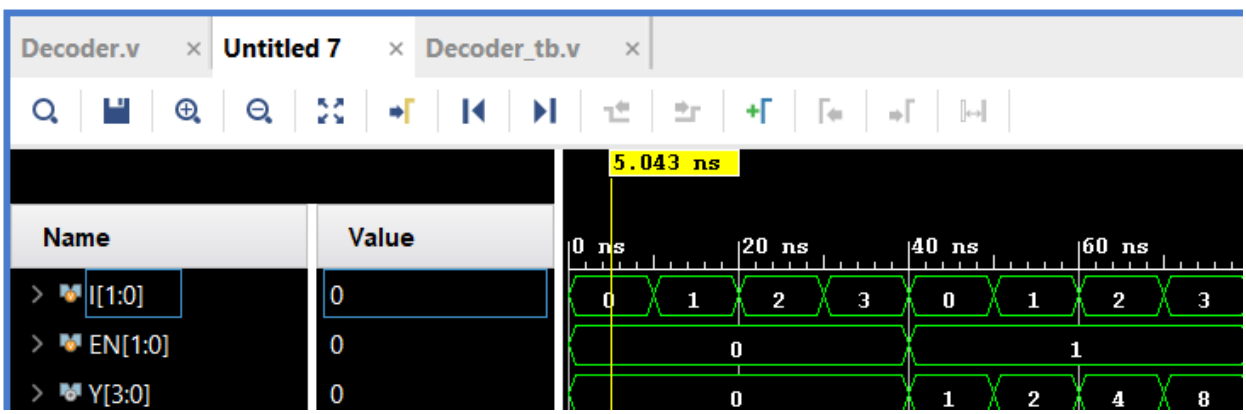




DECODER

2:4

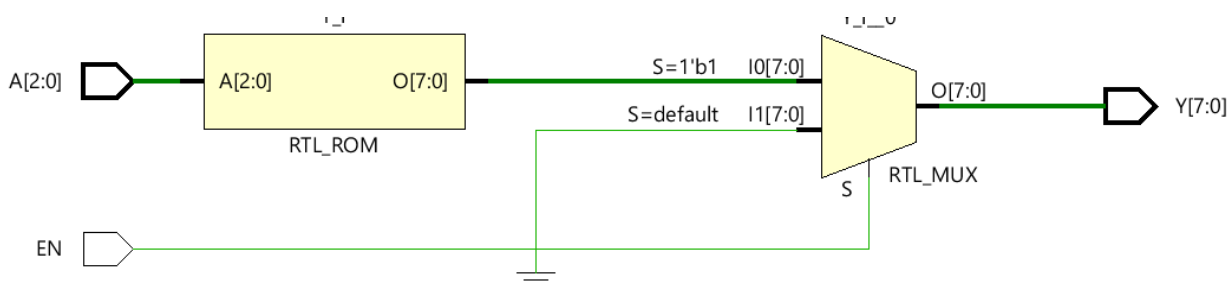
```
module Decoder(input[1:0]I,EN,output reg[3:0]Y);
always@(*)
begin
if(EN)
begin
case(I)
2'b00:Y=4'b0001;
2'b01:Y=4'b0010;
2'b10:Y=4'b0100;
2'b11:Y=4'b1000;
default:Y=4'b0000;
endcase
end
else
begin
Y=4'b0000;
end
end
endmodule
```



3:8

```
module Decoder3to8 (input [2:0] A,
input EN,
output reg [7:0] Y);
always @(*) begin
if (EN) begin
case (A)
3'b000: Y = 8'b00000001;
3'b001: Y = 8'b00000010;
3'b010: Y = 8'b00000100;
3'b011: Y = 8'b00001000;
3'b100: Y = 8'b00010000;
3'b101: Y = 8'b00100000;
3'b110: Y = 8'b01000000;
3'b111: Y = 8'b10000000;
endcase
end
else
Y = 8'b00000000;
end
endmodule
```

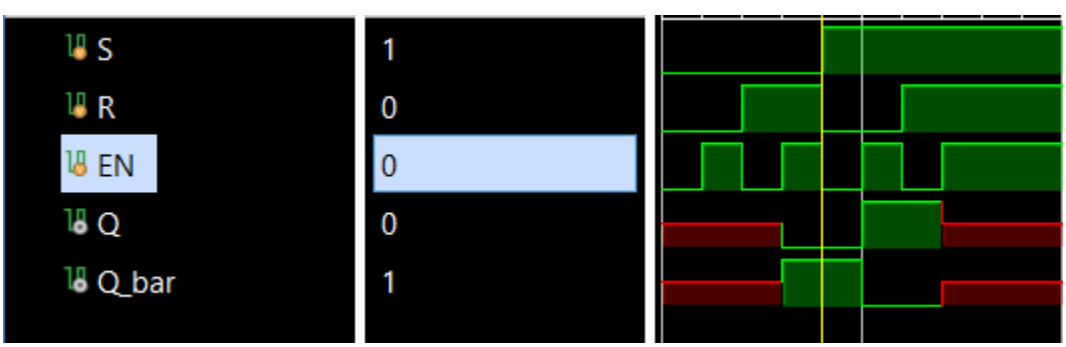
SCHEMATIC



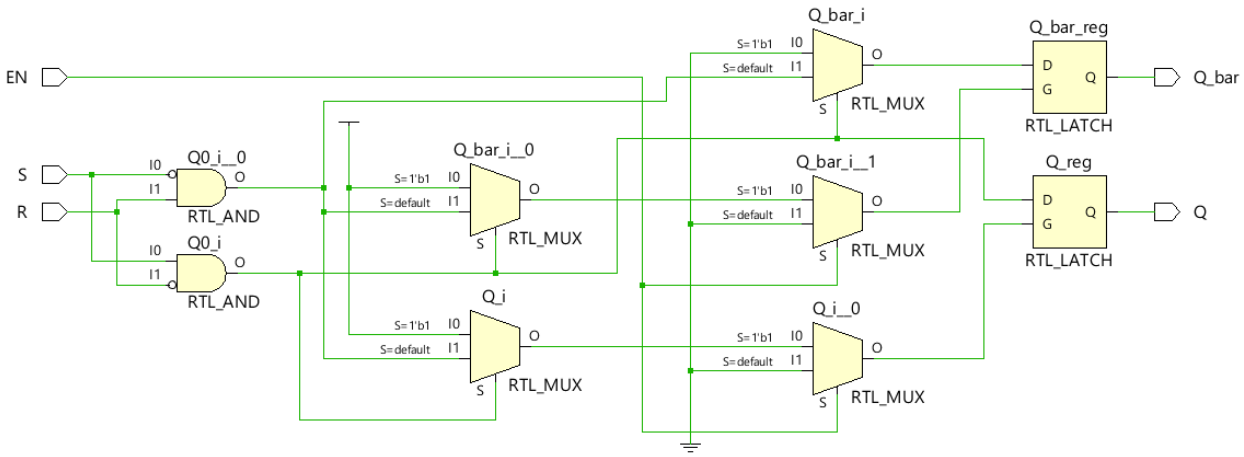
LATCHES

SR LATCH

```
module SR_latch(S,R,Q,EN,Q_bar);
input S,R,EN;
output reg Q,Q_bar;
always@(*)
begin
if(EN)
begin
if(S&~R)
begin
Q=1'b1;
Q_bar=1'b0;
end
else if(~S&R)
begin
Q=1'b0;
Q_bar=1'b1;
end
else if(~S&~R)
begin
Q=Q;
Q_bar=Q_bar;
end
else
begin
Q=1'bx;
Q_bar=1'bx;
end
end
else
begin
Q=Q;
Q_bar=Q_bar;
end
end
endmodule
```



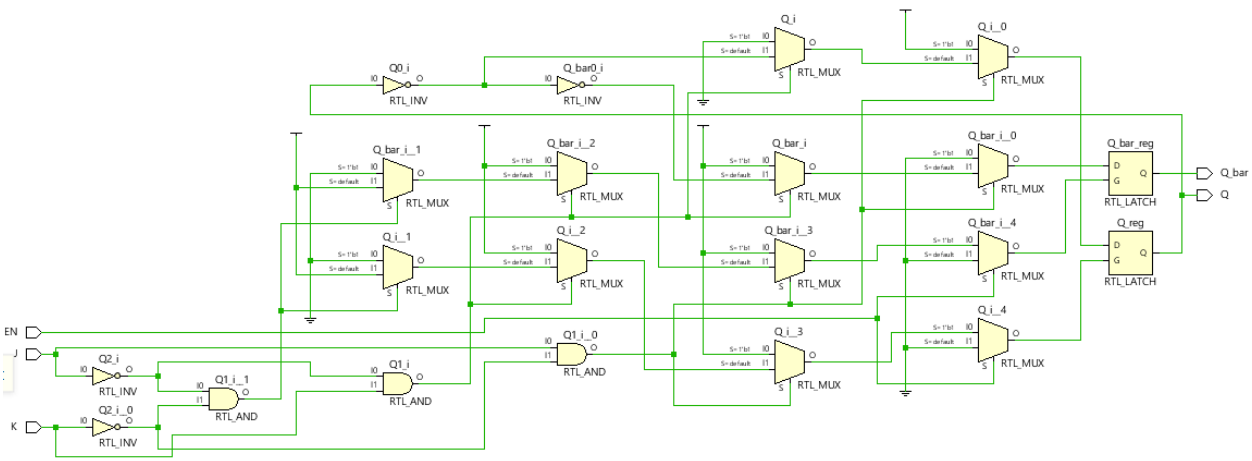
SCHEMATIC


EN
S
R
Q0_i__0
Q0_i
RTL_AND
RTL_AND
Q_bar_i__0
Q_i
Q_bar_i
Q_bar_i__1
Q_i__0
RTL_MUX
RTL_MUX
RTL_MUX
RTL_MUX
RTL_MUX
Q_bar_reg
Q_reg
RTL_LATCH
RTL_LATCH
Q_bar
Q

JK-LATCH

```
module JK_latch(J,K,EN,Q,Q_bar);
input J,K,EN;
output reg Q,Q_bar;
always@(*)
begin
if(EN)
begin
if(J&~K)
begin
Q=1'b1;
Q_bar=1'b0;
end
else if(~J&K)
begin
Q=1'b0;
Q_bar=1'b1;
end
else if(~J&~K)
begin
Q=Q;
Q_bar=Q_bar;
end
else
begin
Q=~Q;
Q_bar=~Q;
end
end
end
endmodule
```

SCHEMATIC

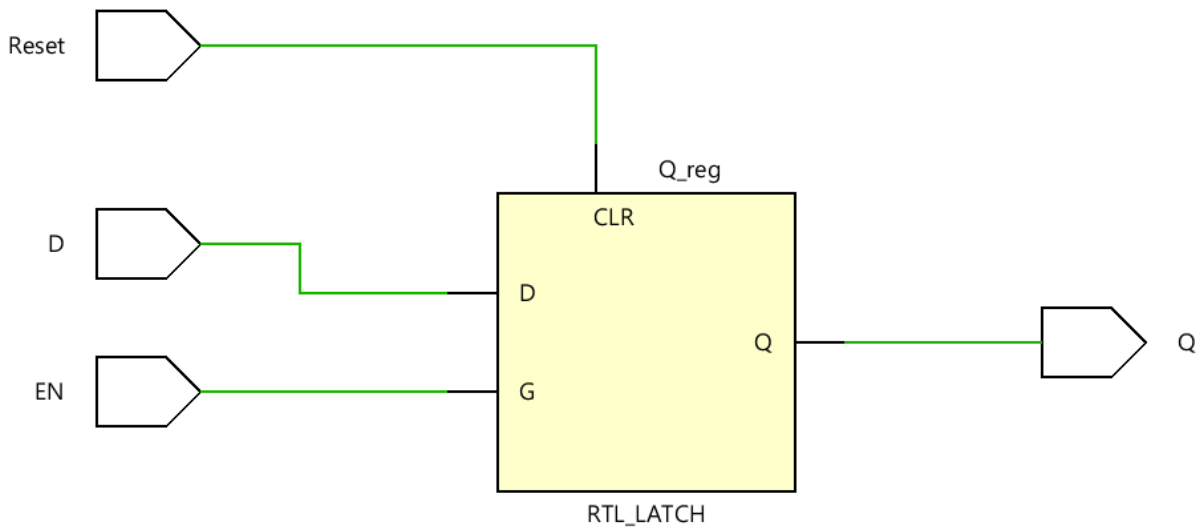


D-LATCH

```
module D_latch(Q,EN,D,Reset);
input EN,D,Reset;
output reg Q;
always@(*)
begin
if(Reset)
Q=0;
else if(EN)
Q=D;
end
endmodule
```

Name
Value
EN
0
D
1
Reset
1
Q
0
30.000 ns
0 ns
50 ns

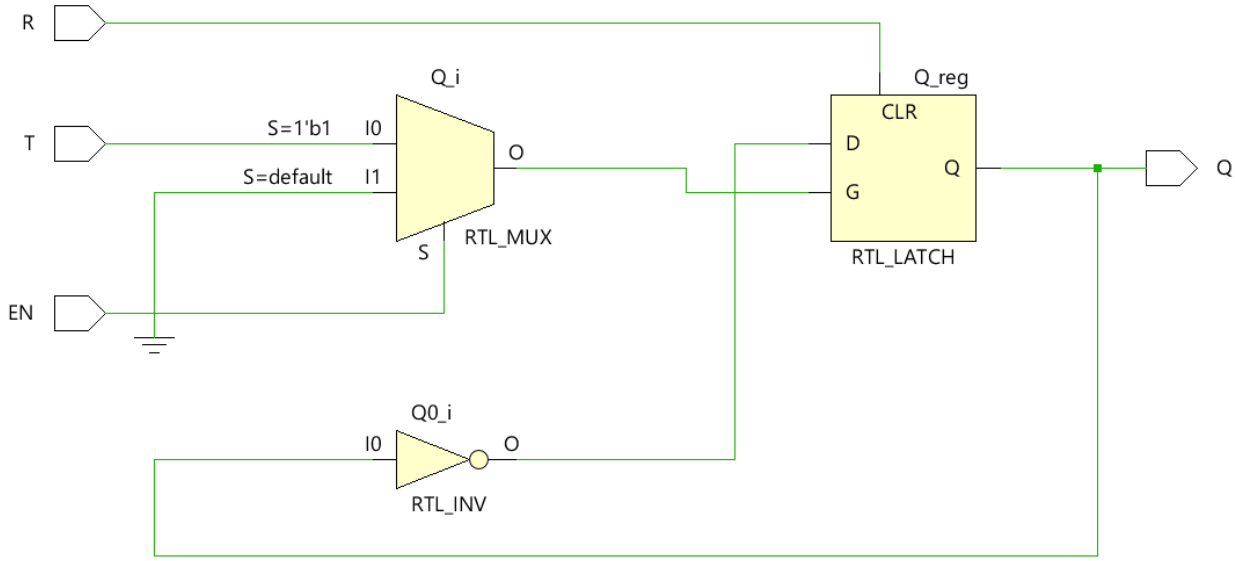
SCHEMATIC


Reset
D
EN
Q_reg
CLR
D
G
Q
RTL_LATCH
Q

T-LATCH

```
module T_latch (T, EN,R,Q);
input T, EN, R;
output reg Q;
always @(*) begin
if (R) begin
Q =0;
end
else if (EN) begin
if (T)
Q = ~Q;
else
Q = Q;
end
else begin
Q = Q;
end
end
endmodule
```

SCHEMATIC



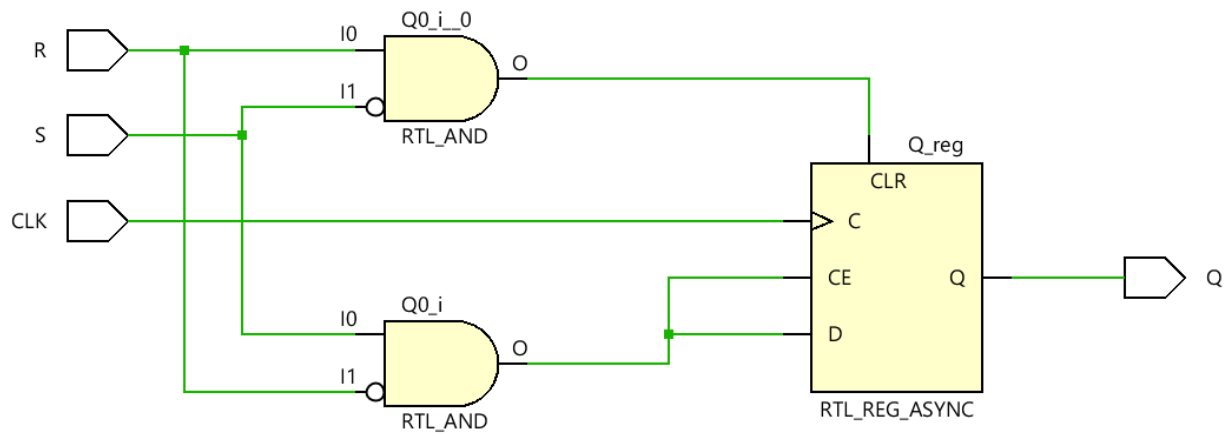
FLIPFLOPS

SR FLIPFLOP

ASYNCHRONOUS

```
module SR_FF(input CLK,S,R, output reg Q);
always @(posedge CLK or posedge S or negedge R)
begin
if (R && ~S)
Q <= 1'b0;
else if (S && ~R)
Q <= 1'b1;
else if (S && R)
Q <= 1'bx;
else
Q <= Q;
end
endmodule
```

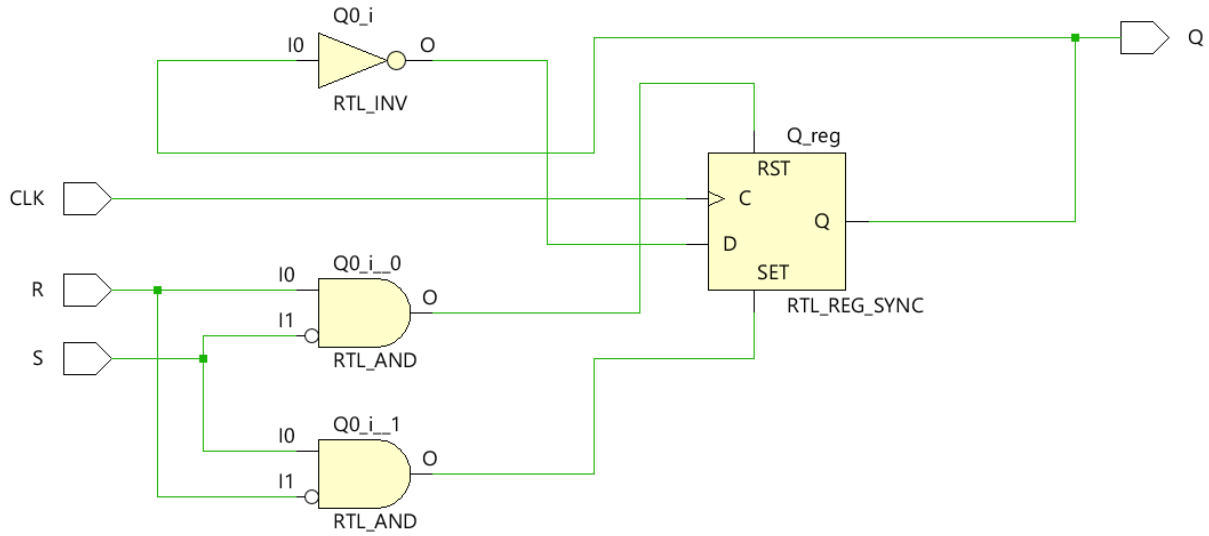
SCHEMATIC

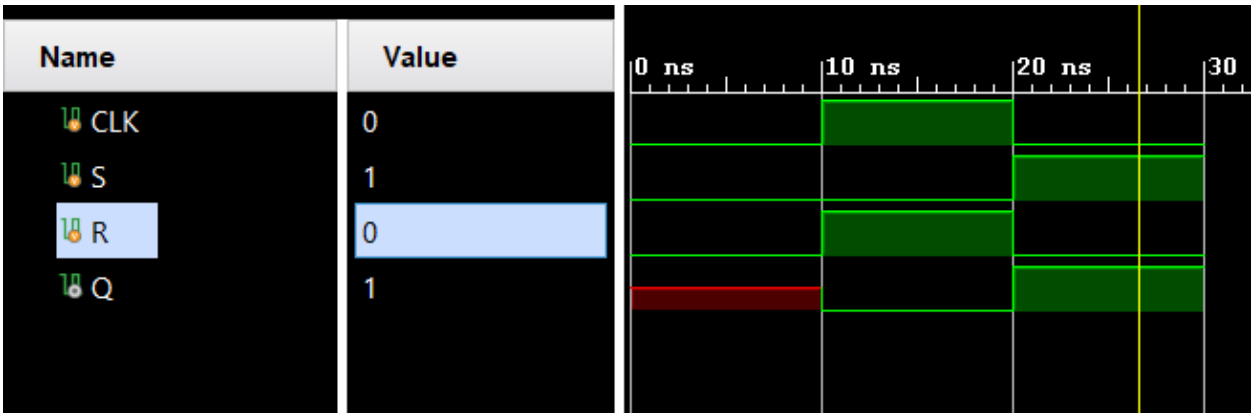


SYNCHRONOUS

```
module SR_FF(input CLK,S,R, output reg Q);
always @(posedge CLK)
begin
if (R && ~S)
Q <= 1'b0;
else if (S && ~R)
Q <= 1'b1;
else if (S && R)
Q <= 1'bx;
else
Q<=~Q;
end
endmodule
```

SCHEMATIC



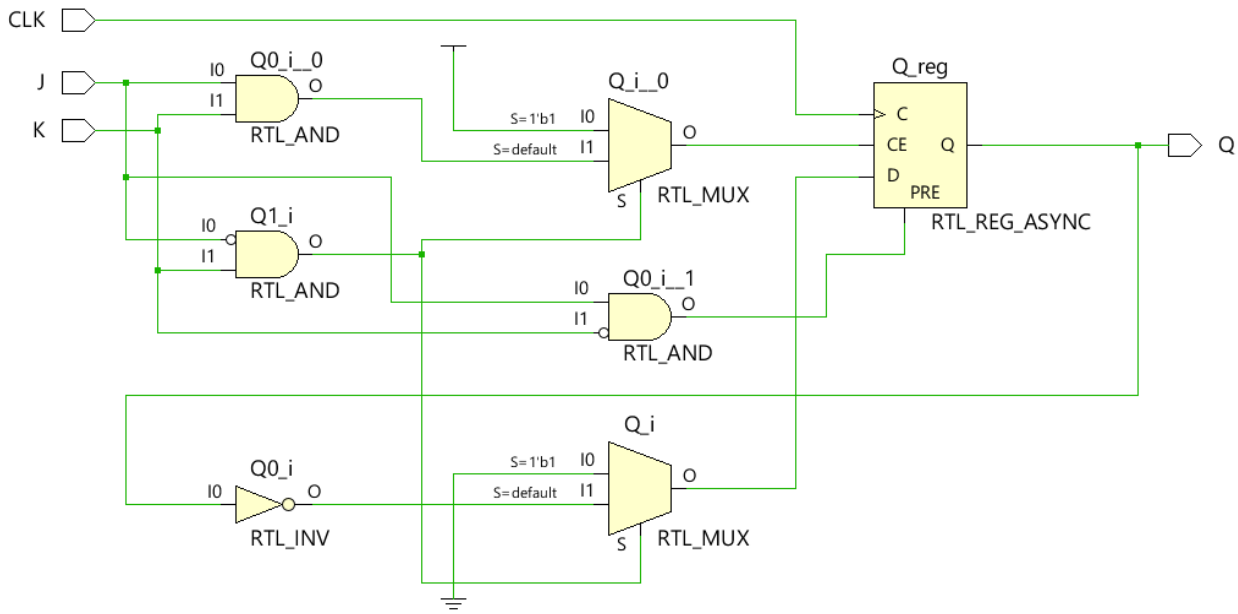


JK_FLIPFLOP

ASYNCHRONOUS

```
module JK_FF(input J,K,CLK, output reg Q);
always @(posedge CLK or negedge K or posedge J)
begin
if(J&&~K)
Q<=1'b1;
else if(~J&&K)
Q<=1'b0;
else if(J&&K)
Q<=~Q;
else
Q<=Q;
end
endmodule
```

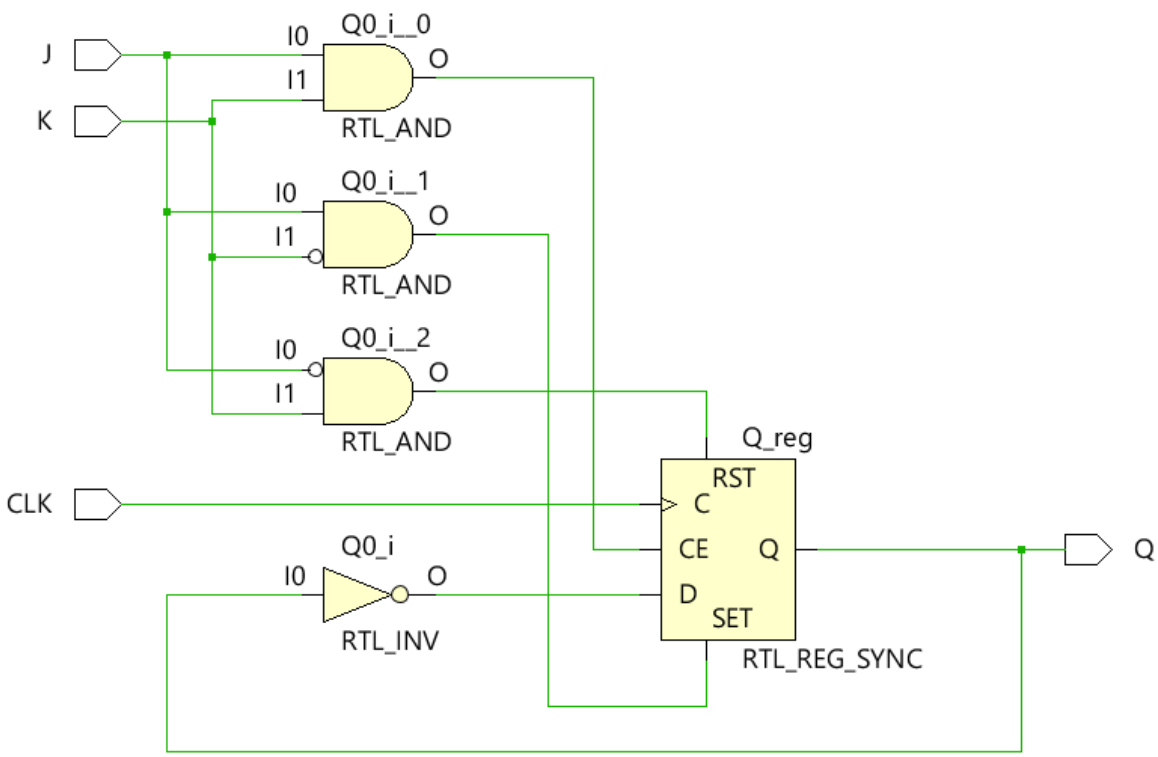
SCHEMATIC

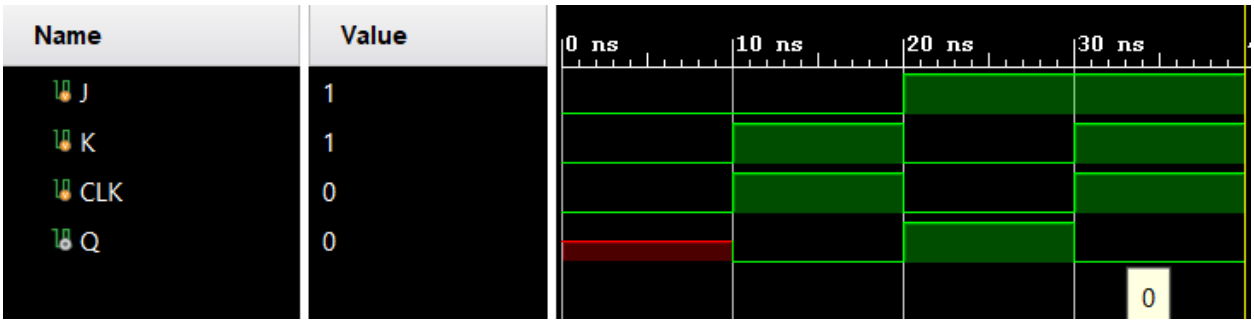


SYNCHRONOUS

```
module JK_FF(input J,K,CLK, output reg Q);
always @(posedge CLK)
begin
if(J&&~K)
Q<=1'b1;
else if(~J&&K)
Q<=1'b0;
else if(J&&K)
Q<=~Q;
else
Q<=Q;
end
endmodule
```

SCHEMATIC



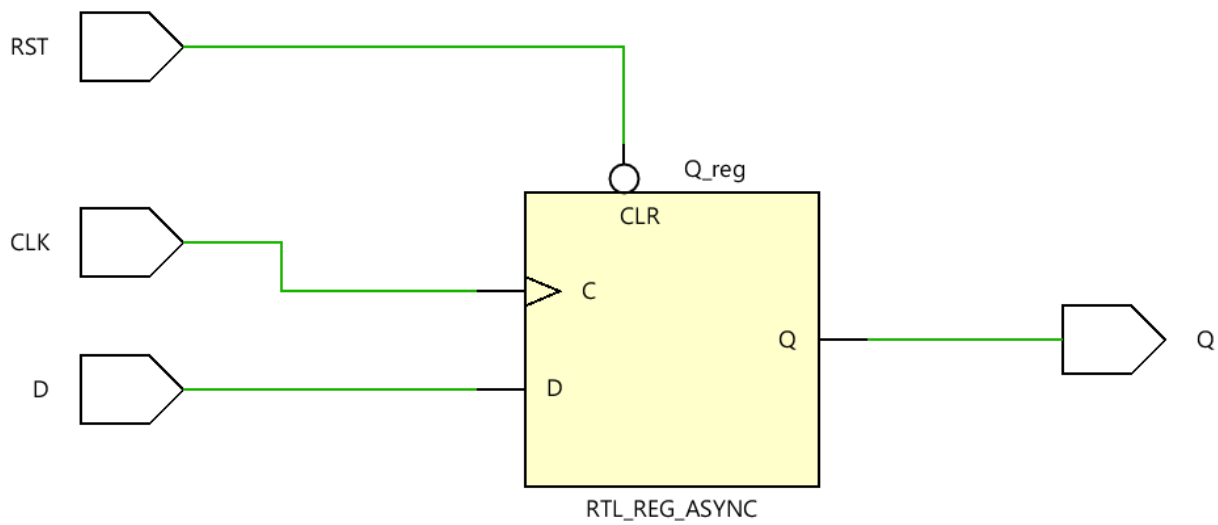


D-FLIPFLOP

ASYNCHRONOUS

```
module D_ff(input CLK,RST,D, output reg Q);
always @(posedge CLK or negedge RST)
begin
if(RST==0)
Q=0;
else
Q=D;
end
endmodule
```

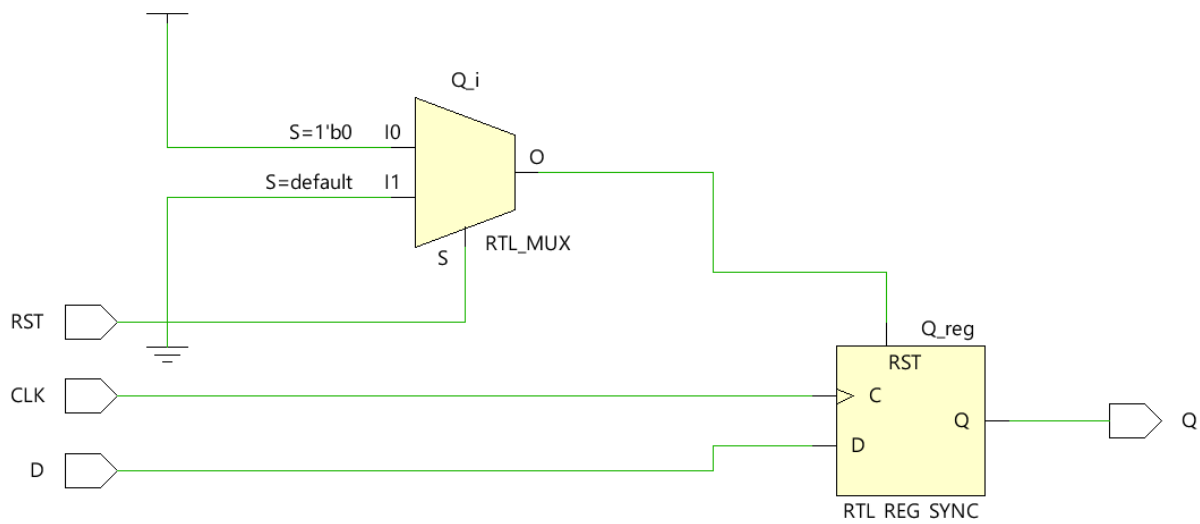
SCHEMATIC

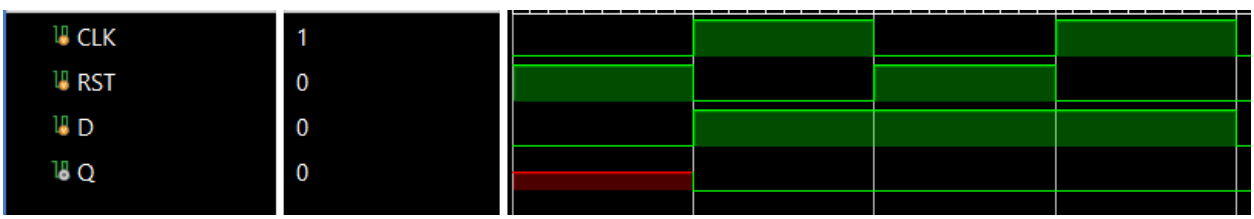


SYNCHRONOUS

```
module D_ff(input CLK,RST,D, output reg Q);
always @(posedge CLK)
begin
if(RST==0)
Q=0;
else
Q=D;
end
endmodule
```

SCHEMATIC

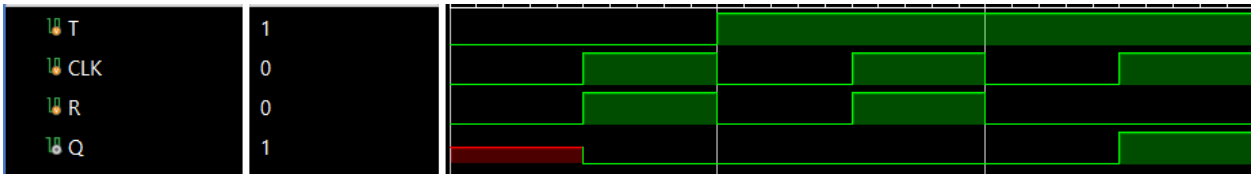


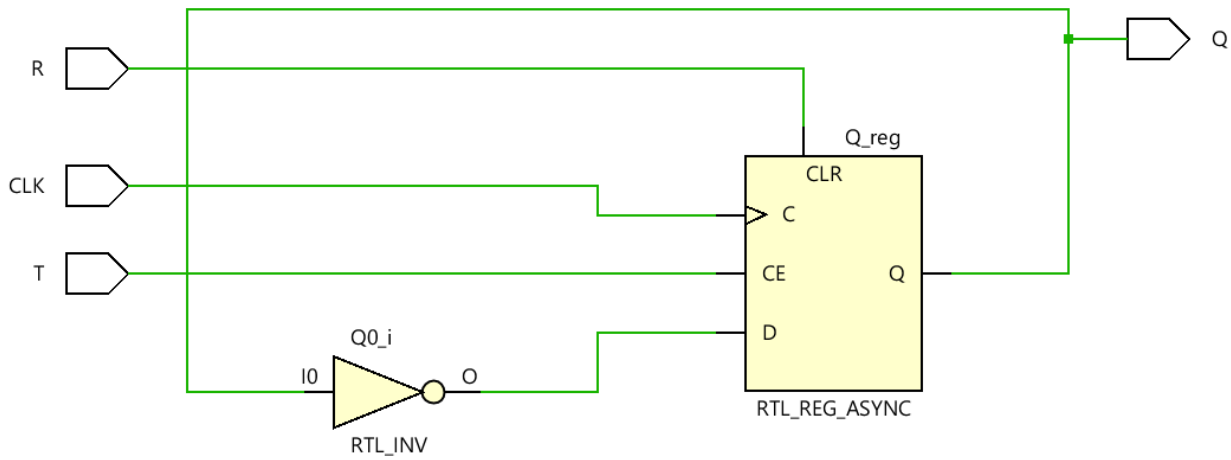


T –FLIPFLOP

ASYNCHRONOUS

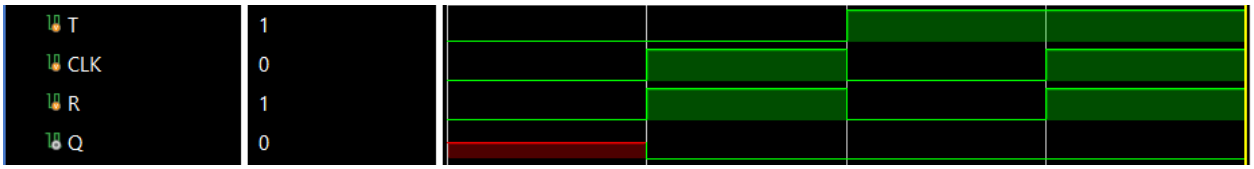
```
module T_FF(input T,CLK,R, output reg Q);
always@(posedge CLK or posedge R)
begin
if(R)
Q<=0;
else if(T)
Q<=~Q;
else
Q<=Q;
end
endmodule
```

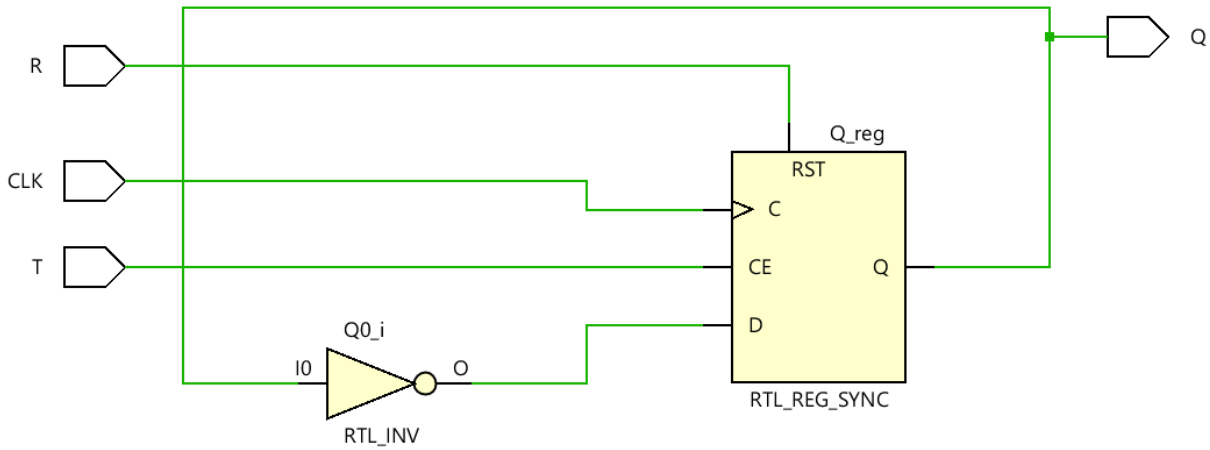




<u>SYNCHRONOUS</u>

```
module T_FF(input T,CLK,R, output reg Q);
always@(posedge CLK)
begin
if(R)
Q<=0;
else if(T)
Q<=~Q;
else
Q<=Q;
end
endmodule
```

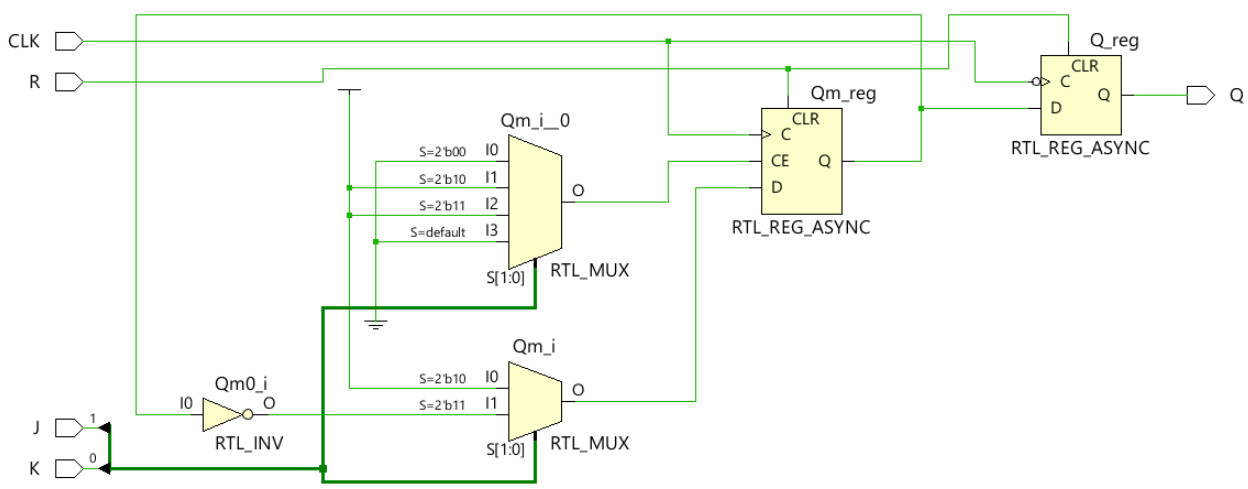


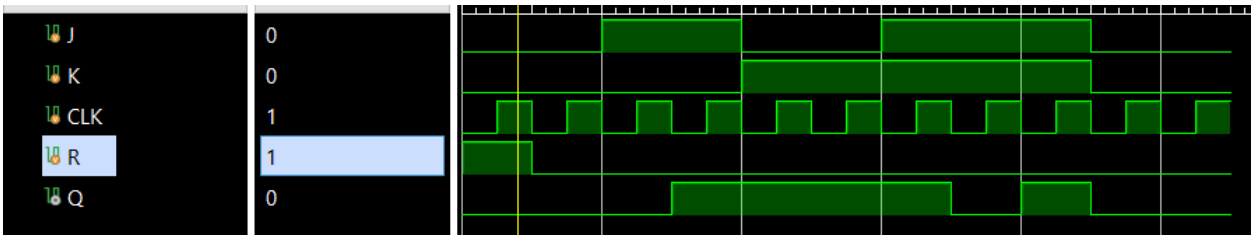


MASTERSLAVE JK

```
module MS_JKFF (
input J, K, CLK, R,
output reg Q
);
reg Qm;
always @(posedge CLK or posedge R) begin
if (R)
Qm <= 0;
else begin
case ({J,K})
2'b00: Qm <= Qm;
2'b10: Qm <= 1;
2'b11: Qm <= ~Qm;
endcase
end
end
always @(negedge CLK or posedge R) begin
if (R)
Q <= 0;
else
Q <= Qm;
end
endmodule
```

SCHEMATIC



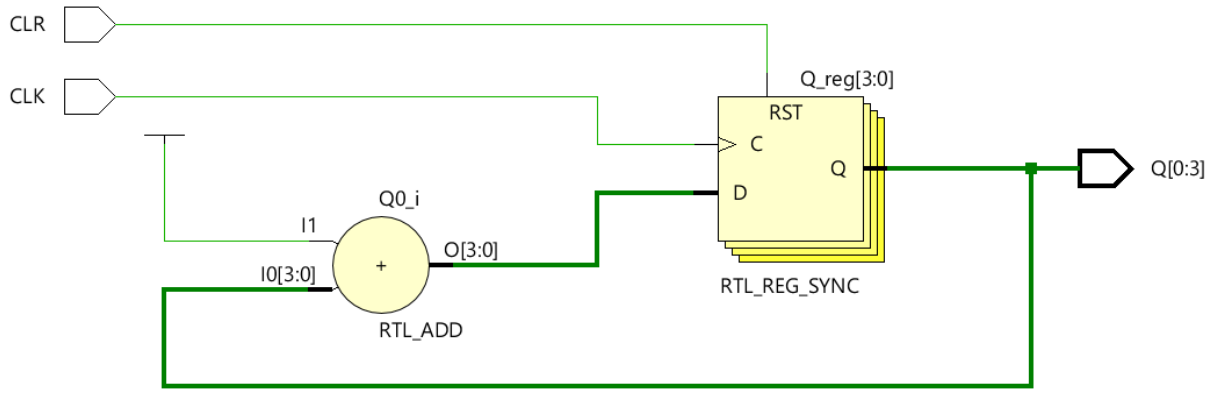


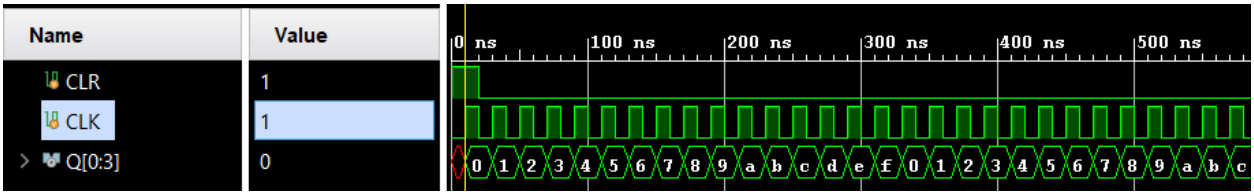
COUNTER

4-Bit COUNTER

```
module Counter(input CLK,CLR,output reg [0:3]Q);
always@(posedge CLK)
begin
if(CLR)
Q=4'b0000;
else
Q=Q+1;
end
endmodule
```

SCHEMATIC

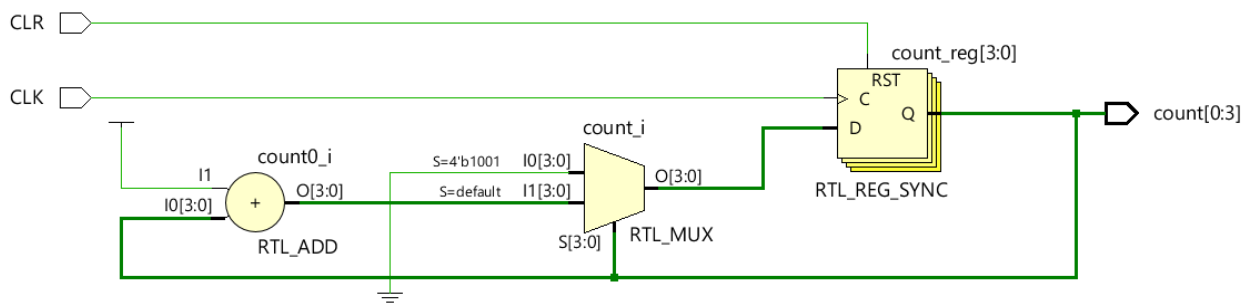


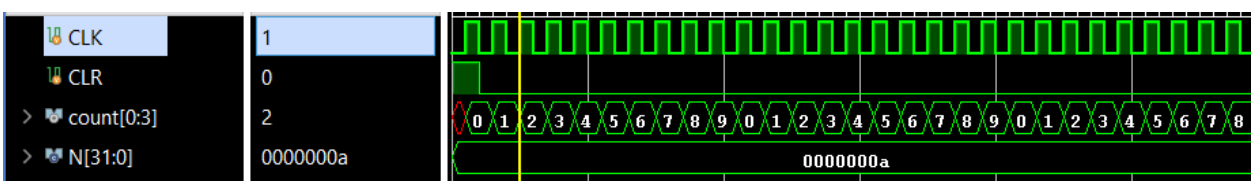


MOD-N COUNTER

```
module modncounter(input CLK,CLR,output reg [0:3]count);
parameter N=10;
always@(posedge CLK)
begin
if(CLR)
count<=0;
else if(count==N-1)
count<=0;
else
count<=count+1;
end
endmodule
```

SCHEMATIC

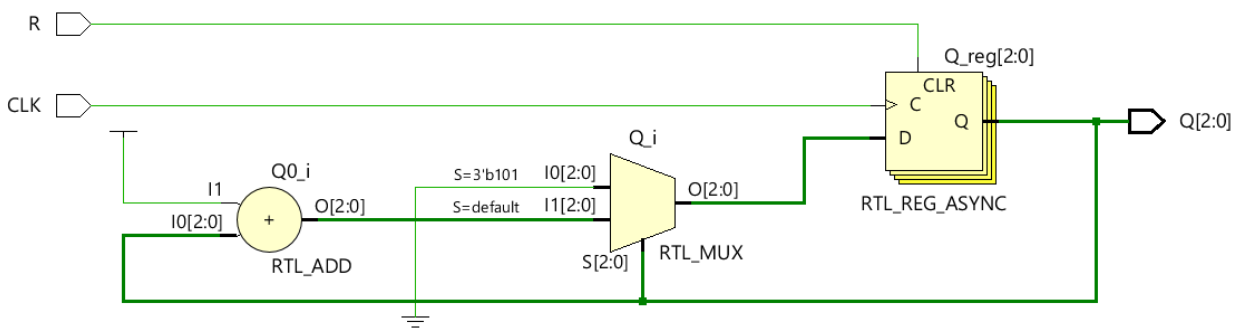


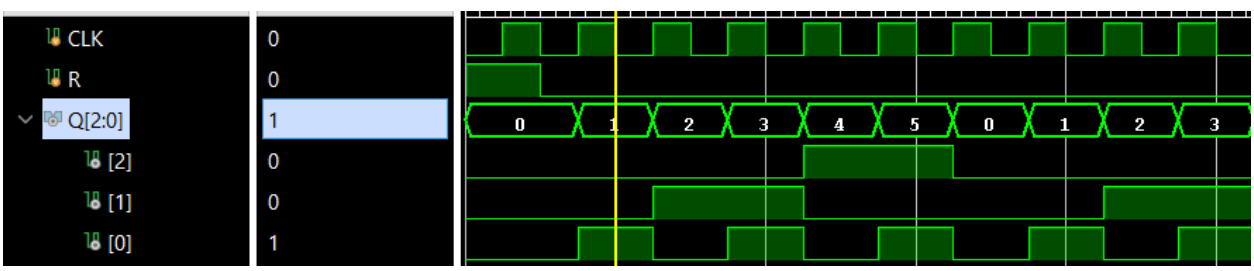


MOD-6(SYNCHRONOUS)

```
module MOD6_SYNC_COUNTER (input CLK, R,output reg [2:0] Q);
always @(posedge CLK or posedge R) begin
if (R)
Q <= 3'b000;
else if (Q == 3'b101)
Q <= 3'b000;
else
Q <= Q + 1;
end
endmodule
```

SCHEMATIC

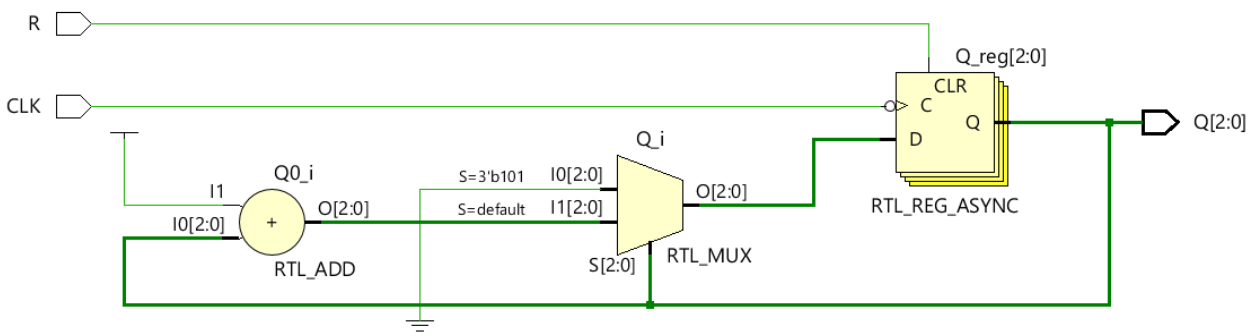


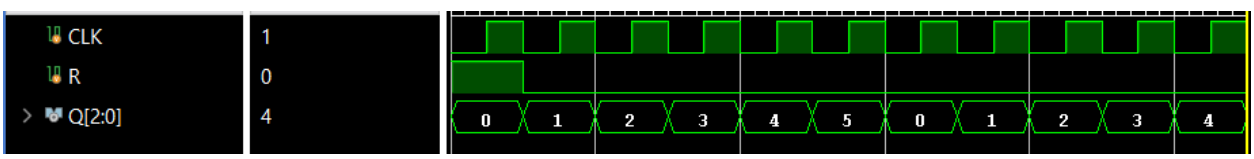


MOD-6(ASYNCHRONOUS)

```
module MOD6_ASYNC_COUNTER (input CLK, R,output reg [2:0] Q);
always @(negedge CLK or posedge R) begin
if (R)
Q <= 3'b000;
else if (Q == 3'b101)
Q <= 3'b000;
else
Q <= Q + 1;
end
endmodule
```

SCHEMATIC

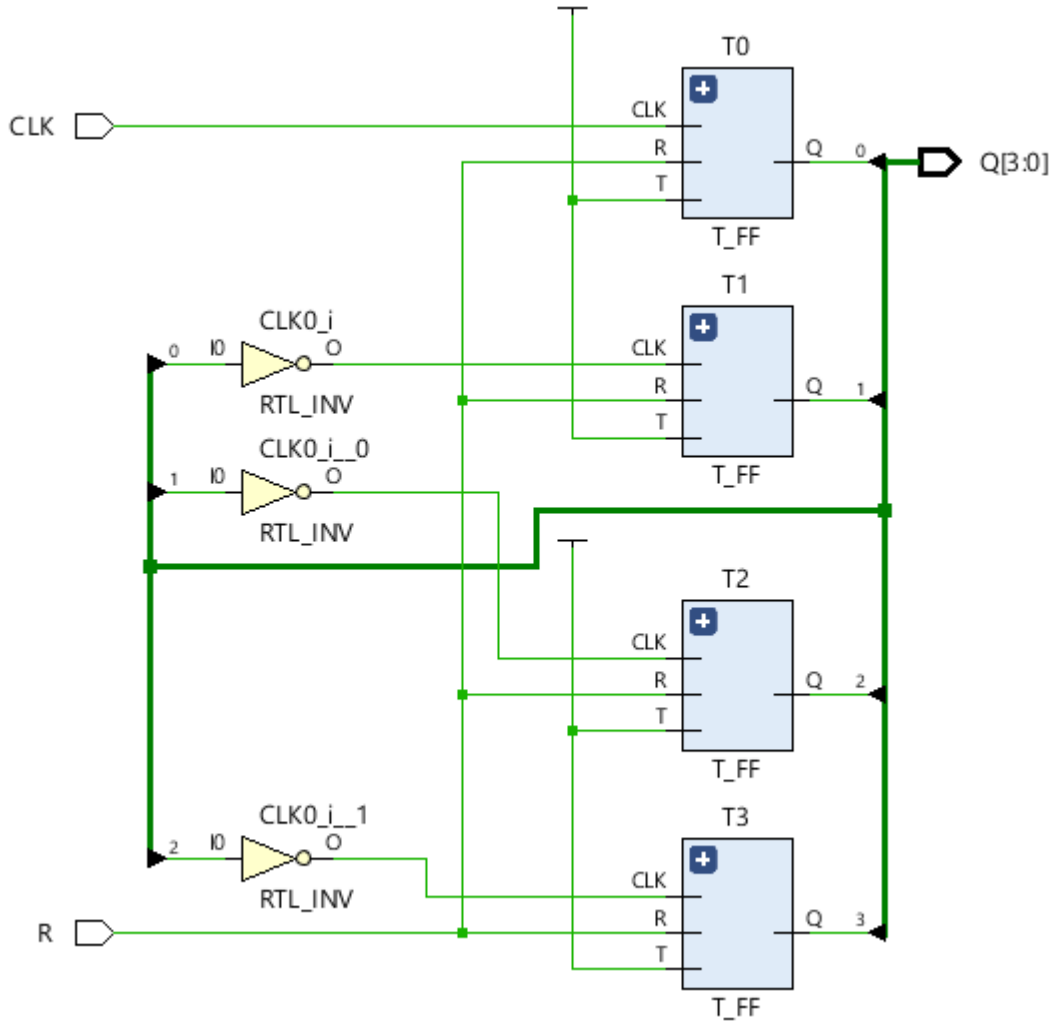


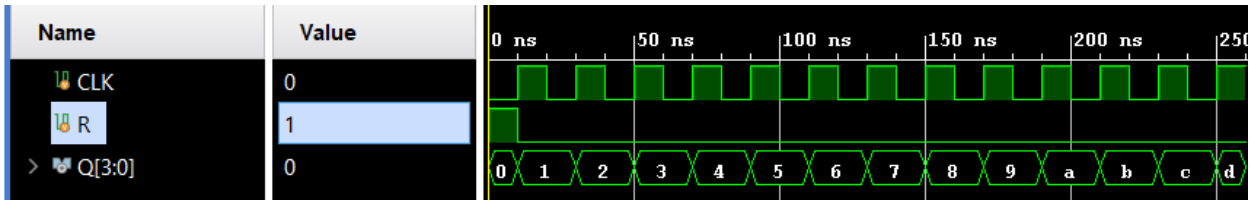


4-BIT RIPPLE COUNTER

```
module T_FF(input T,CLK,R, output reg Q);
always@(posedge CLK or posedge R)
begin
if(R)
Q<=0;
else if(T)
Q<=~Q;
else
Q<=Q;
end
endmodule

module Ripple_counter_4bit(input CLK,R, output [3:0]Q);
T_FF T0(.T(1'b1),.CLK(CLK),.R(R),.Q(Q[0]));
T_FF T1(.T(1'b1),.CLK(~(Q[0])),.R(R),.Q(Q[1]));
T_FF T2(.T(1'b1),.CLK(~(Q[1])),.R(R),.Q(Q[2]));
T_FF T3(.T(1'b1),.CLK(~(Q[2])),.R(R),.Q(Q[3]));
endmodule
```

SCHEMATIC

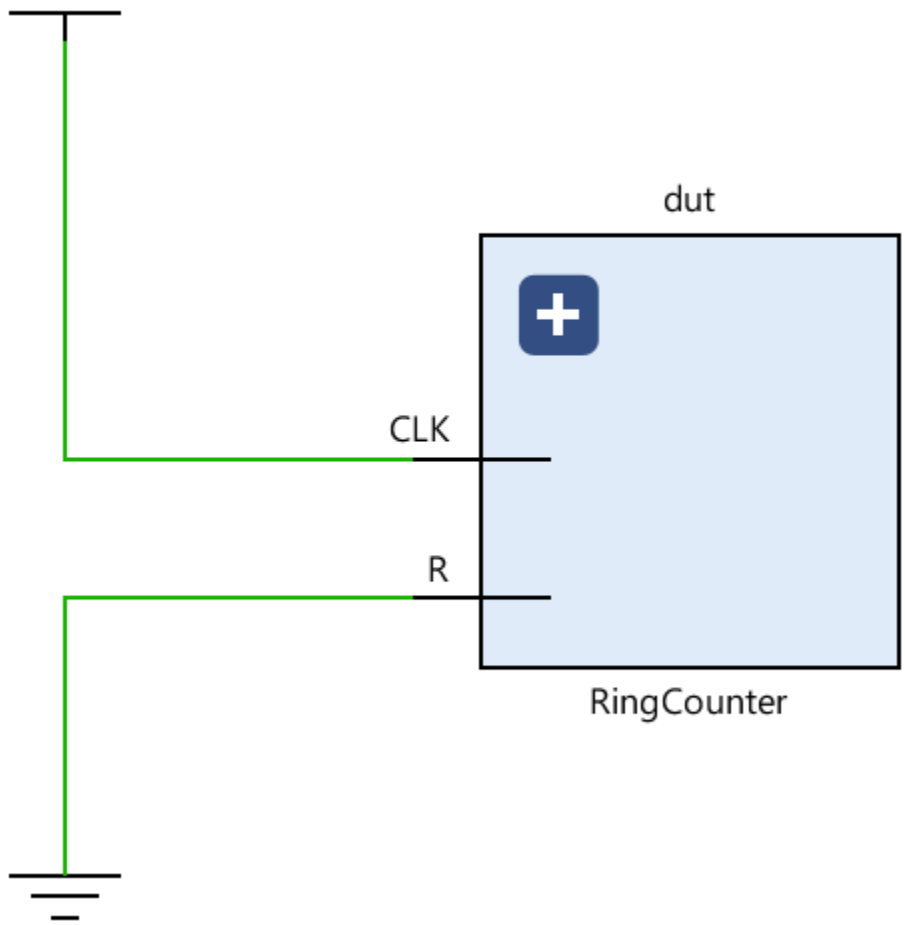


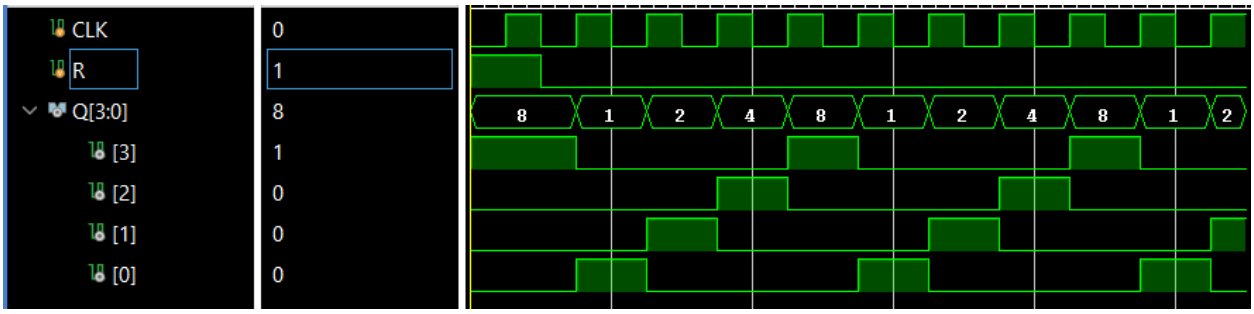


RING-COUNTER

```
module RingCounter #(parameter N=4) (
input CLK, R,
output reg [N-1:0] Q);
always @(posedge CLK or posedge R) begin
if (R)
Q <= 4'b1000;
else
Q <= {Q[N-2:0], Q[N-1]};
end
endmodule
```

SCHEMATIC

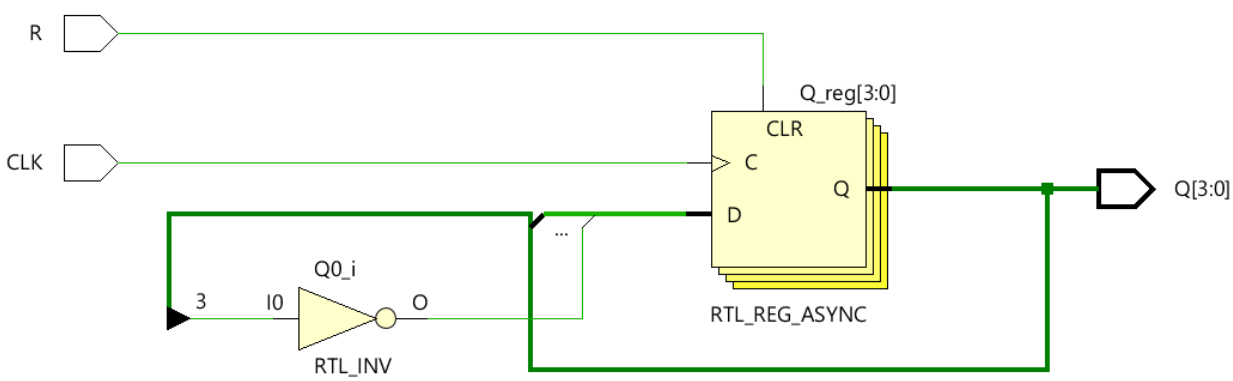


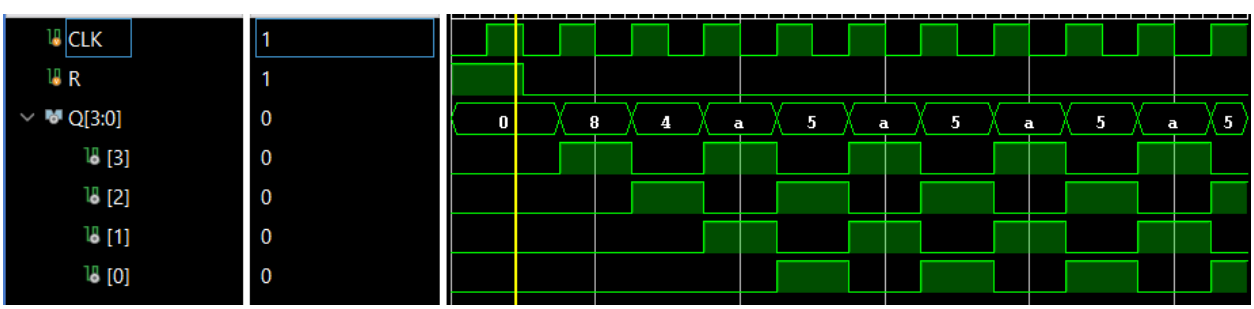


JOHNSON –COUNTER

```
module JohnsonCounter #(parameter N=4) (input CLK, R,output reg [N-1:0] Q);
always @(posedge CLK or posedge R) begin
if (R)
Q <= 4'b0000;
else
Q <= {~Q[N-1], Q[N-1:1]};
end
endmodule
```

SCHEMATIC

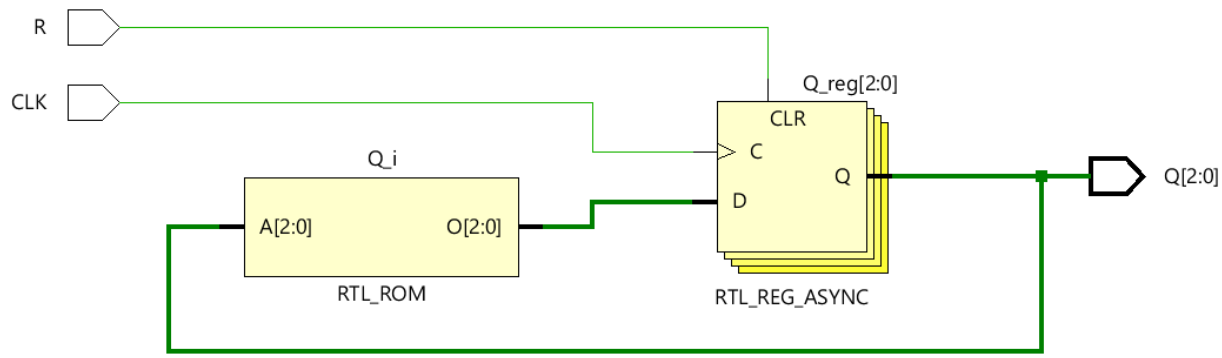


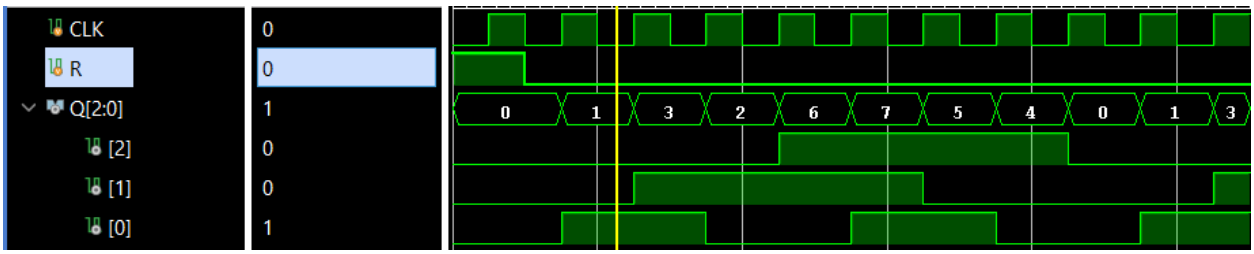


GREY-CODE COUNTER

```
module GrayCodeCounter(input CLK, R,output reg [2:0] Q);
always @(posedge CLK or posedge R) begin
if (R)
Q <= 3'b000;
else
begin
case (Q)
3'b000: Q <= 3'b001;
3'b001: Q <= 3'b011;
3'b011: Q <= 3'b010;
3'b010: Q <= 3'b110;
3'b110: Q <= 3'b111;
3'b111: Q <= 3'b101;
3'b101: Q <= 3'b100;
3'b100:Q <= 3'b000;
default:Q <= 3'b000;
endcase
end
end
endmodule
```

SCHEMATIC

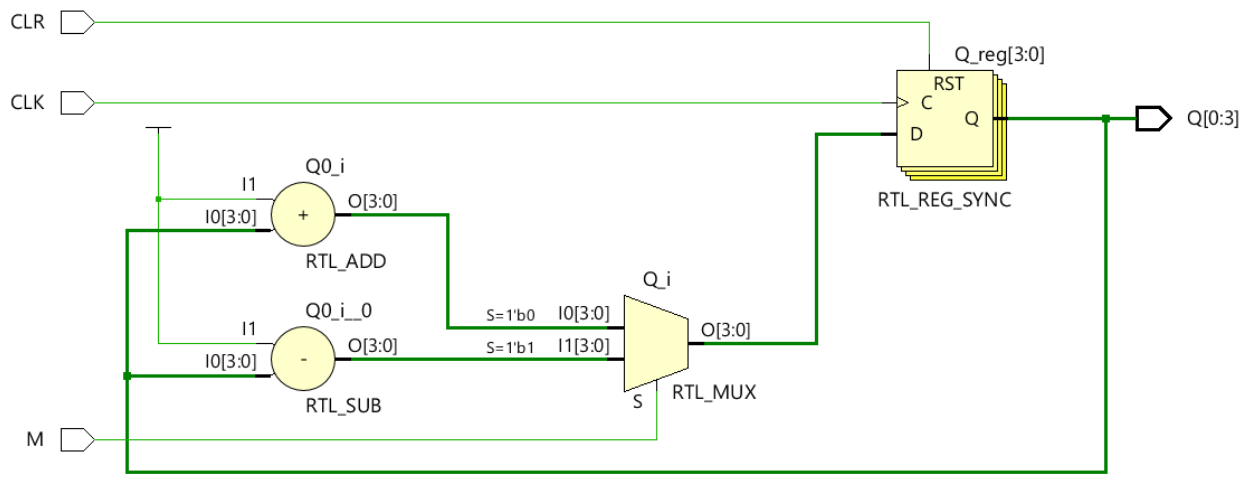


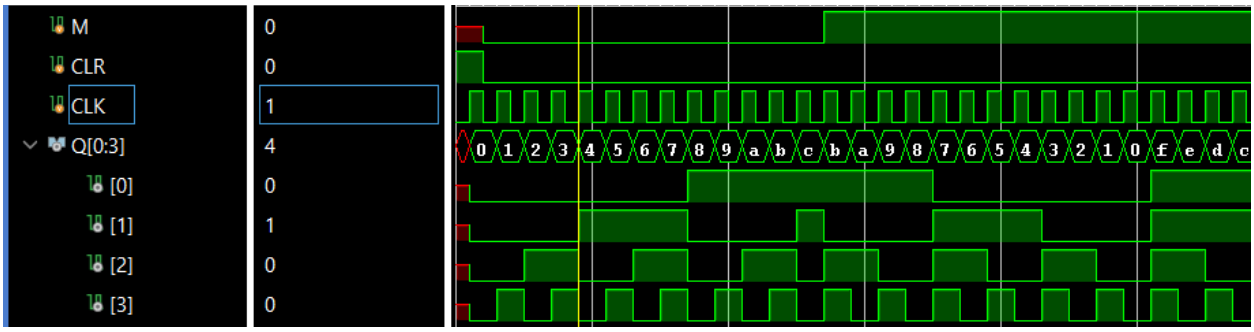


UP-DOWN COUNTER

```
module Up_down_counter(input M,CLR,CLK,output reg [0:3]Q);
always@(posedge CLK)
begin
if(CLR)
Q=0;
else
case(M)
1'b0:Q=Q+1;
1'b1:Q=Q-1;
endcase
end
endmodule
```

SCHEMATIC



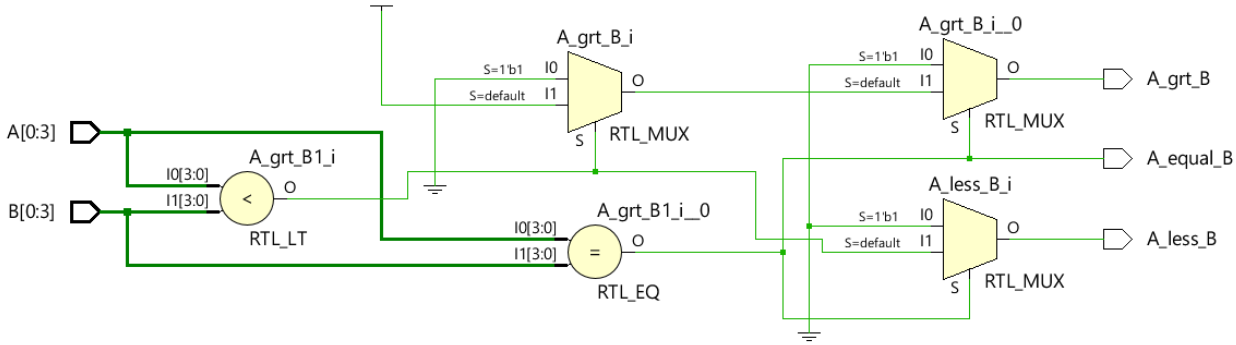


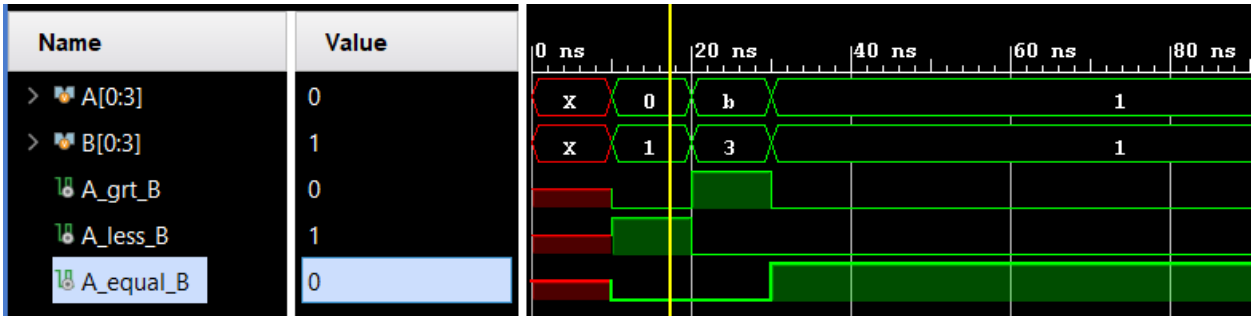
COMPARATOR

4-BIT COMPARATOR

```
module Comp4bit(input [0:3]A,B, output reg A_grt_B,A_less_B,A_equal_B);
always@(*)
begin
if(A==B)
begin
A_grt_B=0;
A_less_B=0;
A_equal_B=1;
end
else if(A<B)
begin
A_less_B=1;
A_grt_B=0;
A_equal_B=0;
end
else
begin
A_less_B=0;
A_grt_B=1;
A_equal_B=0;
end
end
endmodule
```

SCHEMATIC





SHIFT REGISTORS

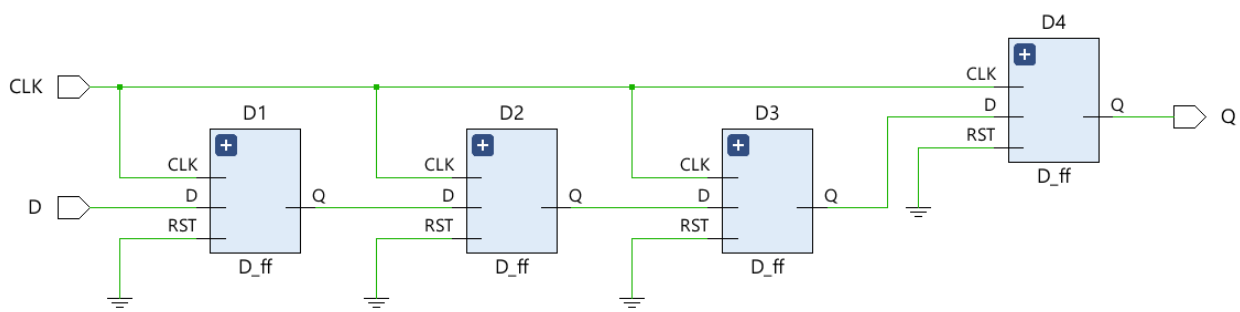
SISO

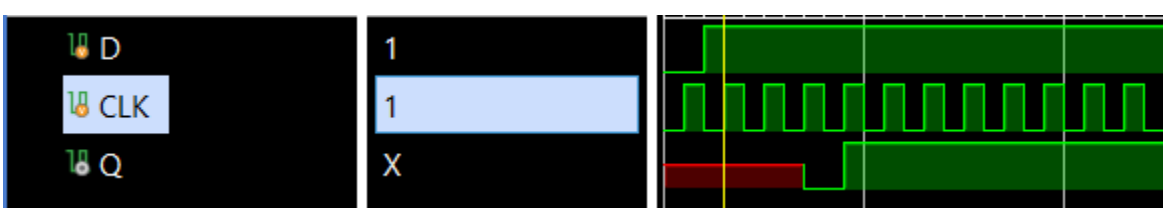
STRUCTURAL

```
module D_ff(input CLK,RST,D, output reg Q);
always @(posedge CLK or negedge RST)
begin
if(RST==0)
Q=0;
else
Q=D;
end
endmodule
```

```
module SISO(input D,CLK,output Q );
wire W1,W2,W3;
D_ff D1(.D(D),.CLK(CLK),.Q(W1));
D_ff D2(.D(W1),.CLK(CLK),.Q(W2));
D_ff D3(.D(W2),.CLK(CLK),.Q(W3));
D_ff D4(.D(W3),.CLK(CLK),.Q(Q));
endmodule
```

SCHEMATIC

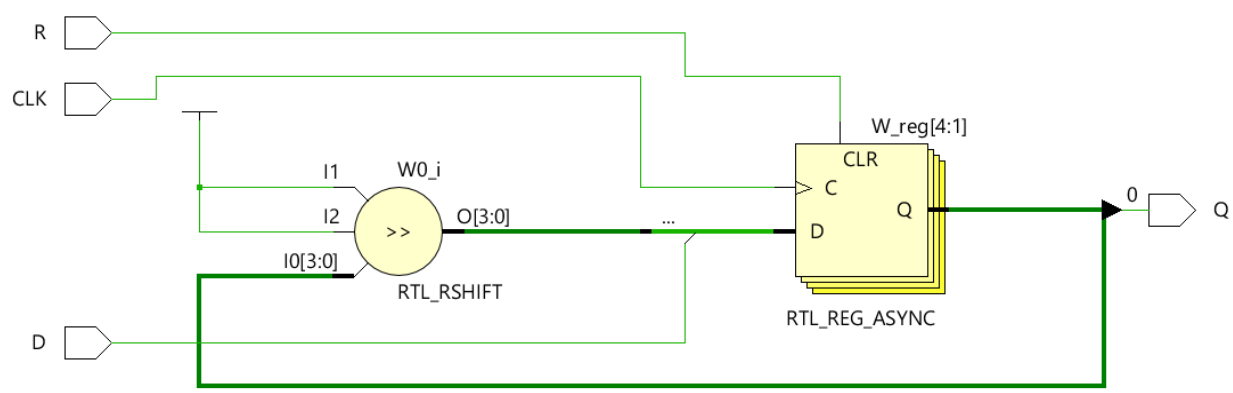


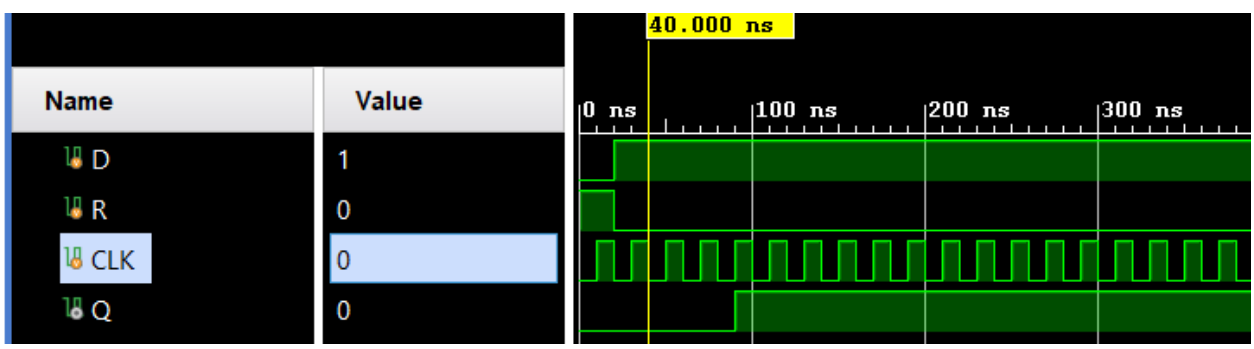


BEHAVIOURAL

```
module SISO(input D,CLK,R, output Q);
reg [1:4]W;
assign Q=W[4];
always@(posedge CLK or posedge R)
begin
if(R)
W<=0;
else
begin
W<=W>>1;
W[1]<=D;
end
end
endmodule
```

SCHEMATIC





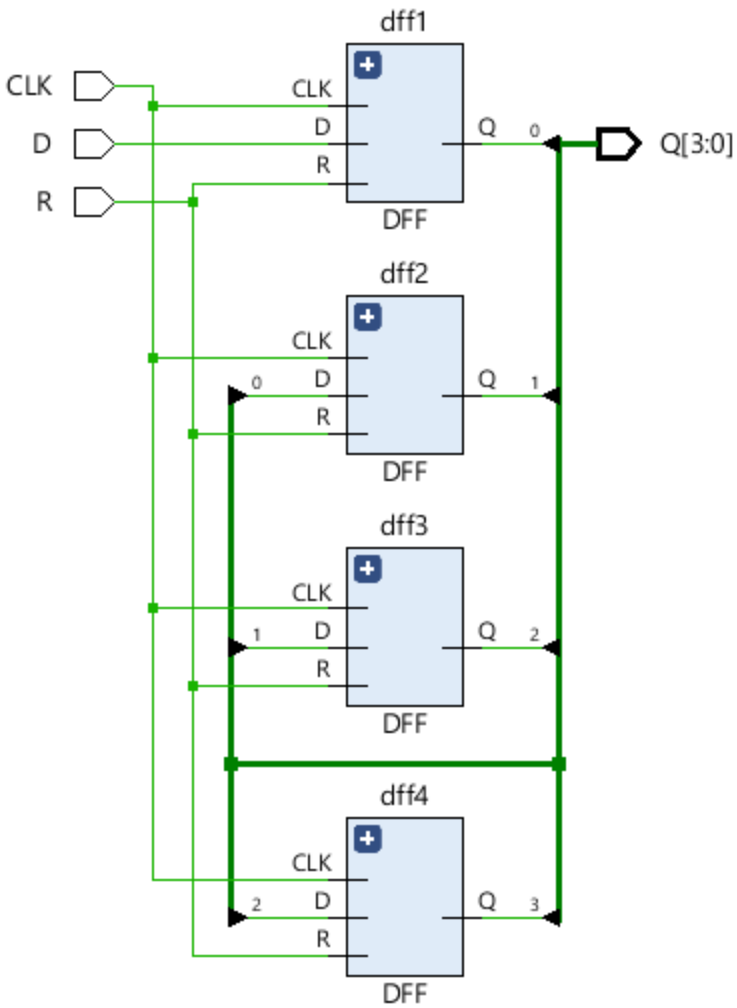
SIPO

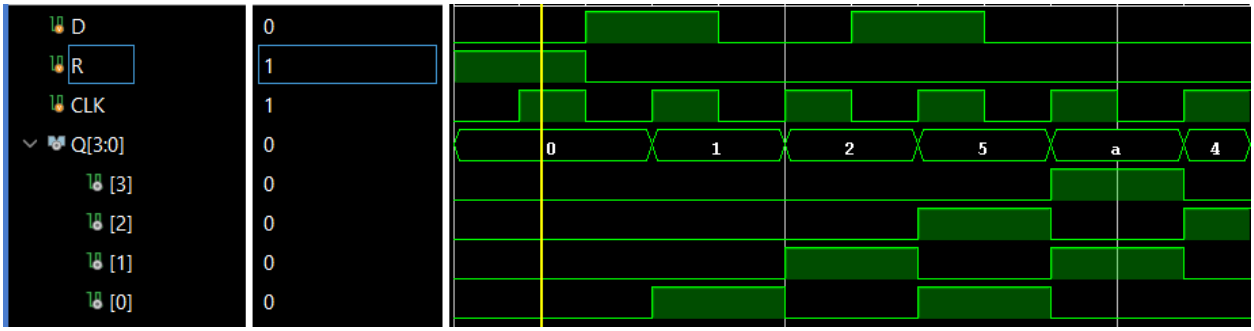
STRUCTURAL

```
module DFF(input D, input CLK, input R, output reg Q);
always @(posedge CLK or posedge R) begin
if (R)
Q <= 0;
else
Q <= D;
end
endmodule
```

```
module SIPO(input D, input CLK, input R, output [3:0] Q);
wire q1, q2, q3, q4;
DFF dff1(.D(D),   .CLK(CLK), .R(R), .Q(q1));
DFF dff2(.D(q1),  .CLK(CLK), .R(R), .Q(q2));
DFF dff3(.D(q2),  .CLK(CLK), .R(R), .Q(q3));
DFF dff4(.D(q3),  .CLK(CLK), .R(R), .Q(q4));
assign Q = {q4, q3, q2, q1};
endmodule
```

SCHEMATIC

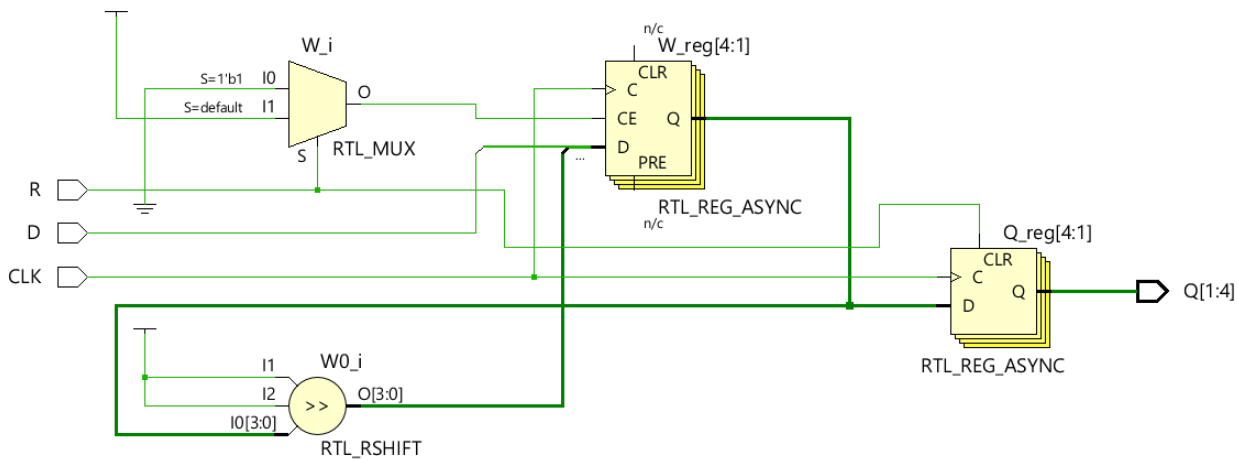


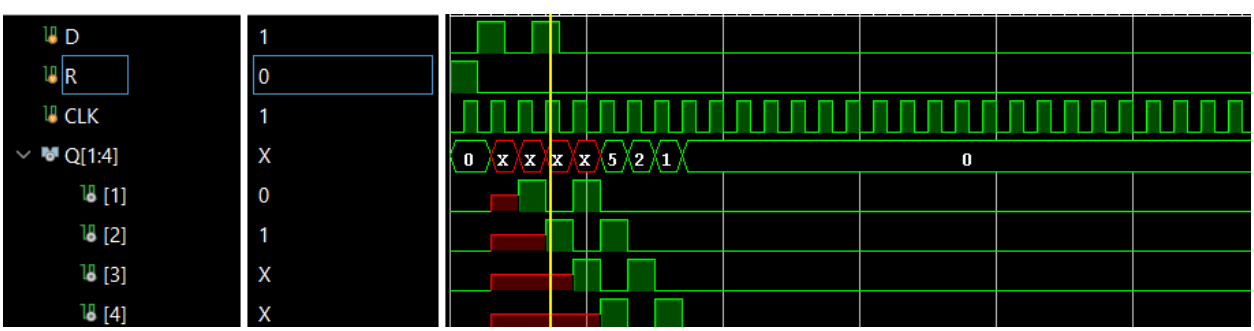


BEHAVIOURAL

```
module SIPO (input D,CLK,R, output reg[1:4]Q);
reg [1:4]W;
always @(posedge CLK or posedge R)
begin
if (R)
Q<=0;
else
begin
W<=W>>1;
W[1]<=D;
Q[1:4]<=W[1:4];
end
end
endmodule
```

SCHEMATIC





PISO

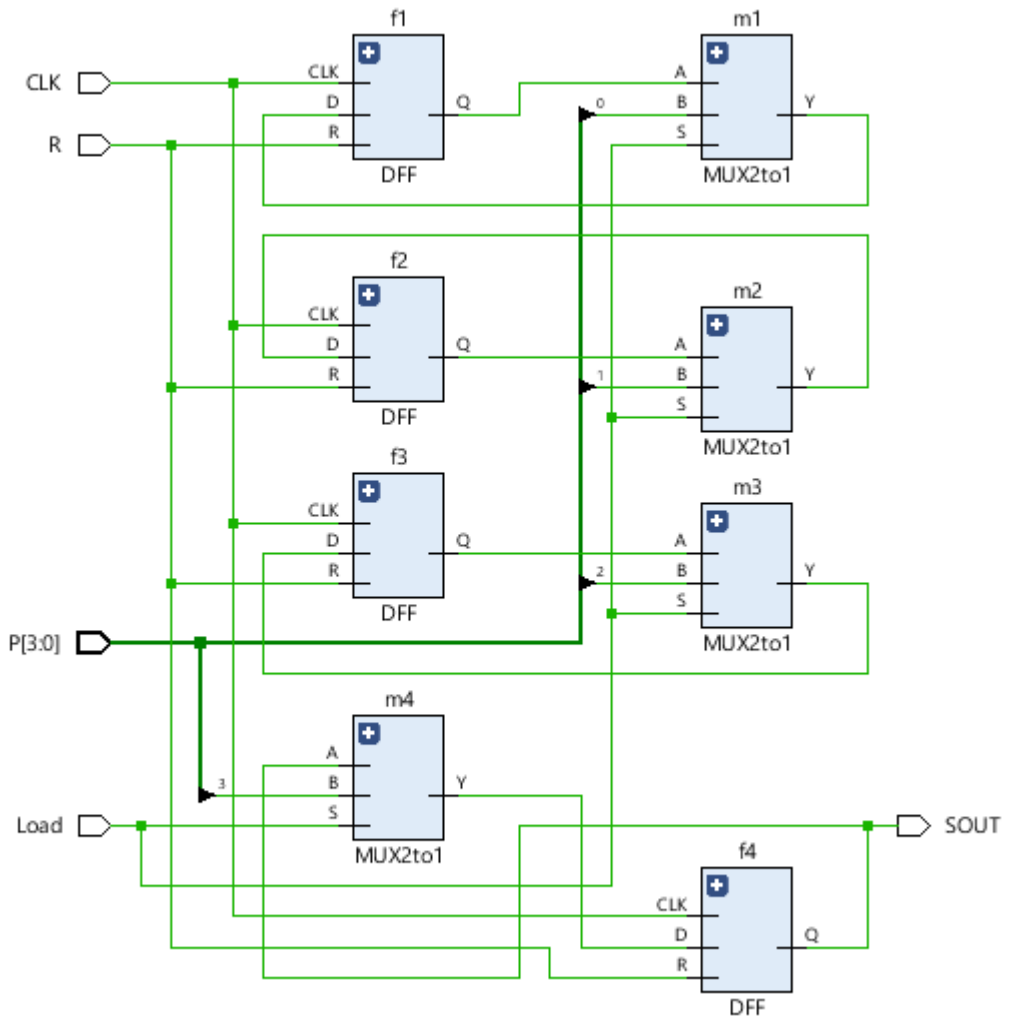
STRUCTURAL

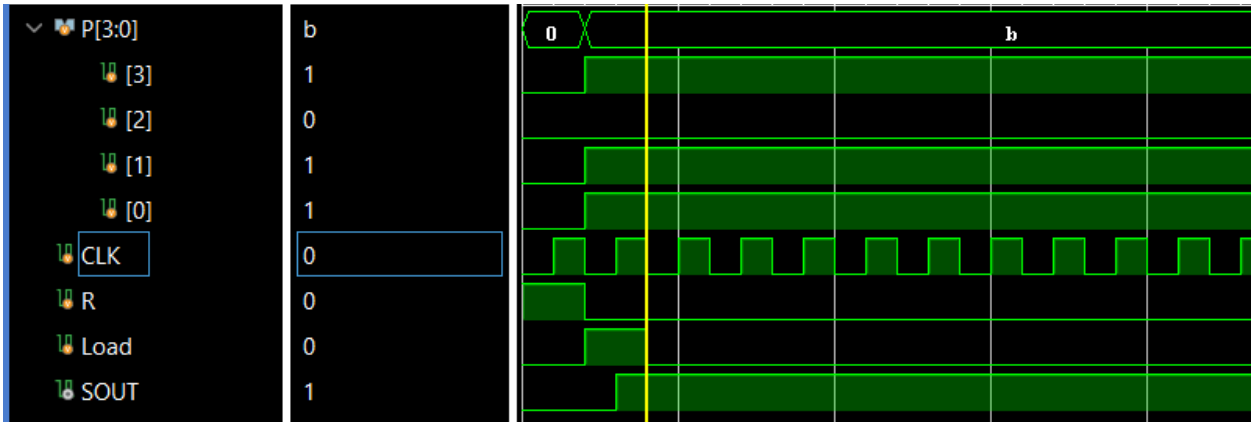
```
module DFF(input D, input CLK, input R, output reg Q);
always @(posedge CLK or posedge R) begin
if (R)
Q <= 0;
else
Q <= D;
end
endmodule
```

```
module MUX2to1(input A, input B, input S, output Y);
assign Y = S ? B : A;
endmodule
```

```
module PISO(input [3:0] P, input CLK, input R, input Load, output SOUT);
wire q1, q2, q3, q4;
wire d1, d2, d3, d4;
MUX2to1 m1(.A(q1), .B(P[0]), .S(Load), .Y(d1));
MUX2to1 m2(.A(q2), .B(P[1]), .S(Load), .Y(d2));
MUX2to1 m3(.A(q3), .B(P[2]), .S(Load), .Y(d3));
MUX2to1 m4(.A(q4), .B(P[3]), .S(Load), .Y(d4));
DFF f1(.D(d1), .CLK(CLK), .R(R), .Q(q1));
DFF f2(.D(d2), .CLK(CLK), .R(R), .Q(q2));
DFF f3(.D(d3), .CLK(CLK), .R(R), .Q(q3));
DFF f4(.D(d4), .CLK(CLK), .R(R), .Q(q4));
assign SOUT = q4;
endmodule
```

SCHEMATIC

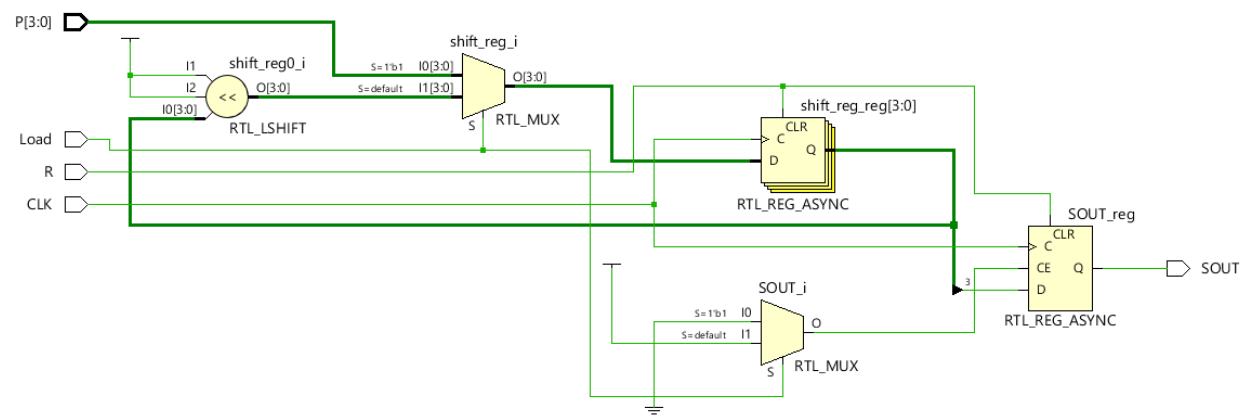


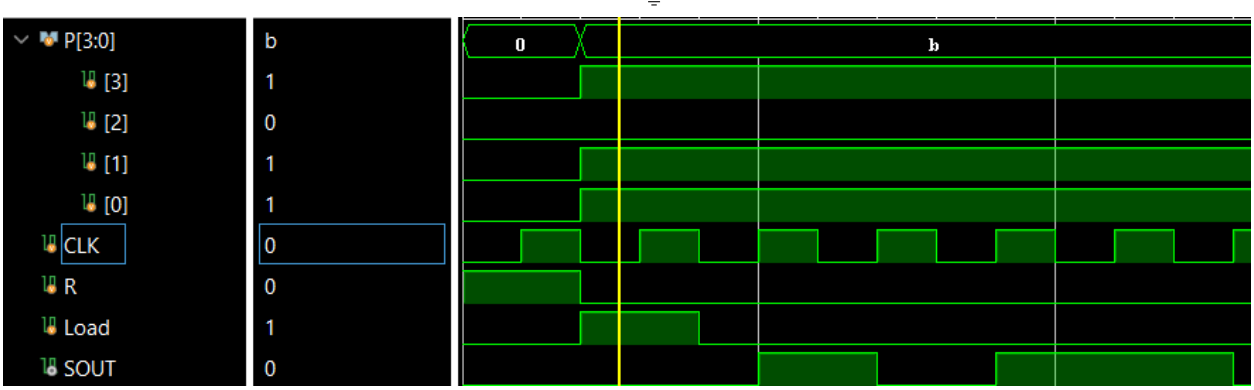


BEHAVIOURAL

```
module PISO(input [3:0] P, input CLK, input R, input Load, output reg SOUT);
reg [3:0] shift_reg;
always @(posedge CLK or posedge R) begin
if (R) begin
shift_reg <= 4'b0000;
SOUT <= 0;
end
else if (Load) begin
shift_reg <= P;
end
else begin
SOUT <= shift_reg[3];
shift_reg <= shift_reg << 1;
end
end
endmodule
```

SCHEMATIC

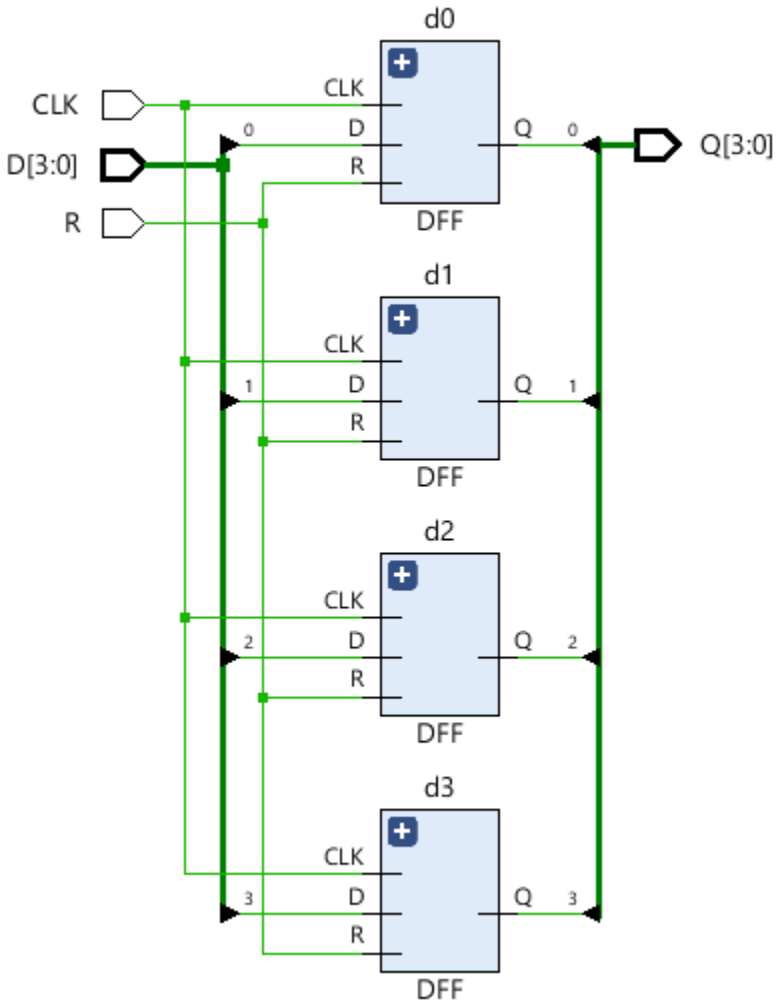


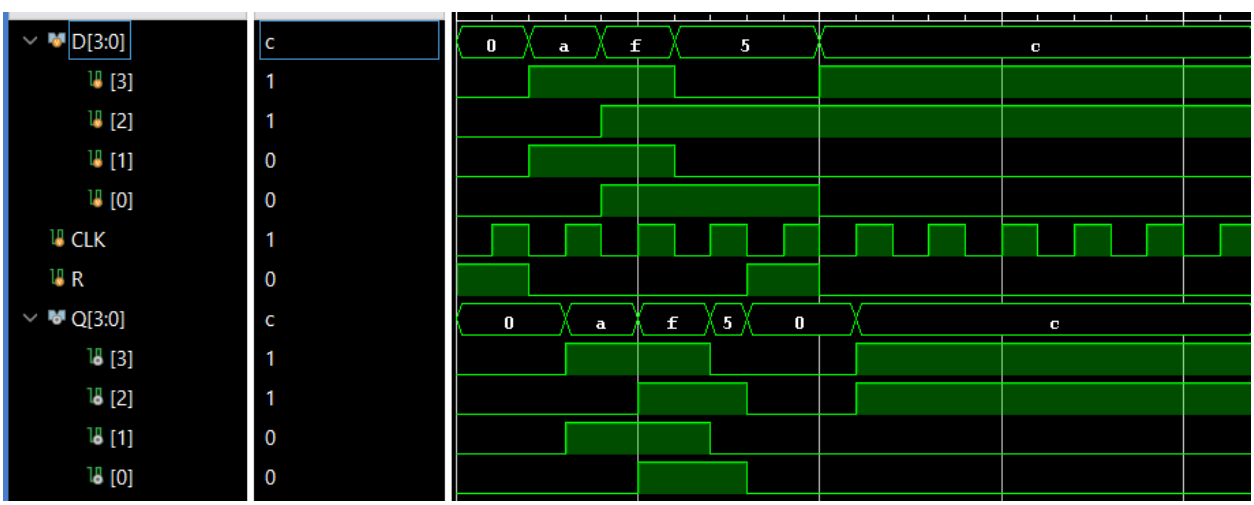


STRUCTURAL

```
module DFF(input D, input CLK, input R, output reg Q);
always @(posedge CLK or posedge R) begin
if (R)
Q <= 0;
else
Q <= D;
end
endmodule

module PIPO(input [3:0] D,input CLK,R,output [3:0] Q);
DFF d0(D[0],CLK,R,Q[0]);
DFF d1(D[1],CLK,R,Q[1]);
DFF d2(D[2],CLK,R,Q[2]);
DFF d3(D[3],CLK,R,Q[3]);
endmodule
```

SCHEMATIC


d0
CLK
D[3:0]
R
CLK
D
R
Q
0
Q[3:0]
DFF
d1
CLK
D
R
Q
1
DFF
d2
CLK
D
R
Q
2
DFF
d3
CLK
D
R
Q
3
DFF

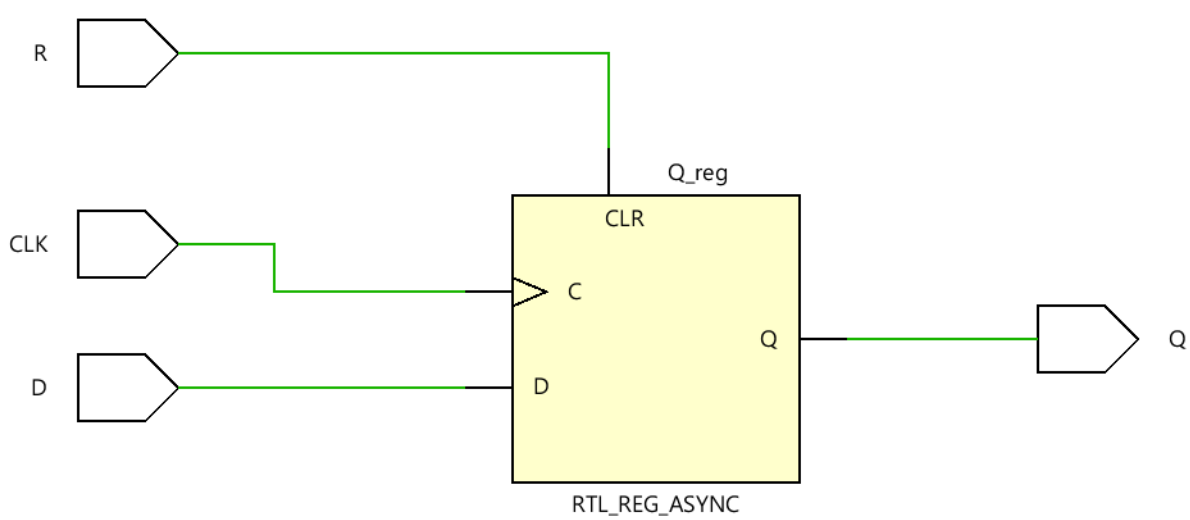

D[3:0]
[3]
[2]
[1]
[0]
CLK
R
Q[3:0]
[3]
[2]
[1]
[0]
c
1
1
0
0
1
0
c
1
1
0
0
0
a
f
5
c
0
a
f
5
0
c

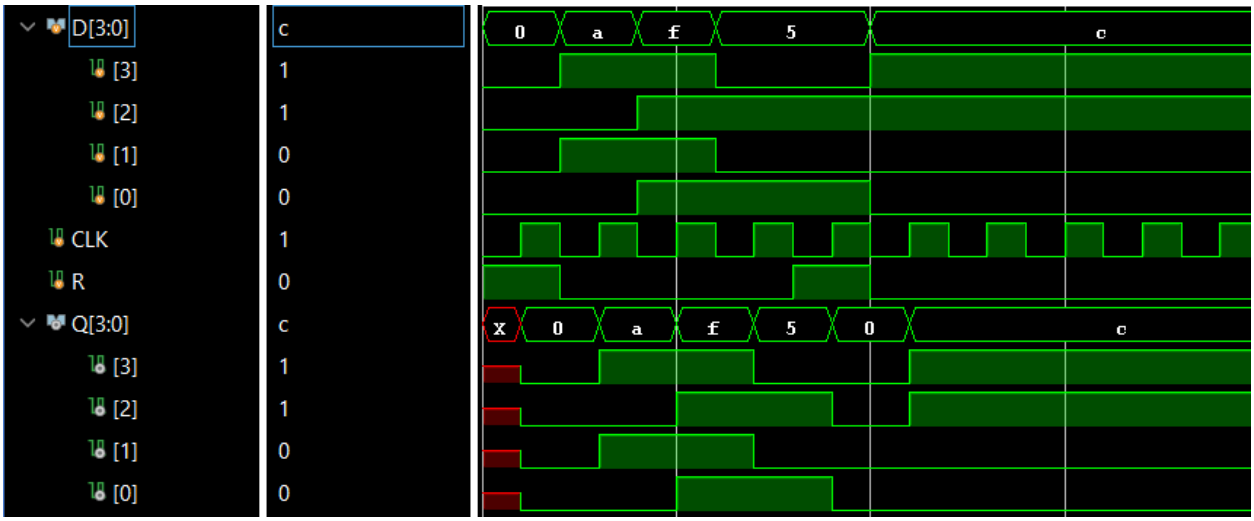
BEHAVIOURAL

```
module DFF(input D, input CLK, input R, output reg Q);
always @(posedge CLK or posedge R) begin
if (R)
Q <= 0;
else
Q <= D;
end
endmodule
```

```
module pipo (input [3:0] d,input clk,r,output reg [3:0] q);
always @(posedge clk)
begin
if (r)
q <= 4'b0000;
else
q<= d;
end
endmodule
```

SCHEMATIC

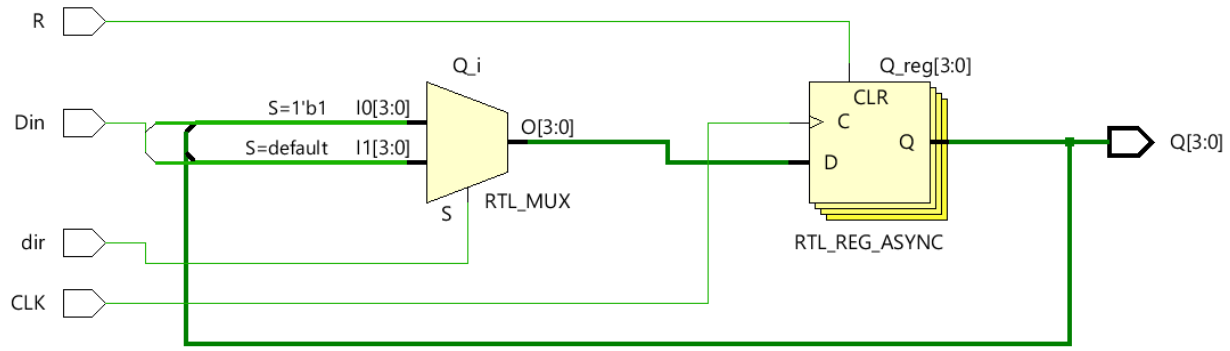


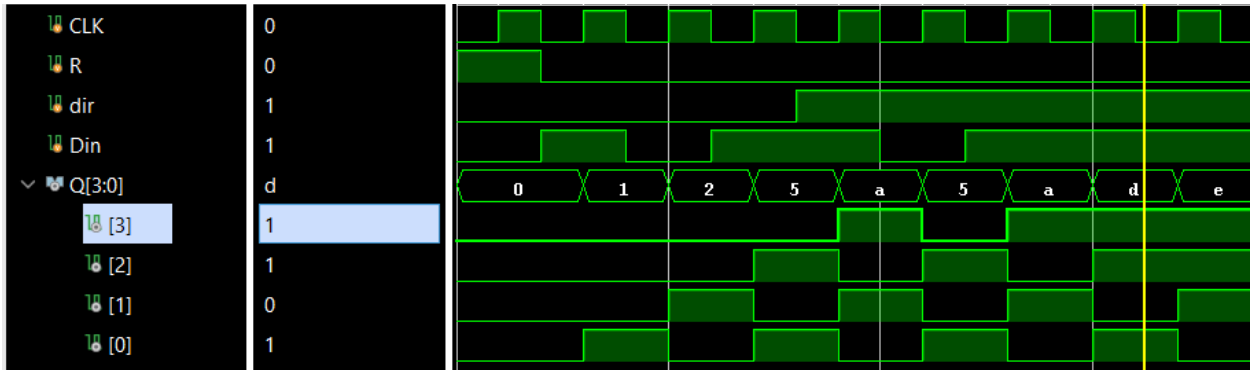


BIDIRECTIONAL SHIFT REGISTER

```
module BiDirectionalShiftRegister_tb;
reg CLK=0, R, dir, Din;
wire [3:0] Q;
always #10 CLK = ~CLK;
BiDirectionalShiftRegister dut(.CLK(CLK), .R(R), .dir(dir), .Din(Din), .Q(Q));
initial
begin
R = 1; dir = 0; Din = 0;
#20 R = 0;
dir = 0;
Din = 1;
#20;Din = 0;
#20;Din = 1;
#20;dir = 1; Din = 1;
#20;Din = 0;
#20;Din = 1;
#20;
#50 $finish;
end
endmodule
```

SCHEMATIC

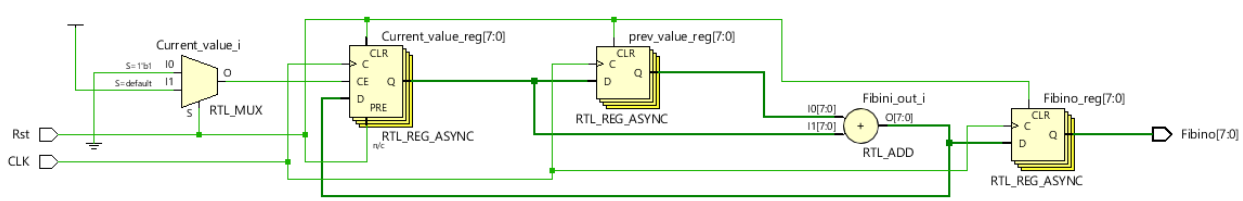

R
Din
dir
CLK
Q_i
S=1'b1
I0[3:0]
S=default
I1[3:0]
O[3:0]
S
RTL_MUX
Q_reg[3:0]
CLR
C
D
Q
RTL_REG_ASYNC
Q[3:0]

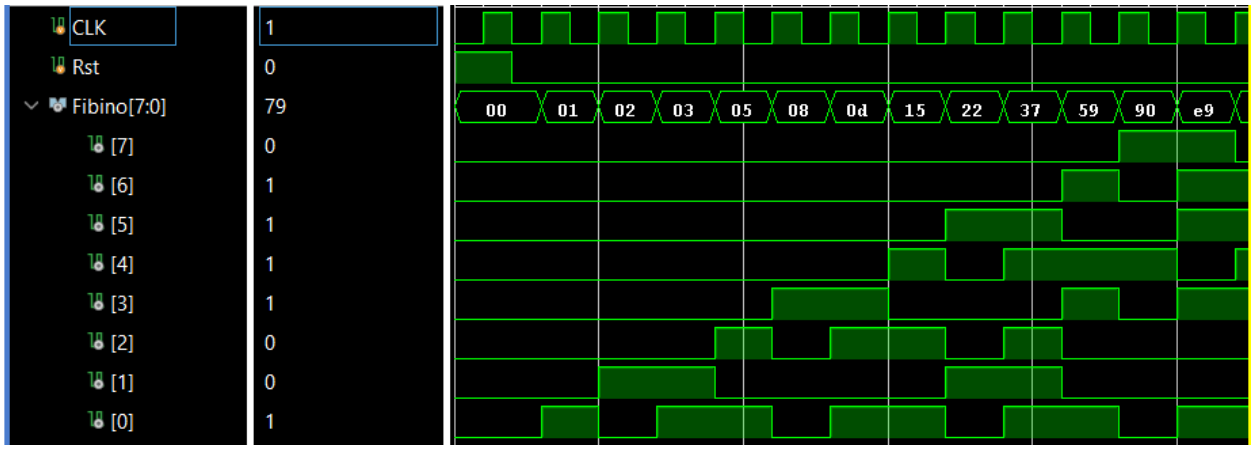

CLK 0
R 0
dir 1
Din 1
Q[3:0] d
[3] 1
[2] 1
[1] 0
[0] 1
0 1 2 5 a 5 a d e

FIBONACCI

```
module Fibonacci(input CLK,Rst, output reg [7:0]Fibino);
reg [7:0] prev_value,Current_value;
task fib ;
input[7:0] prev,current;
output [7:0]Fibini;
begin
Fibini=prev+current;
end
endtask
always@(posedge CLK or posedge Rst)
begin
if(Rst==1)
begin
prev_value<=0;
Current_value<=1;
Fibino<=0;
end
else
begin
prev_value<=Current_value;
fib(prev_value,Current_value,Current_value);
Fibino<=Current_value;
end
end
endmodule
```

SCHEMATIC

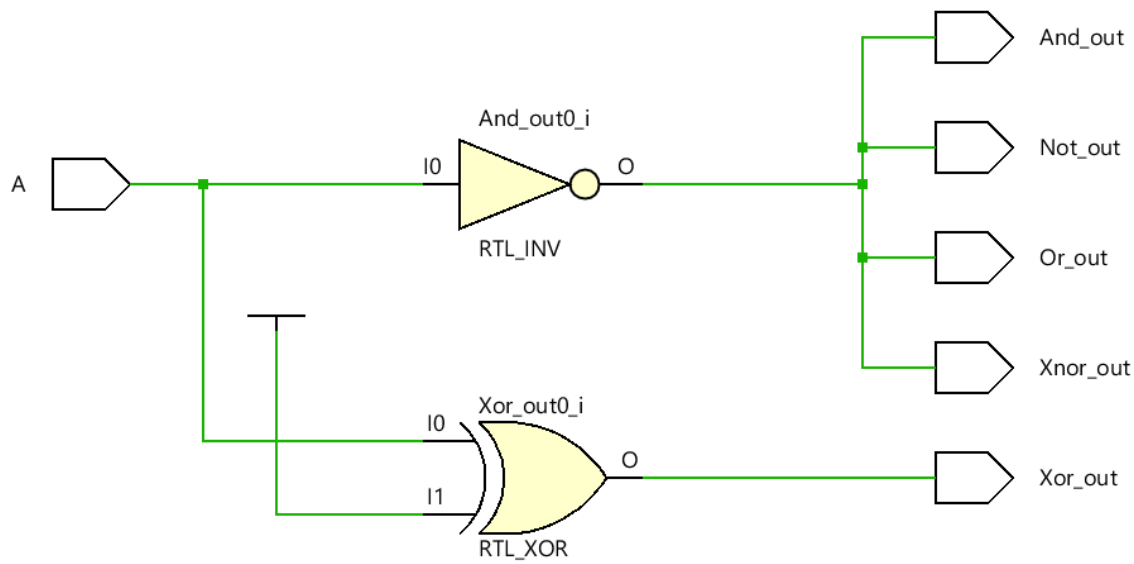


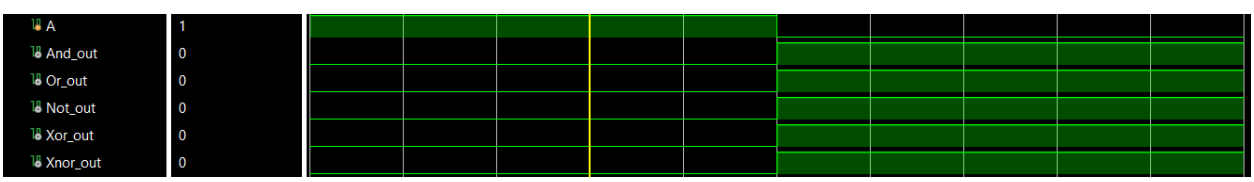

CLK
Rst
Fibino[7:0]
[7]
[6]
[5]
[4]
[3]
[2]
[1]
[0]
1
0
79
0
1
1
1
1
0
0
1
00
01
02
03
05
08
0d
15
22
37
59
90
e9

INVERTOR USING ALL GATES

```
module INVERTOR(input A, output reg And_out,Or_out,Not_out,Xor_out,Xnor_out);
always@(*)
begin
And_out=(~A)&1;
Or_out=(~A)|0;
Not_out=~A;
Xor_out=A^1;
Xnor_out=~(A^0);
end
endmodule
```

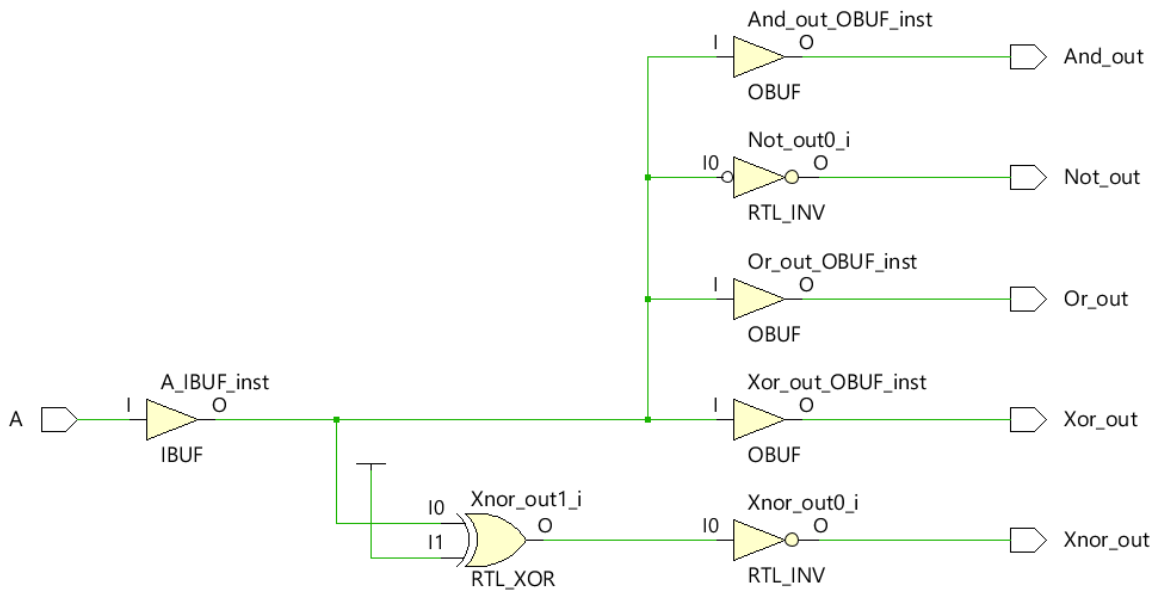
SCHEMATIC

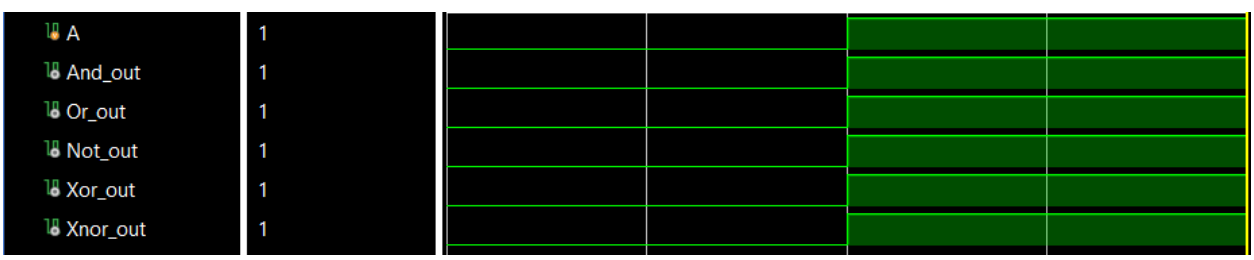




BUFFER USING ALL GATES

```
module Buffer(input A, output reg And_out,Or_out,Not_out,Xor_out,Xnor_out);
always@(*)
begin
And_out=A&1;
Or_out=A|0;
Not_out=~(~A);
Xor_out=A^0;
Xnor_out=~(A^1);
end
endmodule
```

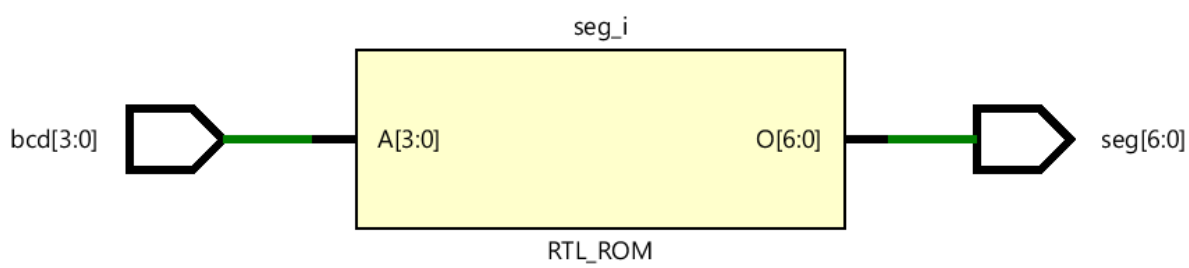


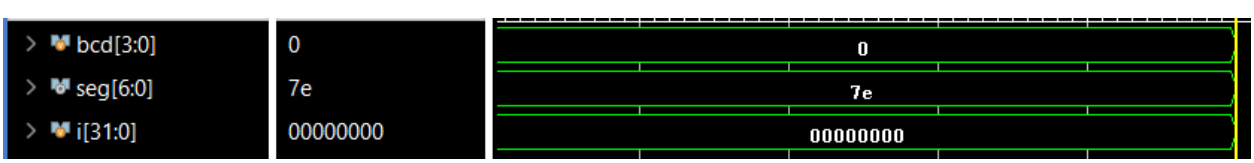


BCD TO 7SEGMENT

```
module bcd_to_7seg (
    input [3:0] bcd,
    output reg  [6:0] seg
);


always @(*) begin
    case (bcd)
        4'b0000: seg = 7'b1111110;
        4'b0001: seg = 7'b0110000;
        4'b0010: seg = 7'b1101101;
        4'b0011: seg = 7'b1111001;
        4'b0100: seg = 7'b0110011;
        4'b0101: seg = 7'b1011011;
        4'b0110: seg = 7'b1011111;
        4'b0111: seg = 7'b1110000;
        4'b1000: seg = 7'b1111111;
        4'b1001: seg = 7'b1111011;
        default: seg = 7'b0000001;
    endcase
end

endmodule
```

SCHEMATIC

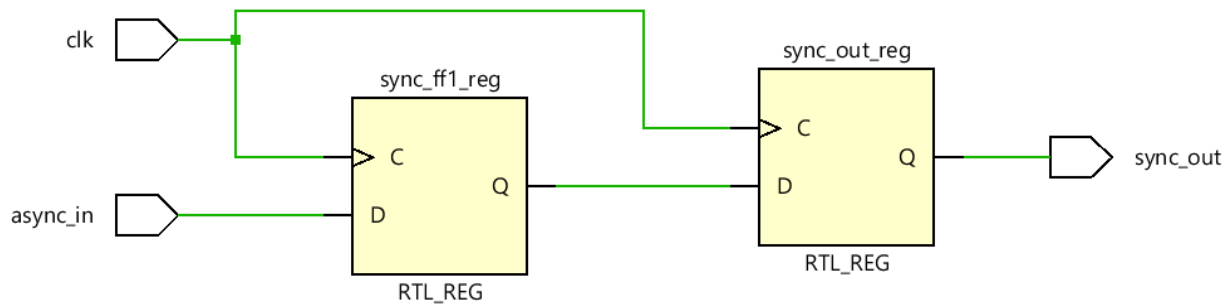


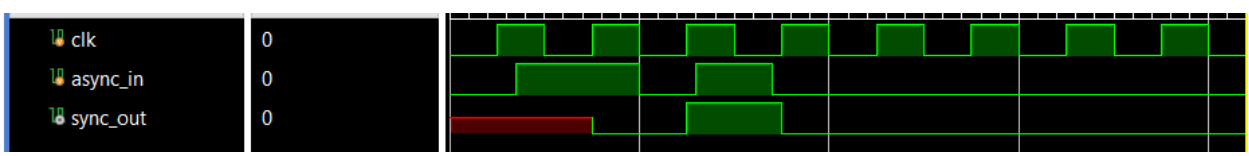


2-FLIPFLOP SYNCHRONISER

```
module two_ff_synchronizer (
input  wire clk,
input  wire async_in,
output reg  sync_out);
reg sync_ff1;
always @(posedge clk) begin
sync_ff1 <= async_in;
sync_out <= sync_ff1;
end
endmodule
```

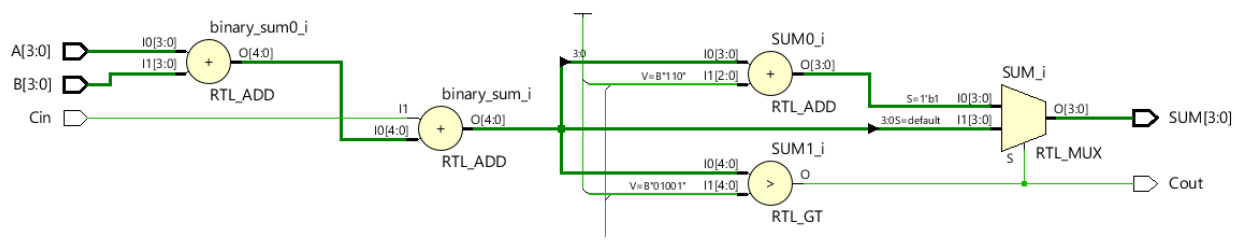
SCHEMATIC

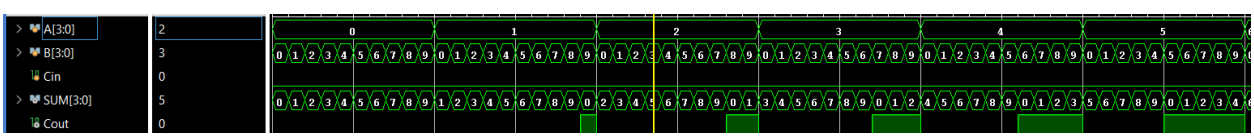




BCD ADDER

```
module bcd_adder(
input[3:0] A,
input [3:0] B,
input Cin,
output reg [3:0] SUM,
output reg Cout);
wire [4:0] binary_sum;
always @(*)
begin
if (binary_sum > 9)
begin
SUM  = binary_sum + 6;
Cout = 1;
end
else
begin
SUM  = binary_sum[3:0];
Cout = 0;
end
end
assign binary_sum = A + B + Cin;
endmodule
```

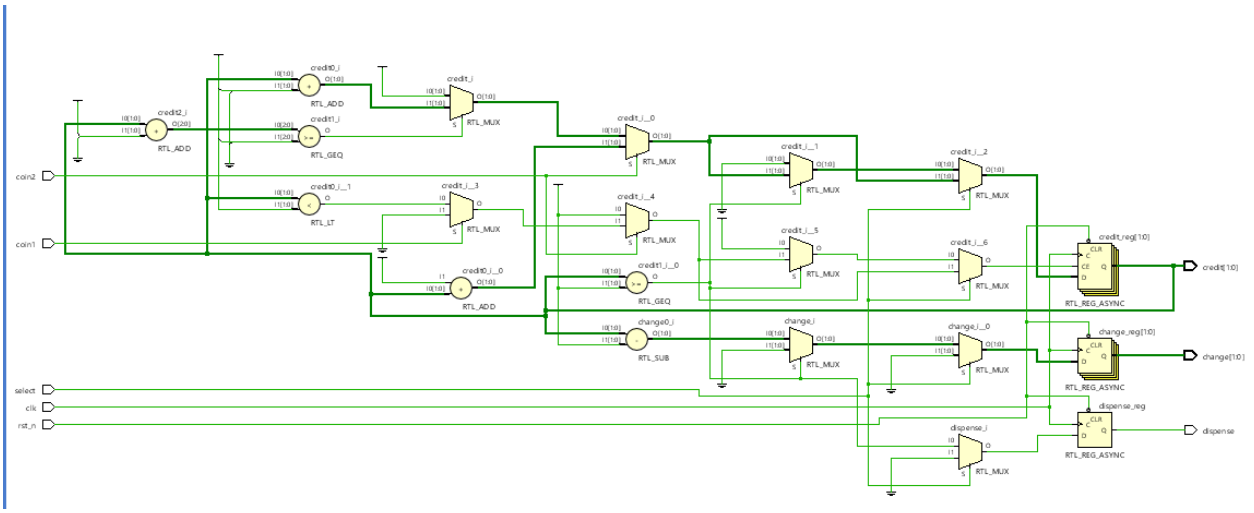


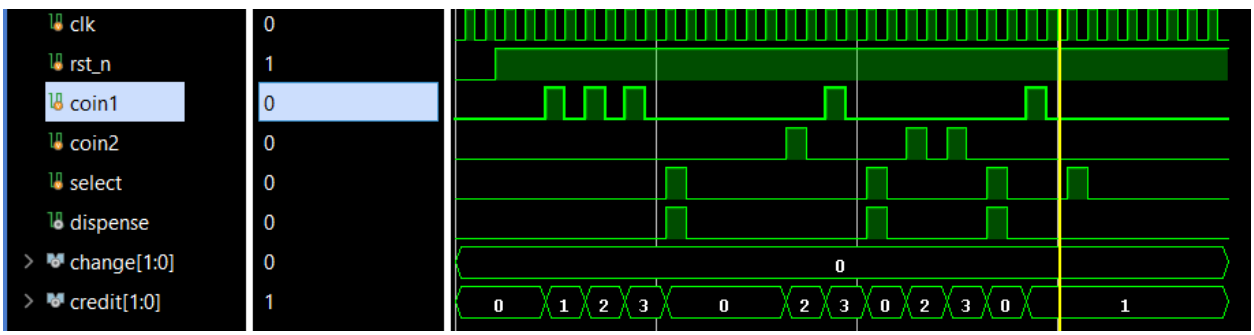


CHOCOLATE VENDING MACHINE

```
module chocolate_vend(input wire clk,input wire rst_n,input wire coin1,input wire coin2,input wire select, output reg dispense,output reg [1:0] change,output reg [1:0] credit );
localparam COST = 3;
always @(posedge clk or negedge rst_n)
begin
if (!rst_n)
begin
credit <= 2'b00;
dispense <= 1'b0;
change <= 2'b00;
end
else
begin
dispense <= 1'b0;
change <= 2'b00;
if (coin1)
begin
if (credit < 3)
credit <= credit + 1;
end
if (coin2)
begin
if (credit + 2 >= 3)
credit <= 3;
else
credit <= credit + 2;
end
if (select)
begin
if (credit >= COST)
begin
dispense <= 1'b1;
change <= credit - COST;
credit <= 2'b00;
end
```

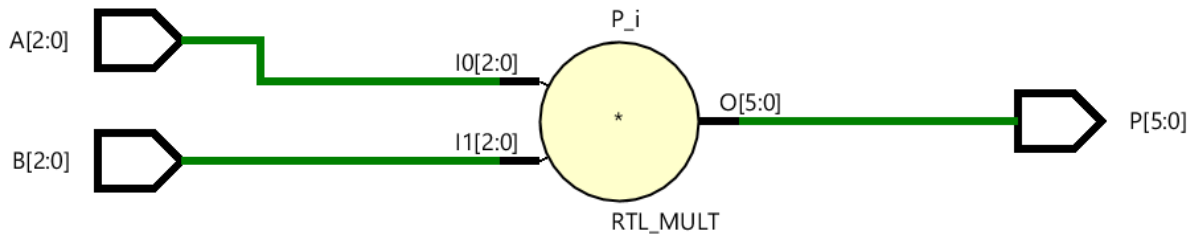

```
if (credit >= COST)
begin
dispense <= 1'b1;
change <= credit - COST;
credit <= 2'b00;
end
else
begin
dispense <= 1'b0;
change <= 2'b00;
end
end
end
end
endmodule
```

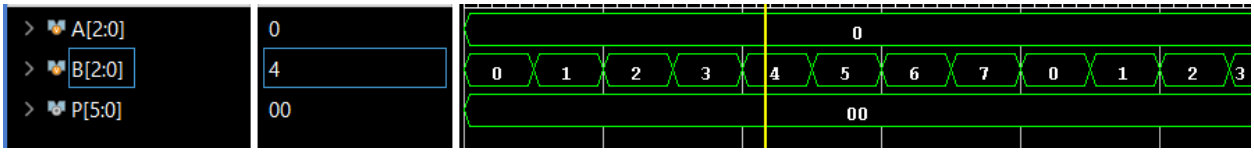




MULTIPLIER

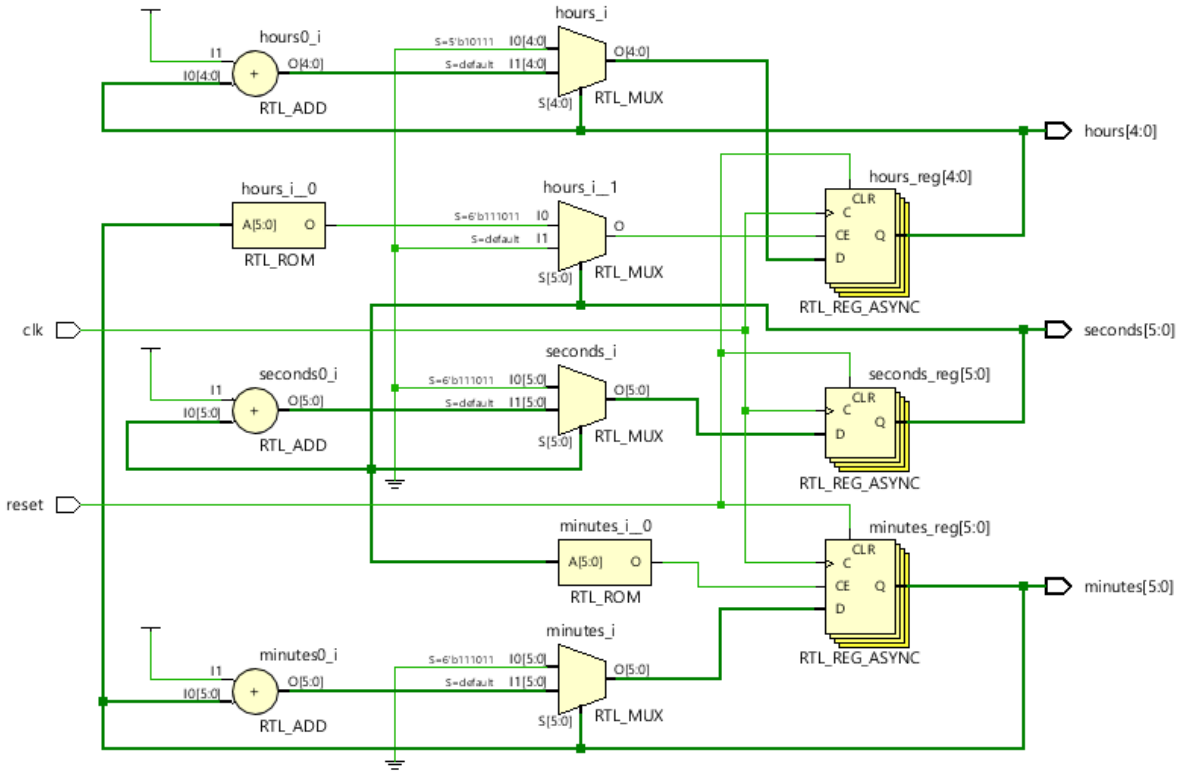
```
module binary_multiplier_3bit (
    input  [2:0] A,
    input  [2:0] B,
    output [5:0] P
);
    assign P = A * B;
    endmodule
```

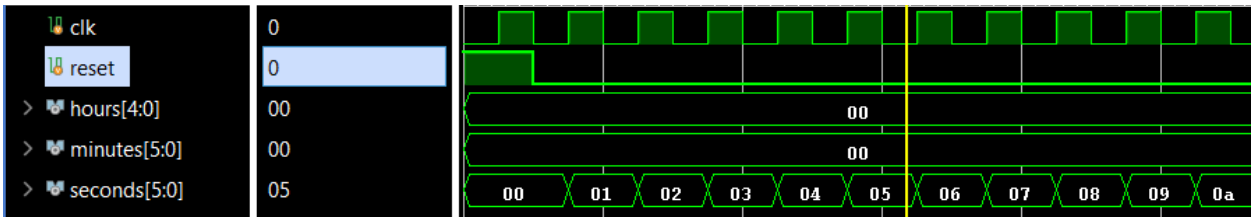


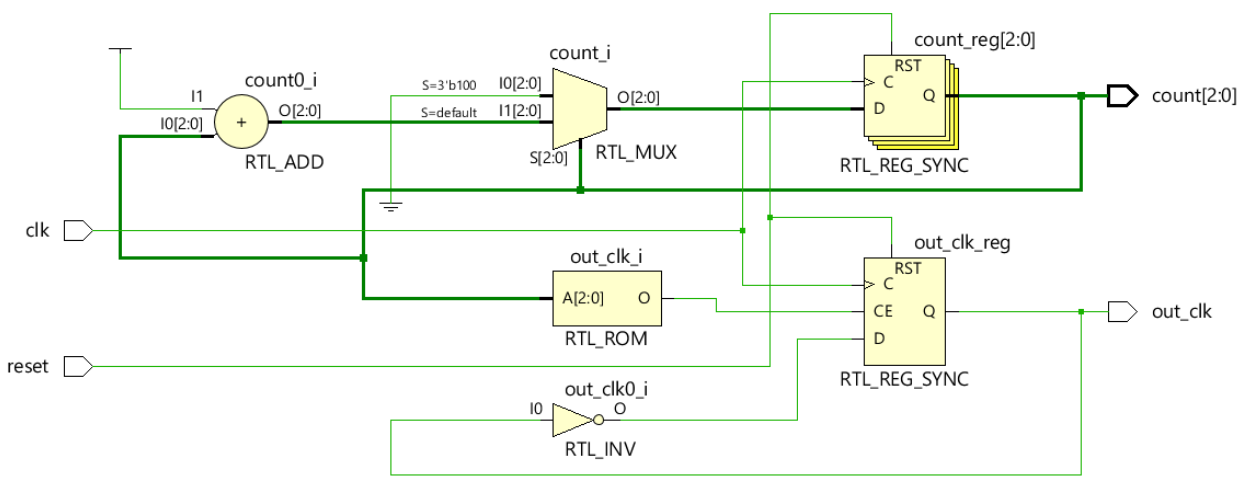
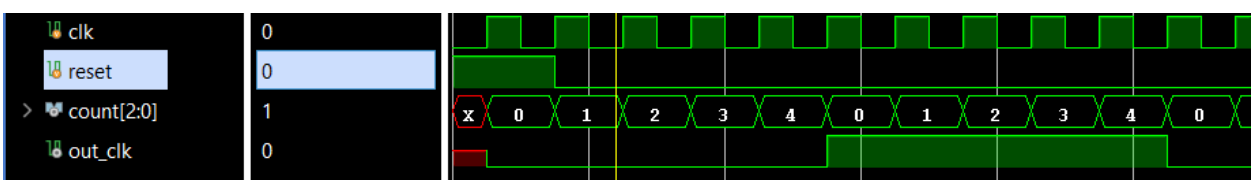


24 HOUR TIMER

```
module timer_24hr(input clk, input reset, output reg [4:0] hours,output reg [5:0] minutes,output reg [5:0] seconds);

always @(posedge clk or posedge reset)
 begin
if (reset)
begin
hours <= 0;
minutes <= 0;
seconds <= 0;
end
else
begin
if (seconds == 59)
begin
seconds <= 0;
if (minutes == 59)
begin
minutes <= 0;
if (hours == 23)
hours <= 0;
else
hours <= hours + 1;
end else
begin
minutes <= minutes + 1;
end
end else begin
seconds <= seconds + 1;
end
end
end
endmodule
```





MOD 5 COUNTER 50 DUTY CYCLE

```
module mod5_upcounter_50duty (input clk,input reset,output reg [2:0] count,output reg  out_clk);
always @(posedge clk)
begin
if (reset)
begin
count   <= 3'd0;
out_clk <= 1'b0;
end
else
begin
if (count == 3'd4)
begin
count   <= 3'd0;
out_clk <= ~out_clk;
end
else
begin
count <= count + 1;
end
end
end
endmodule
```